FAKULTET ELEKTROTEHNIKE
I RAČUNARSTVA

Zavod za elektroničke sustave
i obradbu informacija

PROJEKTIRANJE UGRADBENIH RAČUNALNIH SUSTAVA
Sustavi na čipu

Materijali za predavanja

Mladen Vučić

Zagreb 2022.

Sadržaj

1 Uvod ... 1

2 Računalni sustavi ... 2
2.1 Izvedbe računalnih sustava ... 2
2.2 Izvedbe procesora ... 3
2.3 ARM procesori ... 4
2.3.1 Arhitektura, meka jezgra i njena implementacija ... 4
2.3.2 ARM Arhitektura ... 5
2.3.2.1 Podaci ... 6
2.3.2.2 Instrukcije ... 7
2.3.3 ARM procesorska jezgra ... 8
2.4 Implementacija računalnih sustava ... 9
2.5 Pitanja za provjeru znanja ... 11

3 Sustav na čipu Zynq-7000 ... 12
3.1 Osnovna blokovska shema ... 12
3.2 Procesorski sustav ... 15
3.2.1 Procesorska jedinica ... 15
3.2.1.1 Cortex-A9 procesor ... 16
3.2.1.2 Ugrađena memorija (OCM) ... 32
3.2.1.3 L2 priručna memorija ... 32
3.2.1.4 Prekidni kontroler ... 33
3.2.2 Memorije ... 34
3.2.2.1 Izvedbe memorija ... 34
3.2.2.2 Memorijska mapa ... 35
3.2.3 Periferije i veza s vanjskim svijetom ... 36
3.3 Programabilna logika ... 38
3.3.1 Ideja programabilnog sklopovlja ... 38
3.3.1.1 Implementacija kombinacijske funkcije ... 39
3.3.1.2 Implementacija slijedne funkcije ... 40
3.3.2 Građa programabilne logike ... 41
3.3.2.1 Konfigurabilni logički blokovi ... 42
3.3.2.2 Ulazno-izlazni blokovi i vanjski priključci ... 44
3.3.2.3 Ugrađene komponente ... 45
3.3.3 Svojstva programabilne logike u sklopu Zynq ... 48
3.4 Pitanja za provjeru znanja ... 49
3.5 Zadaci za samostalan rad ... 50

4 Električko spajanje SoC sklopa ... 51
4.1 Vanjski priključci SoC sklopa ... 51
4.2 Razvojni sustavi ... 53
4.3 ZedBoard ... 54
4.4 Pitanja za provjeru znanja ... 56
4.5 Zadaci za samostalan rad ... 56

5 Dizajn temeljen na sklopovskim jezgrama ... 57
5.1 Osnovni pojmovi ... 57
5.2 Razvoj ugradbenih računalnih sustava u SoC okruženju ... 58

5.3 Vivado 59
5.3.1 Tijek razvoja sklopovlja 59
5.3.2 Načini rada 61
5.4 Razvoj sklopovlja u HDL okruženju 63
5.4.1 Izvor pulsova 64
5.4.2 Krovni model 66
5.4.3 Parametri sklopa 66
5.4.4 Ispitno okruženje 67
5.5 Simulacija 68
5.6 RTL analiza 68
5.7 Implementacija na sustavu ZedBoard 69
5.8 Kreiranje jezgri 71
5.9 Razvoj sklopovlja temeljen na jezgrama 73
5.10 Komercijalno dostupne jezgre 75
5.11 Pitanja za provjeru znanja 76
5.12 Zadaci za samostalan rad 76

6 Računalni sustav u SoC okruženju 77
6.1 Tijek razvoja računalnog sustava na čipu 77
6.2 Sklopovlje 78
6.2.1 Procesorski sustav 79
6.2.2 Upravljačko sklopovlje 80
6.2.3 GPIO 83
6.3 Programska podrška za ugradbene računalne sustave 87
6.4 Vitis 88
6.4.1 Radna okolina 88
6.4.2 Programsko okruženje 88
6.4.3 Tijek razvoja programske podrške 91
6.5 Programska podrška 92
6.5.1 Pokretački program 92
6.5.1.1 Učitavanje konfiguracije u Zynq 92
6.5.2 Osnovne inicijalizacije 93
6.5.3 Korisnikova aplikacija 95
6.6 Prevođenje, povezivanje i punjenje 95
6.7 Učitavanje konfiguracije u sklop Zynq 95
6.8 Uhodavanje programske podrške 96
6.9 Pitanja za provjeru znanja 97
6.10 Zadaci za samostalan rad 97

7 Standalone OS 98
7.1 Upravljački programi 98
7.1.1 Arhitektura upravljačkih programa 99
7.1.2 Nazivi komponenata upravljačke programske podrške 99
7.1.2.1 Naziv upravljačkog programa 100
7.1.2.2 Vanjski identifikatori 100
7.1.2.3 Nazivi datoteka 100
7.2 Sučelje upravljačkih programa 102
7.2.1 Upravljački program za GPIO procesorskog sustava 102
7.2.2 Standardna i specifična sučelja 105
7.3 Rad sa sklopom GPIO pomoću upravljačkih programa 106
7.3.1 Sklopovlje 106
7.3.2 Programska podrška 107
7.4 Hello World! 108

7.4.1 Sklopovlje ... 108
7.4.1.1 Asinkrono serijsko sučelje ... 108
7.4.1.2 Procesorski sustav ... 111
7.4.2 Programska podrška ... 112
7.4.2.1 Upravljačko sklopovlje ... 112
7.4.2.2 Upravljački program za UART ... 114
7.4.2.3 Aplikacija ... 116
7.5 Hello again! ... 117
7.5.1 Sklopovlje ... 117
7.5.2 Programska podrška ... 118
7.5.2.1 Ulazno-izlazne funkcije ... 118
7.5.2.2 Aplikacija ... 119
7.6 Pitanja za provjeru znanja ... 120
7.7 Zadaci za samostalan rad ... 120

8 Povezivanje procesorskog sustava i programabilne logike ... 121
8.1 Povezivanje PS i PL preko EMIO ... 121
8.1.1 Sklopovlje ... 122
8.1.2 Programska podrška ... 123
8.2 Povezivanje PS i PL preko AXI sučelja ... 125
8.2.1 AXI sabirnica ... 126
8.2.1.1 Osnovni pojmovi ... 126
8.2.1.2 Prijenos podataka preko AXI sabirnice ... 127
8.2.2 AXI sučelja u sklopu Zynq ... 130
8.3 Primjer spajanja sklopova na AXI sabirnicu ... 131
8.3.1 Sklopovlje ... 131
8.3.2 Upravljački program ... 133
8.3.3 Programska podrška ... 135
8.4 Pitanja za provjeru znanja ... 136
8.5 Zadaci za samostalan rad ... 136

9 Takt ... 137
9.1 Osnovni pojmovi ... 137
9.2 Takt u sklopu Zynq ... 137
9.3 Pitanja za provjeru znanja ... 139
9.4 Zadaci za samostalan rad ... 139

10 Prekidi ... 140
10.1 Globalno brojilo ... 141
10.1.1 Sklopovlje ... 141
10.1.2 Upravljački program ... 142
10.2 Konfiguriranje prekida ... 143
10.2.1 Upravljanje prekidima na razini procesora ... 144
10.2.2 Upravljanje prekidnim kontrolerom ... 145
10.3 Primjer prekida globalnog brojila ... 146
10.3.1 Sklopovlje ... 147
10.3.1.1 Procesorski sustav i vanjsko serijsko sklopovlje ... 147
10.3.1.2 Globalno brojilo ... 147
10.3.2 Programska podrška ... 148
10.4 Pitanja za provjeru znanja ... 151
10.5 Zadaci za samostalan rad ... 151

11 Periferno sklopovlje u SoC Zynq ... 152
11.1 Serial Peripheral Interface ... 152
11.2 I2C ... 155

11.3 USB 157
11.3.1 Prijenos signala linijom 157
11.3.2 Sklopovlje USB sučelja 159
11.4 Pitanja za provjeru znanja 163
12 Višeprocesorski sustav 164
12.1 Način korištenja višejezgrenih procesora u URS-ovima 164
12.1.1 Simetrični višeprocesorski sustavi 164
12.1.2 Asimetrični višeprocesorski sustavi 164
12.2 Pitanja za provjeru znanja 166
13 Završne napomene 167
14 Literatura 168

1 Uvod

- svrha ove nastavne cjeline je upoznati se s
  - arhitekturom programabilnih SoC platformi
  - razvojem sklopovlja i programske podrške računalnih sustava izvedenih na čipu
  - alatima za razvoj računalnih sustava temeljenih na SoC platformama
- predznanje
  - osnove digitalnog dizajna
  - građa računalnih sustava
  - programski jezici: C
  - jezici za opis sklopovlja: VHDL
- poveznice prema dosad odslušanim predmetima
  - Arhitektura računala, AR
  - Ugradbeni računalni sustavi, URS
  - Alati za razvoj digitalnih sustava, AZRDS
- gradivo obuhvaća sljedeće cjeline
  - uvodne napomene
  - izvedbe računalnih sustava
  - programabilni SoC, procesorski sustav, programabilna logika
  - električko spajanje SoC komponenata, razvojni sustavi
  - alati za razvoj sklopovlja
  - FPGA, razvoj sklopovlja, IP jezgre
  - alati za razvoj i uhodavanje programske podrške
  - *bare machine*
  - upravljački programi, jednostavan operacijski sustav
  - prekidi
  - povezivanje programabilne logike i procesorskog sustava
  - vanjsko i unutrašnje periferno sklopovlje
- primjeri će biti dani za
  - integrirano okruženje Xilinx SoC Zynq-7000
  - skup alata za razvoj sklopovlja i programske podrške Vivado i Vitis

2 Računalni sustavi

2.1 Izvedbe računalnih sustava

- izvedbe s aspekta korisnika
  - "samostojeći"
    - računala opće namjene, radne stanice, poslužitelji
    - orijentiran prema nekoj aplikaciji
      ⇒obrada teksta, CAD/CAM, računalne igre, pohrana podataka, rad s bazama podataka i sl.
    - složeno sučelje prema korisniku
      - prema korisniku (zaslon i tipkovnica, grafičko sučelje)
      - prema drugom računalu (kod poslužitelja)
  - ugradbeni
    - dio nekog drugog sustava
      ⇒osnovna namjena - upravljanje sustavima u koji su ugrađeni (mobilnog telefon, stroj za foto kopiranje, automobil, miš, tenisice, ..)
    - sučelje prema korisniku
      - namjensko
        - fotokopirni uređaj, upravljačka ploča automobila, sportski sat
      - često vrlo rudimentarno
        - nekoliko svjetlećih dioda, 7-segmentni pokaznik (mrežni usmjerivač, set-top box prijamnici, i sl.)
      - ponekad uopće ne postoji ne postoji
        - kočioni sustav u automobilu, samonavođeni projektil
    - posebni zahtjevi
      - male dimenzije
      - mala potrošnja
      - namjensko sklopovlje za upravljanje i obradu signala
      - pouzdanost
      - rad u stvarnom vremenu

2.2 Izvedbe procesora

- s aspekta implementacijske tehnologije
  - procesor s mekom jezgrom (*Soft Processor* ili *Soft-Core Processor*)
    - napravljen je kao formalni model korištenjem jezika za opis sklopovlja (VHDL, Verilog i sl.)
    - može se implementirati na raznim platformama (VLSI, FPGA)
    - može se konfigurirati prije same implementacije
      ⇒ npr. može se definirati veličina adresnog prostora, podatkovnog prostora, broj pojedinih periferija i sl.)
  - fiksno ožičeni odnosno implementirani procesori (*Hard-Core Processor* ili *Hard-Wired Processor*)
    - fizička komponenta implementirana u VLSI tehnologiji
- s aspekta primjene
  - procesori namijenjeni širem rasponu aplikacija (*Application Processors*)
    - koriste se u sustavima koji sadrže veći broj različitih aplikacija
    - podržavaju rad sa složenim operacijskim sustavima
  - ugradbeni procesori (*Embedded Processors*)
    - najčešće se koriste u specifičnoj aplikaciji
      - ponekad te aplikacije zahtijevaju rad u stvarnom vremenu (*Real Time*)
    - obično imaju manje mogućnosti
    - često se izvode s malom potrošnjom, malim naponom napajanja i sl.
  - procesori opće namjene (*General Purpose Processors*)
    - koriste se u sustavima koji sadrže veći broj različitih aplikacija
    - po složenosti kompromis između gornja dva tipa
- procesorska jedinica
  - procesor zajedno sa svojim neposrednim sklopovskim okruženjem (priručnom memorijom, prekidnim kontrolerom i dr.)
- više procesora mogu se objediniti u jednu procesorsku jedinicu
  ⇒ govorimo o višejezgrenoj procesorskoj jedinici

2.3 ARM procesori

- ARM procesori
  - ubrajaju se među najpoznatije i najviše korištene procesore s mekom jezgrom
- meka jezgre proizvodi kompanija Arm Ltd.
- jezgre licenciraju i implementiraju razne kompanije
  - STMicroelectronics ⇒ vidi URS
  - Xilinx ⇒ vidi daljnji tekst

2.3.1 Arhitektura, meka jezgra i njena implementacija

- potrebno je razlikovati pojmove
  - arhitektura
    - ⇒ funkcijska specifikacija procesora
      - ⇒ skup instrukcija, skup registara, memorijski model, model iznimki i posebne funkcije (uhodavanje, i sl.)
    - ⇒ u osnovi, arhitektura predstavlja ponašajni (*behavioral*) model procesora
  - mikroarhitektura
    - način na koji neki procesor implementira arhitekturu
      - uključuje dubinu protočnog sklopovlja, broj ciklusa po instrukciji, veličinu priručne memorije i sl.
  - procesorska jezgra
    - ⇒ procesor s mekom jezgrom koji implementira neku inačicu arhitekture
      - ⇒ predstavlja mikroarhitekturu
    - ponekad se procesorska jezgra naziva kraće - procesor
  - (mikro)procesor ili (mikro)kontroler u užem smislu riječi
    - ⇒ fizička implementacija procesora s mekom jezgrom
    - ⇒ kod mikrokontrolera implementacija uključuje i dodatne komponente

2.3.2 ARM Arhitektura

- označavanje arhitektura
  - inačica

    ARMv1, ... ARMv7 (32-bitne arhitekture)
    ARMv8, ARMv8.1, ... (32-bitne i 64-bitne arhitekture)
    ARMv9 (64-bitna arhitekture)
  - jednog ili više slova koja govore podržava li arhitektura

    T *Thumb* skup instrukcija
    T2 *Thumb 2* proširenje Thumb skupa instrukcija
    E *Enhanced DSP Extension*
    instrukcije karakteristične za digitalnu obradu signala
    J *Jazelle DBX* (*Direct Bytecode eXecution*)
    izravno izvršavanje dijela instrukcija Java bajt koda
    i sl.
  - profila arhitekture

    -A *Application profile*
    -R *Real-time profile*
    -M *Microcontroller profile*

- profili arhitektura
  - govore za kakvu primjenu je arhitektura oblikovana
  - starije arhitekture nisu bile posebno optimirane za pojedine primjene

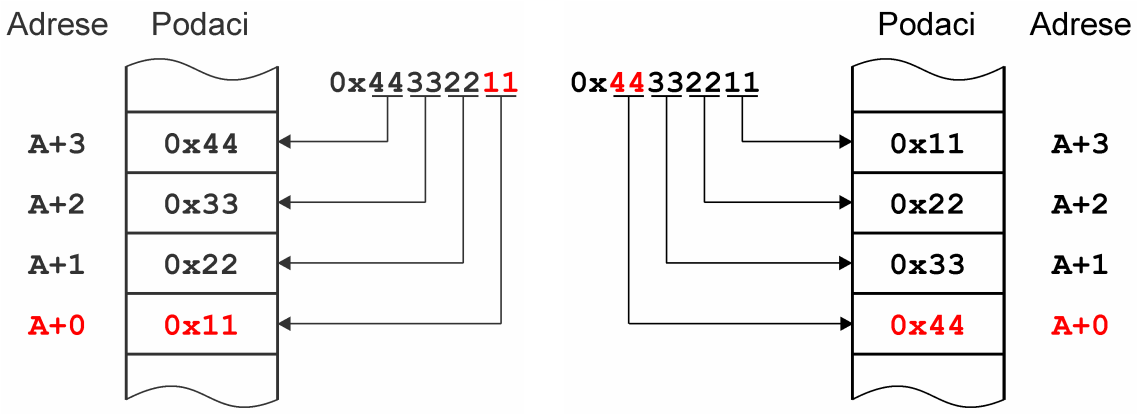
    $\Rightarrow$ sadržavale su većinu mogućnosti koje je omogućavalo trenutno stanje razvoja implementacijske tehnologije
  - novije arhitekture (ARMv7 i ARMv8) sadrže sljedeće profile
    - Application profile, -A
      - visoke performance
      - pogodan za implementaciju složenih operacijskih sustava
    - Real-time profile, -R
      - prilagođen aplikacijama koje traže rad u stvarnom vremenu
      - nalazimo ga u mrežnoj opremi
    - Microcontroller profile, -M
      - manji, energetski učinkovit
      - pogodan za mikrokontrolere

2.3.2.1 Podaci

- veličine podataka u ARM arhitekturama
  - 8 bitova, bajt (*Byte*)
  - 16 bitova, poluriječ (*Halfword*)
  - 32 bita, riječ (*Word*)
  - 64 bita, dvostruka riječ (*Doubleword*)

- podaci u memoriji su poravnati (a*lignment*)
  - 32 bitni podatak
    ⇒ zauzima 4 bajta počevši od adrese koja je višekratnik broja 4
    ⇒ početne adrese: 0, 4, 8, 12, 16, ...
  - 16 bitni podatak
    ⇒ zauzima 2 bajta počevši od parne adrese
    ⇒ početne adrese: 0, 2, 4, 6, ...
  - 8 bitni podatak
    ⇒ zauzima 1 bajta i može stajati na bilo kojoj adresi

- većina arhitektura podržava oba zapisa podataka u memoriji
  - *little endian*
  - *big endian*



- pojedine implementacije podržavaju samo jedan tip zapisa
  ⇒ odabire se kod implementacije jezgre

2.3.2.2 Instrukcije

- 32-bitne arhitekture podržavaju tri skupa instrukcija
  - ARM skup instrukcija (naziva se i A32)
    - operacijski kod je 32 bitni
    - instrukcije su u memoriji poravnate na 4 bajta
  - Thumb skup instrukcija (naziva se i T32)
    - operacijski kod je 16-bitni ili 32-bitni (varijabilna širina koda)
    - instrukcije su u memoriji poravnate na 2 bajta
  - Jazelle DBX (*Direct Bytecode eXecution*)
    - operacijski kod je 8 bitni
    - podržava jednostavnije instrukcije Java Bytecode-a (JBC)
      ⇒ Java virtualna mašina (*Java Virtual Machine*, JVM) izvršavat će dio instrukcija JBC-a sklopovski tj. kao asemblerske instrukcije procesora
- 64-bitne arhitekture podržavaju tri skupa instrukcija
  - A64 (novi skup 32-bitnih instrukcija)
  - A32, T32
  - Jazelle DBX
- postoje tri varijante Thumb skupa instrukcija
  - Thumb 1
    - sadrži 16-bitne instrukcije
    - koristio se u starijim arhitekturama
  - Thumb 2
    - u Thumb skup dodano je još instrukcija (uglavnom 32-bitnih)
      ⇒ sadrži 16- i 32-bitne instrukcije
      ⇒ varijabilna duljina operacijskog koda
  - ThumbEE (*Thumb Execution Environment*)
    - malo izmijenjen skup Thumb 2
    - prilagođen dinamički generiranom kodu (kod koji nastaje prevođenjem neposredno prije izvođenja, npr. prevođenjem JBC-a)
- različiti profili podržavaju samo različite skupove instrukcija
  - primjer
    - M profil podržava samo T32 skup instr. ⇒ vidi URS
    - A profil podržava sve skupove instrukcija ⇒ vidi daljnji tekst

2.3.3 ARM procesorska jezgra

- označavanje mekih jezgara
  - starije jezgre sadrže
    - oznaka serije (familije)
      ARM6, ARM7, ARM9, ARM11 i sl.
  - novije jezgre sadrže
    - oznaku serije
      Cortex-A (*Applications Processing for Mobile Devices*)
      Cortex-R (*Real-Time Processing*)
      Cortex-M (*Microcontroller*, *Energy efficient*)
    - broj koji pobliže označava izvedbu
      - primjer
        Cortex-M4 ⇒ vidi URS
        Cortex-A9 ⇒ vidi daljnji tekst
  - brojka u oznaci nije nužno vezana uz arhitekturu koju jezgra implementira

- meke jezgre mogu imati različite implementacije za istu ARM arhitekturu
  - primjer - arhitektura ARMv4T (AR 1)
    - procesorska jezgra ARM7TDMI (AR 1)
      - Von Neuman
      - trostupanjsko protočnom sklopovlje (*pipeline*)
    - procesorska jezgra ARM920T
      - Harvard
      - peterostupanjsko protočno sklopovlje
  - uočiti
    - ovakve jezgre su kompatibilne s aspekta programske podrške
    - ARMv4T je arhitektura, a spomenute jezgre predstavljaju dvije različite mikroarhitekture

2.4 Implementacija računalnih sustava

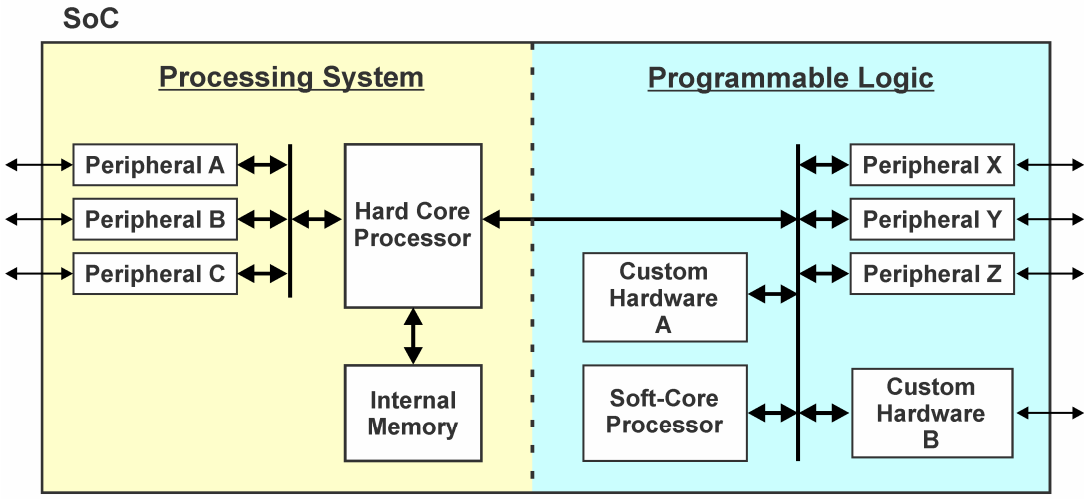
- izvedbe s aspekta implementacijske tehnologije
    - sustav na tiskanoj pločici (*System on Board*, SoB)
        - klasični dizajn računalnih sustava
        - tehnologija spajanja komponenata: PCB
        - URS, PURS, PPIUS
    - sustav na čipu (*System on Chip*, SoC)
        - noviji pristup, u porastu
        - tehnologija spajanja komponenata: FPGA, VLSI (ASIC)
        - URS, AZRDS, PURS

- fiksno ožičene komponente
    - razvoj
        - bipolarni tranzistor
        - pojedini logički sklopovi (familije 74xx, 40xx)
        - pojedini dijelovi procesora
        - procesori (Z80, 68HC11)
        - mikrokontroleri (8051, STM32F4)
    - uočiti
        - mikrokontroleri sadrže sve elemente računalnog sustava
          $\Rightarrow$ predstavljaju sustave na čipu
        - korisnik ni na jednoj od ovih komponenata ne može mijenjati sklopovlje

- programabilne komponente
    - razvoj
        - PAL (*Programmable Array Logic*), GAL (*Generic Array Logic*)
        - CPLD (*Complex Programmable Logic Device*) $\Rightarrow$ URS
        - FPGA (*Field Programmable Gate Array*) $\Rightarrow$ AZRDS, URS
    - uočiti
        - u FPGA korisnik može realizirati cijeli računalni sustav
        - sklopovlje se može mijenjani neograničen broj puta

$\Rightarrow$ sustavi na čipu kao komponente
- objedinjuju fiksno ožičene i programabilne komponenata
  $\Rightarrow$ koriste dobre strane jednih i drugih

- princip dizajna temeljen na SoC sklopovima



- uočiti
  - procesori i standardne periferije su na raspolaganju kao fiksno ožičene
  - programabilna logika omogućava vlastiti razvoj namjenskog sklopovlja
    $\Rightarrow$ po želji, u PL se može dodati jedan ili više procesora
- proizvođači SoC sklopova
  - Xilinx (Xilinx/AMD)
  - Altera (Altera/Intel)

- u sljedećim poglavljima kao primjer će biti obrađeni SoC sklopovi familije Zynq 7000, proizvođača Xilinx Inc.
  - sklopovi familije Zynq 7000 u sebi sadrže
    - procesorski sustav
      - procesorski sustav sadrži procesorsku jedinicu *Cortex A9 MPCore*
        - procesorska jedinica *Cortex A9 MPCore* implementira arhitekturu ARMv7-A
    - programabilnu logiku
      - temelji se na FPGA familijama Artix 7 i Kintex 7

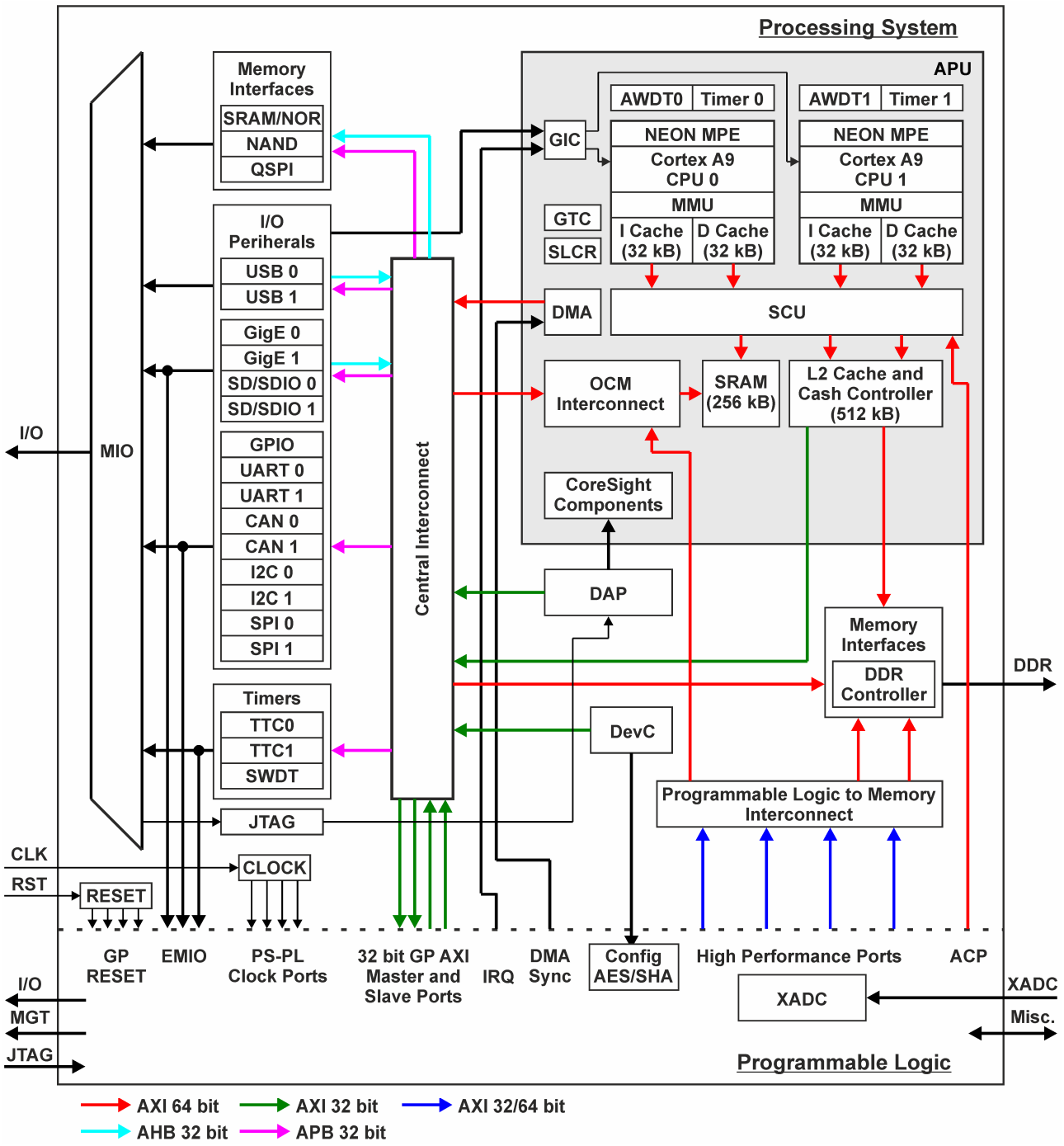
- razlikovati **SoC** i **SoC** :)
  - prvo je dizajn u kojem sve komponente objedinjujemo na čipu
  - drugo je komponenta koja se sastoji od PS i PL

2.5 Pitanja za provjeru znanja

1. Objasniti razliku između ugradbenog računalnog sustava i računalnog sustava opće namjene.
2. Kakve izvedbe procesora postoje s obzirom na implementacijske tehnologije? Kakve vrste procesora razlikujemo s obzirom na područje primjene?
3. Objasniti pojmove: arhitektura, mikroarhitektura, procesorska jezgra, procesor.
4. Opisati način označavanja ARM arhitektura. O čemu govore profili arhitektura?
5. Koje tipove podataka susrećemo u ARM arhitekturama s obzirom na veličinu i način zapisa u memoriji? Kako ovi podaci moraju biti poravnati?
6. Nabrojiti i opisati skupove instrukcija koje podržavaju ARM arhitekture. Opisati varijante *Thumb* skupa instrukcija.
7. Objasniti pojmove *System on Board Design* i *System on Chip Design*? Objasniti razliku između SoC tehnologije i SoC sklopova koji se temelje na programabilnim logičkim poljima.

3 Sustav na čipu Zynq-7000

3.1 Osnovna blokovska shema



- dva podsustava
  - procesorski sustav (*Processing System*, PS) ⇒ fiksno ožičen
  - programabilna logika (*Programmable Logic*, PL) ⇒ FPGA

- uočiti komponente sustava
    - procesorski sustav - *Processing System*, PL
        - APU - *Application Processor Unit*
            - Cortex A9 Processor
                - NEON MPE - NEON *Media Processing Engine*
                - CPU - *Central Processing Unit*
                - MMU - *Memory Management Unit*
                - I *Cache*, D *Cache*
            - SCU - *Snoop Control Unit*
            - OCM (*On-Chip-Memory*) *Interconnect and* SRAM
            - L2 *Cache Memory and Controller*
            - GIC - *Generic Interrupt Controller*
            - DMA - *Direct Memory Access*
            - SLCR - *System Level Control Registers*
            - brojila i sklopovlje za nadzor rada (privatna za svaki CPU)
                - AWDT - *private processor's Watch-Dog*
                - *Timer - private processor's timer*
        - brojila i sklopovlje za nadzor rada (na razini sustava)
            - SWDT - *System Watch-Dog Timer*
            - TTC - *Triple Timer Counters*
        - memorijska sučelja - *Memory Interfaces*
            - za statičke memorije
                - SRAM, NOR *Flash*, NAND *Flash*
            - za brze serijske flash memorije
                - QSPI - *Quad* SPI
            - za dinamičke memorije
                - DDR - *Double Data Rate Memory Synchronous DRAM*
        - periferije - I/O *Peripherals*
            - brze
                - USB - *Universal Serial Bus*
                - GigE - *Gigabit Ethernet*
                - SD/SDIO - *Secure Digital / Secure Digital Input Output*
            - klasične
                - GPIO - *General Purpose Input Output*
                - UART - *Universal Asynchronous Receiver/Transmitter*
                - CAN - *Controller Area Network*
                - I2C - *Inter Integrated Circuits*
                - SPI - *Serial Peripheral Interface*
        - MIO - *Multiplexed* I/O
        - sklopovlje za inicijalizaciju i uhodavanje
            - DAP - *Debug Access Port*
            - DevC - *Device Configuration interface*
        - RESET, CLOCK
    - programabilna logika - *Programmable Logic*, PL
        - Config AES/SHA - *chip configuration encrypted with* AES/SHA
        - XADC - *Xilinx Analog to Digital Converter*

- uočiti način povezivanja komponenata sustava
    - prospoji
        - središnji prospoj - *Central Interconnect*
        - prospoj za pristup OCM SRAM-u (OCM *Interconnect*
        - prospoj između programabilne logike i memorije - *Programmable-Logic-to-Memory Interconnect*
    - sabirnice
        - AXI - *Advanced eXtensible Interface*
        - AHB - *Advanced High-performance Bus*
        - APB - *Advanced Peripheral Bus*

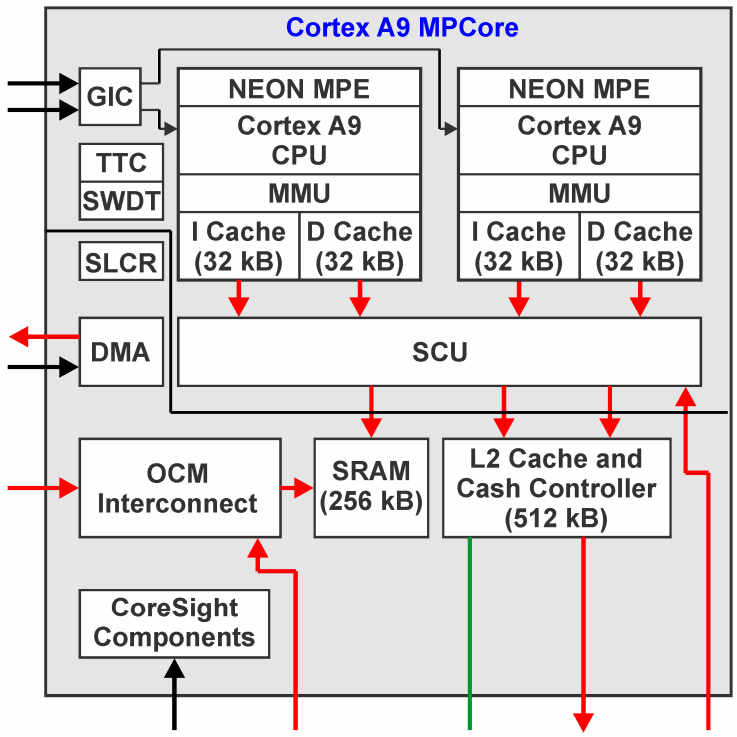
- sustav na čipu
    - procesorski sustav $\Rightarrow$ cjelovit konfigurabilan računalni sustav
    - programabilna logika $\Rightarrow$ namijenjena dizajnu namjenskih sklopova

- Zynq je složen sklop
    - proizvodi ga Xilinx
    - mnoge IP jezgre razvila je "treća strana" (ne Xilinx niti njegovi kupci)
        - Xilinx $\Rightarrow$ PL, integracija cijelog sustava
        - ARM $\Rightarrow$ CPU, GIC, L2 cache, SRAM, i DMA kontroler prospojnu logiku (NIC)
        - Synopsis $\Rightarrow$ DDR kontroler, USB kontroler
        - Cadance $\Rightarrow$ WDT, I2C, SPI, UART, TTC, GigE
        - Arasan $\Rightarrow$ AHB *Host kontroler*

- izvedbena tehnologija
    - 28 nm

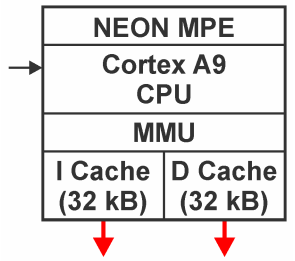
3.2 Procesorski sustav

3.2.1 Procesorska jedinica



- procesorska jedinica (*Application Processor Unit*, APU)
  - Cortex-A9 višeprocesorska jedinica (*Cortex A9 MPCore*)
    - 1 do 4 Cortex A9 procesora
    - zajednički SWDT i dva TTC-a
    - jedinica za upravljanje tokom podataka, SCU
    - prekidni kontroler, GIC
  - ugrađena memorija (OCM) i njeno upravljačko sklopovlje
  - L2 priručna memorija i njeno upravljačko sklopovlje
  - sklopovlje za izravan pristup memoriji, DMA
  - sklopovlje za uhodavanje i nadzor rada
  - upravljački registri sustava, SLCR
- APU-i sklopovima familije Zynq-7000 mogu imati jedan ili dva procesora
  - jednojezgrene (*single core*) procesorske jedinice
    ⇒Z-7007S, Z-7012S, Z-7014S
  - dvojezgreni (*dual core*) procesorske jedinice
    ⇒Z-7010, Z-7015, Z-7020, ...

3.2.1.1 Cortex-A9 procesor



Arhitektura i mikroarhitektura

- Cortex-A9 procesor (*Cortex A9*)
  - implementira arhitekturu ARMv7-A
- Kako izgleda implementacija (mikroarhitektura)?
  - Harvard
    - uočiti zasebne priručne memorije za kod i za podatke
      - svaka s vanjskim svijetom komunicira preko 64-bitnog AXI sučelja
  - 8-stupanjska protočnost s promjenjivom duljinom izvršavanja
  - superskalarna arhitektura
    - obrađuje dvije instrukcije po ciklusu
    - obrađuje ih neovisno o redoslijedu
      - ⇒ to se naziva dinamičko izvršavanje instrukcija (*dynamic execution*, *out-of-order execution*)
        - instrukcije se izvršavaju tako da se popuni prazna mjesta u ciklusima, a ne redoslijedom kojim se pojavljuju u programu
    - sadrži 4 paralelna lanca u protočnom sklopovlju za obradu
      - sadrži dvostruko sklopovlje za aritmetiku i *load/store*
  - koristi predčitanje operacijskog koda
    - čita unaprijed do 4 linije iz instrukcijske priručne memorije
    - prosljeđuje dekoderu 2 do 4 instrukcije
      - ⇒ ako proslijedi 4, tako omogućuje da se predčitaju nove instrukcije
  - sadrži statičko i dinamičko predviđanje grananja
  - podržava
    - virtualizaciju memorije
      - uočiti jedinicu za upravljanje memorijom (MMU)
    - aritmetiku u pomičnom zarezu i vektorsku aritmetiku
      - uočiti NEON koprocesor

Primjer

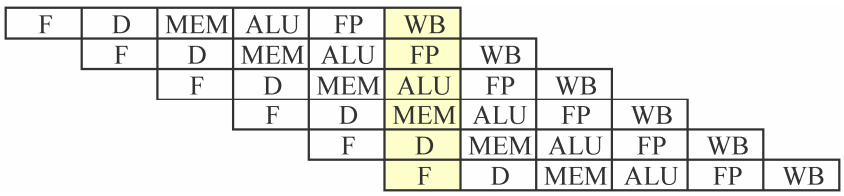
Opisati superskalarnu arhitekturu i objasniti princip promjenjive duljine protočnosti.

- pretpostavimo da instrukcije procesora obavljaju
  - dohvat operacijskog koda (*Fetch*, F)
  - dekodiranje (*Decode*, D)
  - izvršavanje (*Execute*, EXE) koje može biti
    - aritmetička ili logička operacija (ALU)
    - pristup memoriji (MEM)
    - aritmetika u pomičnom zarezu (*Floating point*, FP)
  - pohrana rezultata (*Write Back*, WB)

  ⇒izvršavanje instrukcija sastoji se od sljedećih ciklusa

F	D	MEM	ALU	FP	WB

- skalarni procesor s klasičnim protočnim sklopovljem



  - uočiti
    - sve instrukcije traju jednak broj ciklusa
    - izvršava se jedna instrukcija po ciklusu (periodu takta)

      ⇒**SI**SD ili **SI**MD
    - izvodi se samo jedna operacije (MEM, ALU i FP)

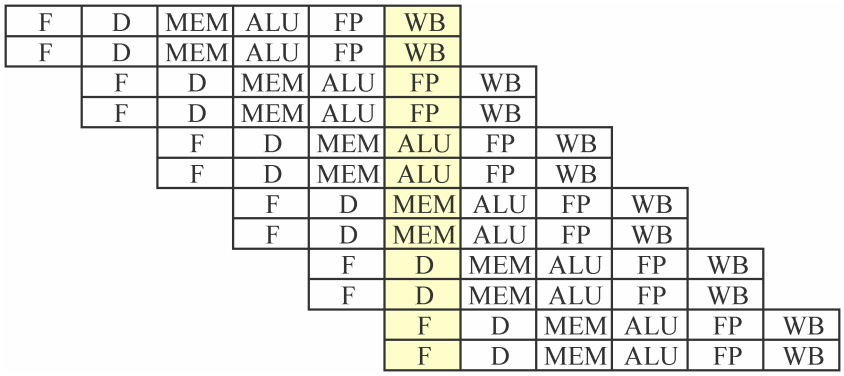
      ⇒dva ciklusa ostaju neiskorištena
    - posljedice duljeg protočnog sklopovlja

      ⇒veća latencija

      ⇒veći mjehurići kod neuspješno predviđenog grananja
  - ideja
    - obraditi paralelno dvije instrukcije

      ⇒superskalarni procesor

- superskalarni procesor s klasičnim protočnim sklopovljem



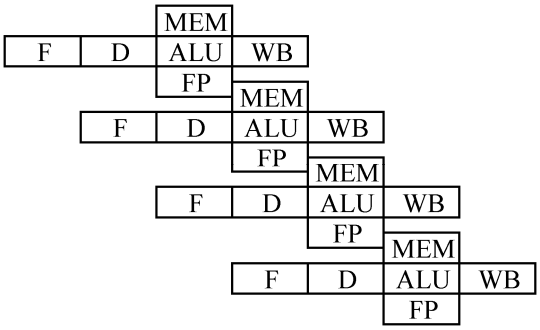
  - uočiti
    - dvije instrukcije se obrađuju istovremeno
      ⇒jedna instrukcija ne smiju ovisiti o rezultatu one druge
      ⇒moraju postojati dva sklopa F i dva D
      ⇒može postojati samo po jedan MEM, ALU i FP sklop
        - npr. jedna koristi ALU, a druga FP
  - ideja
    - diverzificirano protočno sklopovlje

- diverzificirano protočno sklopovljem
  - sklopovi za operacije MEM, ALU i FP mogu se izvesti paralelno
    ⇒operacija se dodjeljuje (*dispatch*) sklopu koji je u tom trenutku potreban



  - uočiti
    - izvršavanje je sada dva ciklusa kraće
    - izvršava jednu instrukciju po jednom ciklusu (periodu takta)
      ⇒**SI**SD ili **SI**MD

- skalarni procesor s diverzificiranim protočnim sklopovljem

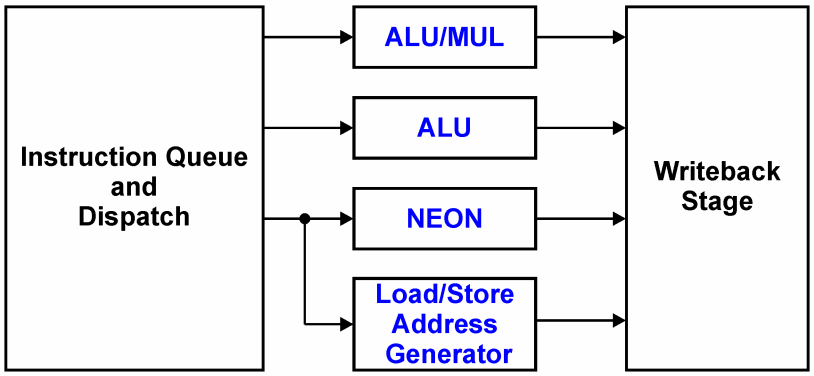


- diverzificirano protočno sklopovljem s varijabilnom duljinom izvršavanja
  - neki sklopovi su složeni
    ⇒ npr. FP treba imati dva stupnja



- primjer - Cortex A9 CPU
  - superskalar ⇒ izvršava paralelno 2 instrukcije
  - – što ako dođu dvije instrukcije koje rade istu operaciju
    ⇒ udvostručiti često korištene jedinice

  ⇒ paralelni lanci u protočnom sklopovlju

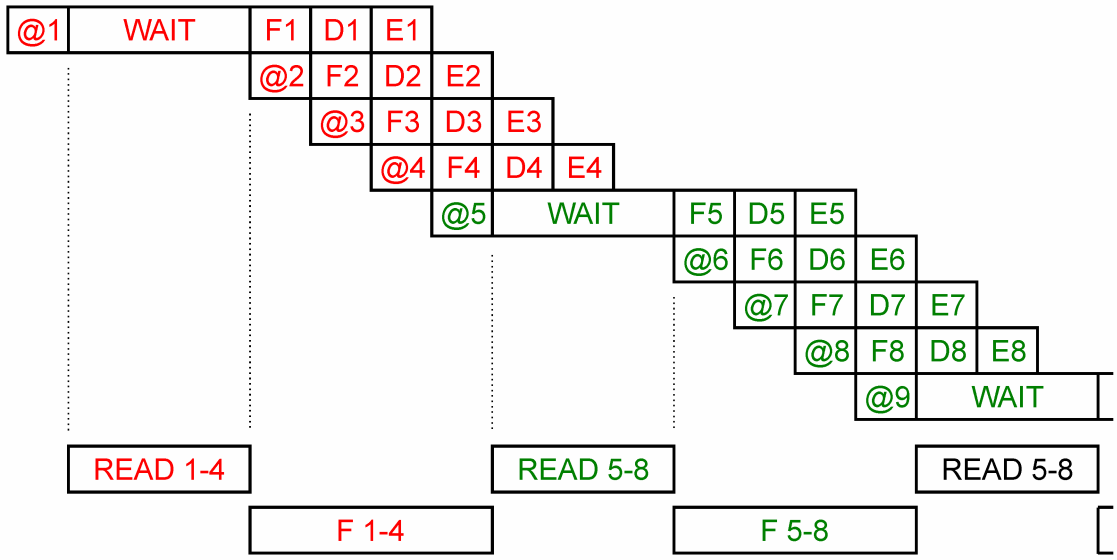


- razlikovati superskalarni procesor od vektorskog procesora
  - vektorski procesor istovremeno se izvršava polje instrukcija
    ⇒ **MI**MD
  - superskalarni procesor raspodjeljuje posao na slobodne komponente
    ⇒ tretira ga se kao **SI**SD ili **SI**MD

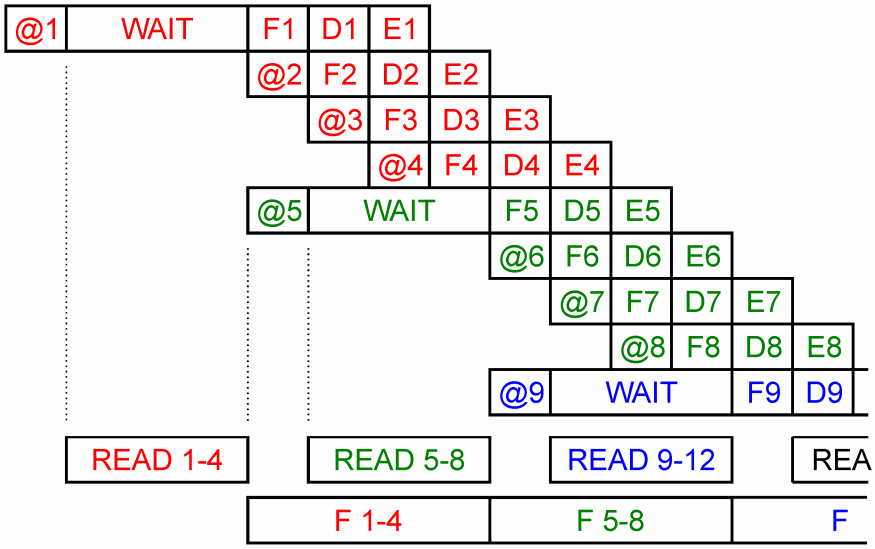
Primjer

Objasniti na primjeru što je predčitanje operacijskog koda.

- dohvat operacijskog koda bez predčitanja



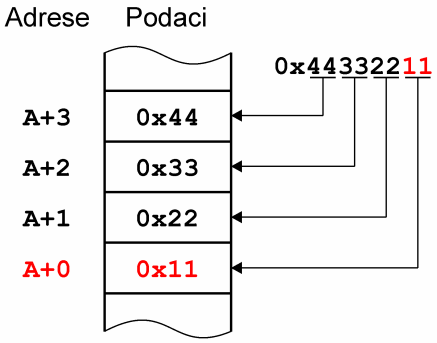
- dohvat operacijskog koda s predčitanjem



Podaci

- jezgra radi s podacima širine
  - 8 bitova, bajt (*Byte*)
  - 16 bitova, poluriječ (*Halfword*)
  - 32 bita, riječ (*Word*)
  - 64 bita, dvostruka riječ (*Doubleword*)

- zapis podataka u memoriji
  - jezgra Cortex-A9
    - podržava *little* i *big endian*
  - – APU u Zynq sklopovima podržava isključivo *little endian*



Instrukcije i instrukcijska stanja

- procesor podržava
  - ARM skup instrukcija (32-bitne instrukcije)
  - Thumb skup instrukcija (16 i 32 bitne instrukcije)
  - ThumbEE skup instrukcija (16 i 32 bitne instrukcije)
  - Java byte code (8 bitne instrukcije)

⇒procesor može biti (raditi) u jednom od sljedećih instrukcijskih stanja
  - ARM
  - Thumb
  - ThumbEE
  - Jazelle

- razlikujemo i stanja procesora vezana uz uhodavanje
  - Debug
    - procesor se zaustavlja (*halt*) zbog uhodavanja (*debugging*)
  - Non-debug
    - normalan način rada (izvršavanja instrukcija)

- S kojim registrima rade ove instrukcije?
  ⇒vidi daljnji tekst

Razine privilegija i načini rada procesora

- program koji se izvršava može imati jedu od sljedećih privilegija
  - PL0 (*Privilege Level 0*) ⇒ neprivilegirano izvršavanje
    - program koji se izvršava u PL0 ne može pristupiti svim resursima sustava
    - iz PL0 može se preći u PL1 jedino u slučaju iznimke
  - PL1 (*Privilege Level 1*) ⇒ privilegirano izvršavanje
    - program koji se izvršava u PL1 može pristupiti svim resursima sustava
- načini rada procesora (*Operating Modes*) - ukupno 7 načina rada
  - PL0
    - *User*, USR
  - PL1
    - iznimke (*Exception modes*)
      - *Fast Interrupt Mode*, FIQ
      - *Interrupt Mode*, IRQ
      - *Supervisor*, SVC
      - *Monitor*, MON
      - *Abort*, ABT
      - *Undefined*, UND
    - regularni rad (nije iznimka)
      - *System*, SYS
- uočiti
  - u načinu rada *User* obično se izvršava korisnikov program
    - PL0 ⇒ ne može pristupiti zaštićenim dijelovima sustava
      ⇒npr. ne može preprogramirati prekide
  - način rada *System* tipično se koristi kod inicijalizacije
    - može pristupiti svim resursima (registrima)
    - ne može biti prekinut prekidom
  - u ostalim načinima rada poslužuju se iznimke
  - nakon reseta procesor je u načinu rada *Supervisor*

Registri procesora

- programski model sadrži 40 registra
  - 33 registra opće i posebne namjene
  - 1 registar sa statusnim i upravljačkim zastavicama
  - 6 registara za pohranu stanja
  - veći broj upravljačkih registara
- svi registri imaju širinu od 32 bita
- registri opće i posebne namjene

PL0	**PL1**						
		Iznimke					
USR	**SYS**	**SVC**	**ABT**	**UND**	**MON**	**IRQ**	**FIQ**
R0	R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7	R7
R8	R8	R8	R8	R8	R8	R8	R8_fiq
R9	R9	R9	R9	R9	R9	R9	R9_fiq
R10	R10	R10	R10	R10	R10	R10	R10_fiq
R11	R11	R11	R11	R11	R11	R11	R11_fiq
R12	R12	R12	R12	R12	R12	R12	R12_fiq
SP	SP	SP_svc	SP_abt	SP_und	SP_mon	SP_irq	SP_fiq
LR	LR	LR_svc	LR_abt	LR_und	LR_mon	LR_irq	LR_fiq
PC	PC	PC	PC	PC	PC	PC	PC

  - u praksi se koriste i nazivi
    - niži registri (*Low Registers*) $\Rightarrow$ R0-R7
    - viši registri (*High Registers*) $\Rightarrow$ R8-R15
  - uočiti
    - niži registri su zajednički za sve načine rada
    - žuto označeni registri se razlikuju za pojedine načine rada (postoji banka paralelnih registara)

- posebne funkcije registara
  - SP (R13) koristi se kao pokazivač stoga (*Stack Pointer*)
  - LR (R14) sadrži povratnu adresu potprograma ili programa koji obrađuje iznimku (*Link Register*, LR)
  - PC (R15) je programsko brojilo (*Program Counter*, PC)

- registar trenutnih stanja, CPSR (*Current Program Status Register*)
  - vidljiv je u svim načinima rada
  - sadrži zastavice i upravljačke bitove

31	30	29	28	27	26-25	24	23-20	19-16	15-10	9	8	7	6	5	4-0
N	**Z**	**C**	**V**	**Q**				**GE[3:0]**							
					IT[1:0]	**J**			**IT[7:2]**	**E**				**T**	
											A	**I**	**F**		**M[4:0]**

  - sve ovo je jedan registar
  - bojama su izdvojene funkcije kako iz vidi korisnik
    - **zastavice uvjeta** (*Condition Flags*)
      - **N** *Negative*
      - **Z** *Zero*
      - **C** *Carry*
      - **V** *oVerflow*
      - **Q** *DSP overflow and saturation flag*
      - **GE[3:0]** *Greater than or Equal*
        - ⇒ sadrži 4 bita, koriste se u operacijama koje rade sa svakim pojedinim bajtom u riječi
    - **bitovi koji određuju stanja**
      - **IT** - indicira trenutno stanje *If-Then* instrukcije (za razumijevanje ICI/IT vidi URS)
      - **J** i **T** - način rada procesora
        - 00b ARM
        - 01 Thumb
        - 10 Jazelle
        - 11 ThumbEE
      - **E** - zapis podataka u memoriji
        - = 1 (procesor u Zynqu podržava samo *little endian*)
    - **upravljački bitovi**
      - **A** Async. WDT Abort disable (samo od WDT)
      - **I** IRQ disable
      - **F** FIQ disable
      - **M[0:4]** način rada

- registar za pohranu stanja, SPSR (*Saved Program Status Register*)
  - služi za pohranu stanja registra CPSR kod obrade iznimaka
  - svaki način rada ima svoju "kopiju" registra SPSR

PL0	**PL1**						
		Iznimke					
USR	**SYS**	**SVC**	**ABT**	**UND**	**MON**	**IRQ**	**FIQ**
CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_svc	SPSR_abt	SPSR_und	SPSR_mon	SPSR_irq	SPSR_fiq

  - registar trenutnih stanja, CPSR (*Current Program Status Register*)
    - vidljiv je u svim načinima rada
    - sadrži zastavice i upravljačke bitove

  - registar za pohranu stanja, SPSR (*Saved Program Status Register*)
    - služi za pohranu stanja registra CPSR kod obrade iznimaka
    - svaki način rada ima svoju "kopiju" registra SPSR

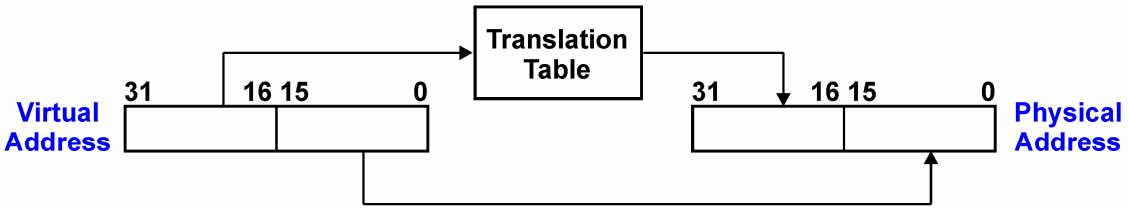
- upravljački registri (*System Control Coprocessor. C15*)
  - sadrže upravljačke bitove i zastavice stanja raznih funkcija procesora
    - konfiguracija i upravljanje priručnim memorijama
    - upravljanje MMU-om
    - upravljanje Jazelle DBX podrškom
    - itd.
  - podijeljeni su u 16 grupa registara
    c0, c1, ..., c15

Jedinica za upravljanje memorijom

- mnogi operacijski sustavi dodjeljuju procesima područja u virtualnom memorijskom prostoru
  - prednosti
    - proces ima privid da mu je dodijeljena kontinuirana memorija
    - svaki proces ima svoje virtualno područje i ne može pristupiti području koje je dodijeljeno drugom procesu

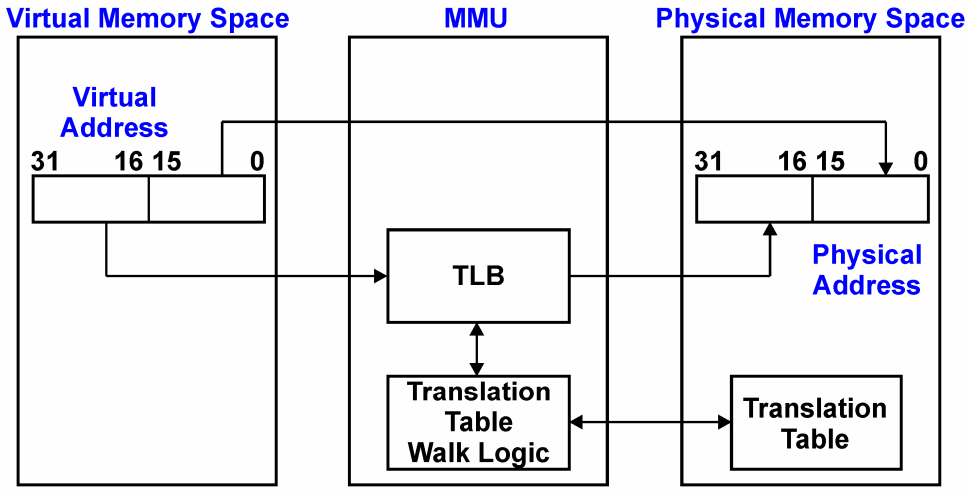
⇒jedinica za upravljanje memorijom (*Memory Management Unit*, MMU)
- translatira virtualne adrese u fizičke (*memory translation*)
- procesu onemogućava pristup izvan dodijeljenog prostora (*memory protection*)

- virtualan memorijski prostor
  - obuhvaća cijelo područje koje procesor može adresirati
  - virtualan memorijski prostor dijeli se na stranice (*pages*)
  - primjer
    - ARM Cortex A9
      - može adresirati 2^{32} = 4GB memorijskog prostora
      - veličina stranice: 4 kB, 64 kB, 1 MB, 16 MB
- fizički memorijski prostor
  - obuhvaća prostor na kojem se nalaze adrese fizičkih memorija
- translacijska tablica (*translation table*, *page table*)
  - sadrži adrese za translaciju virtualnih adrese u fizičke
    ⇒jedan redak (*Page Table Entry*, PTE) za svaku stranicu
- princip translacije (primjer za stranice veličine 64 kB)



  - donjih 16 bitova adrese je pomak unutar stranice
    ⇒ne mijenja se
  - gornjih 16 bitova je adresa retka u tablici stranica
    ⇒redak tablice sadrži gornjih 16 bitova fizičke adrese
- problem
  - tablica se nalazi u memoriji
    ⇒promjena stranice traži čitanje tablice iz memorije
    ⇒sporo
- rješenje
  - priručna memorija za pristup tablici (*Translation Lookaside Buffer*, TLB)

- izvedba MMU

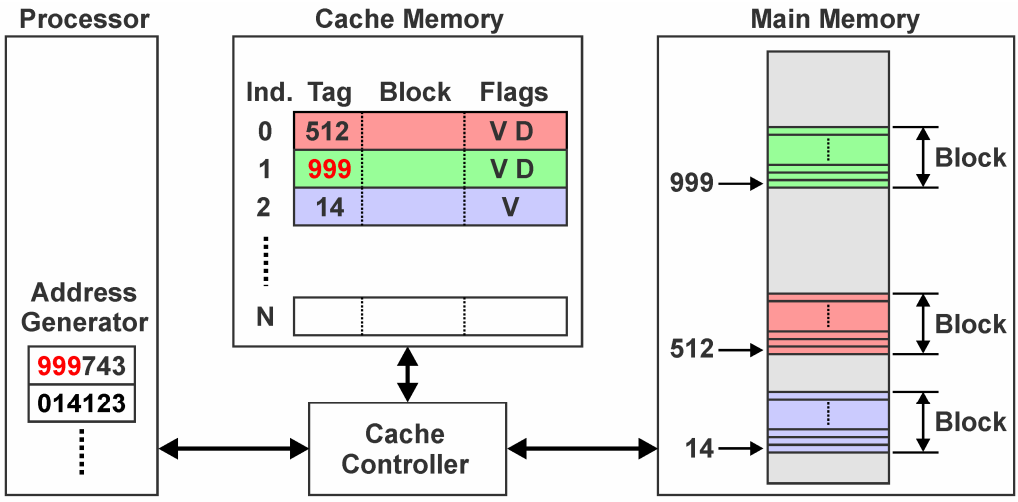


  - postupak translacije
    - čita se TLB
    - ako virtualne adresa postoji u TLB (TLB *hit*)
      ⇒generira se fizička adresa
    - ako virtualna adresa ne postoji u TLB (TLB *miss*)
      ⇒pretražuje se translacijska tablica u memoriji (*table walk*)
      - ako virtualna adresa postoji u tablici
        ⇒adresa se pohranjuje u TLB i generira fizička adresa
      - ako virtualna adresa ne postoji u tablici
        ⇒greška (*memory abort*)

L1 Priručna memorija

- problem - glavna memorija je spora
  ⇒potrebno je više ciklusa za čitanje podatka
  - moguće je brže čitati podatke u nizu (blokove podataka)
- rješenje - dodati malu brzu priručnu memoriju izravno u procesor
- podsjetimo se
  - TLB u MMU je jedna od priručnih memorija
- ostale priručne memorije
  - L1 priručna memorija
    - u (svakoj) procesorskoj jezgri
      - instrukcijska priručna memorija (I *Cache*)
      - podatkovna priručna memorija (D *Cache*)
  - L2 priručna memorija
    - između MPU i vanjske dinamičke memorije (DDR)

- Kako procesor pristupa (čita ili piše) glavnoj memoriji preko priručne?

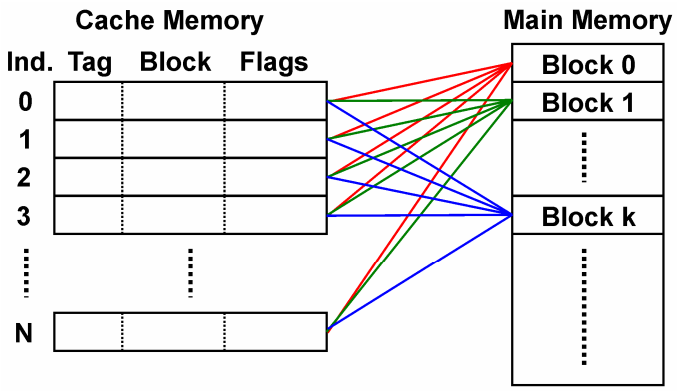


- priručna memorija sadrži kopije blokova iz glavne memorije
- blok i podaci koji ga opisuju tvore jedan unos (*cache entry*)
  - oznaka (*tag*)
    - početna adresa s koje je iz glavne memorije uzet blok
  - blok (*block*, *cache block*, *cache line*)
    - lokacije (u nizu) koje se prenosi između glavne i priručne memorije prilikom svake komunikacije
  - zastavice (*flags*) - opisuju unos, npr.
    V - unos je važeći (*valid*)
    ⇒ npr. nakon reseta unosi nisu važeći
    D - procesor je upisivao podatke u blok (*dirty*)
    ⇒ blok će kasnije trebati upisati u glavnu memoriju
- primjer - čitanje
  - kontroler priručne memorije provjerava postoji li tražena lokacija u priručnoj memoriji
  - ako postoji (*cache hit*)
    ⇒ čita se iz priručne memorije
  - ako ne postoji (*cache miss*)
    ⇒ kontroler kreira novi unos, kopira sadržaj memorije i procesor čita iz priručne memorije
- primjer - pisanje
  - *write policy* - određuje način upisa određuje u memoriju
    - piše se izravno u glavnu memoriju (*write through*)
    - piše se u priručnu memoriju (*write back*)
      ⇒ zastavica D se postavlja
      ⇒ naknadno se blok upisuje u glavnu memoriju

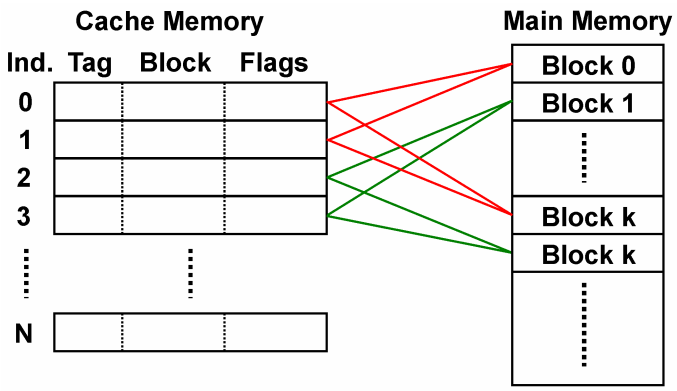
- Što se događa kad se novim unosima popuni cijela priručna memorija?
  ⇒potrebno je zamijeniti postojeći unos novim
  - *replacement policy* - način na koji se stari unosi zamjenjuju novima
    - primjeri
      - zamjenjuje se zadnji korišteni unos
      - *round robin*
      - *pseudo random*

- Može li neki blok iz glavne memorije biti pohranjen u bilo koji unos?
  - da - priručna memorija s potpunim pridruživanjem (*fully associative*)
    - veća je vjerojatnost da će podatak biti u priručnoj memoriji
    – provjera je li neka lokacija u priručnoj memoriji je dugotrajna ili sklopovski složena

    ⇒bolje je to nekako ograničiti
  - *placement policy*
    - određuje na koliko mjesta u priručnoj memoriji se može pohraniti neki blok iz glavne memorije
    - priručna memorija s N-strukim pridruživanjem (*N set-associative*)
      - svaki blok iz glavne memorije može biti pohranjen na N mjesta u priručnoj memoriji
      - blokovi glavne memorije povezani su sa skupovima unosa u priručnoj memoriji
  - primjer
    - potpuno pridruživanje
      - svaki blok glavne memorije može biti pridružen bilo kojem unosu priručne memorije

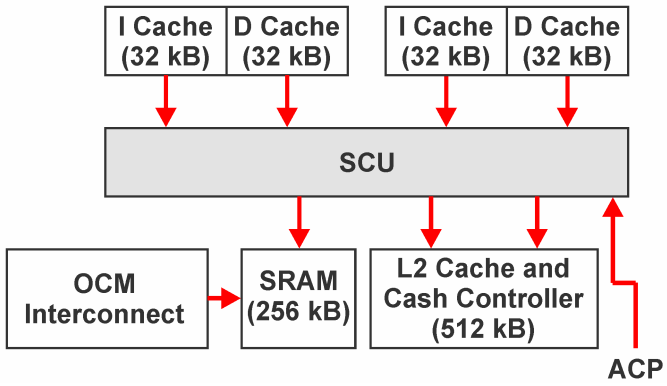


- primjer
  - dvostruko pridruživanje (*2-way set-associative*)
    - svaki blok glavne memorije može biti pridružen jednom od dvaju unosa priručne memorije



- L1 priručna memorija u procesoru Cortex A9
  - veličina: 32 kB
  - duljina bloka: 32 B
  - *replacement policy*: *pseudo round robin* ili *pseudo random*
  - *placement policy*: *4-way set-associative*
  - parametri rada su podesivi kroz upravljačke registre

Osiguravanje koherencije



- uočiti
  - svaki procesor ima vlastite priručne memorije
- problem
  - pretpostavimo da obje D priručne memorije sadrže istu lokaciju glavne memorije
  - ako lijevi procesor promijeni sadržaj te lokacije u priručnoj memoriju, desna priručna memorija više nema ispravan podatak

    $\Rightarrow$ potrebno je osigurati koherenciju priručnih memorija

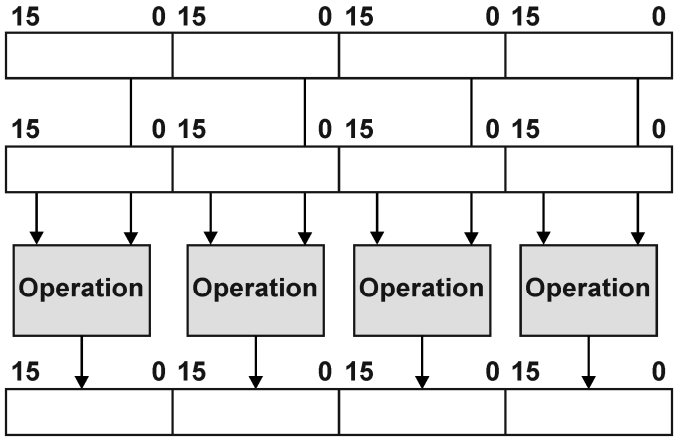
- *Snoop Control Unit* (SCU)
  - osigurava koherenciju priručnih memorija (*cache coherency*)

  ⇒u našem slučaju
    - osiguravanje koherenciju dviju L1 memorija o odnosu na L2 memoriju i/ili OCM
    - može se uključiti i dodatni memorijski prostor sklopa koji je implementiran u PL

      ⇒preko ACP (*Accelerator Coherency Port*)

NEON koprocesor

- NEON *Media Processing Engine*, NEON MPE
  - ima ulogu koprocesora
  - proširuje instrukcijski skup
    - *scalar double-precision floating-point*
    - SIMD instrukcijama nad podacima tipa
      - *unsigned and signed integer*
      - *single-precision floating-point*
- SIMD
  - jednostruki tok instrukcija, višestruki tok podataka (*Single Instruction Multiple Data*)
  - primjer



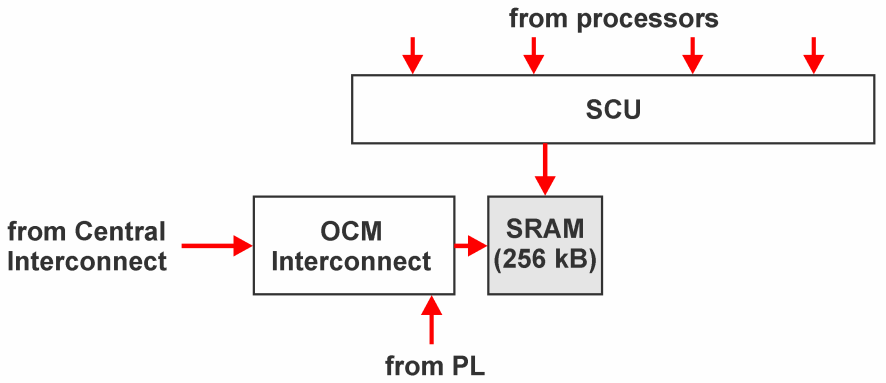
- uočiti
  - SIMD omogućuje učinkovitu implementaciju postupaka za obradu signala i slika

    ⇒postoje asemblerske instrukcije koje izvršavaju ove operacije

      ⇒na raspolaganju su C biblioteke koje koriste ove mogućnosti

3.2.1.2 Ugrađena memorija (OCM)

- *On Chip Memory*, OCM
  - 256 kB SRAM
    - dvopristupna memorija (*dual-ported*)



      - uočiti
        - omogućava brzu komunikacija procesora i PL
  - 128 kB ROM (BootROM)
    - nije vidljiva korisniku
- podržava pristup od 128 bitova

  ⇒dvije instrukcije

  ⇒vrlo brz pristup, npr. za dohvat operacijskog koda
- brzina rada
  - latencija 23 perioda takta (uz najbrži takt procesora, 667 MHz)
  - podsjetimo se
    - priručne memorije imaju blokove od 32 bajta

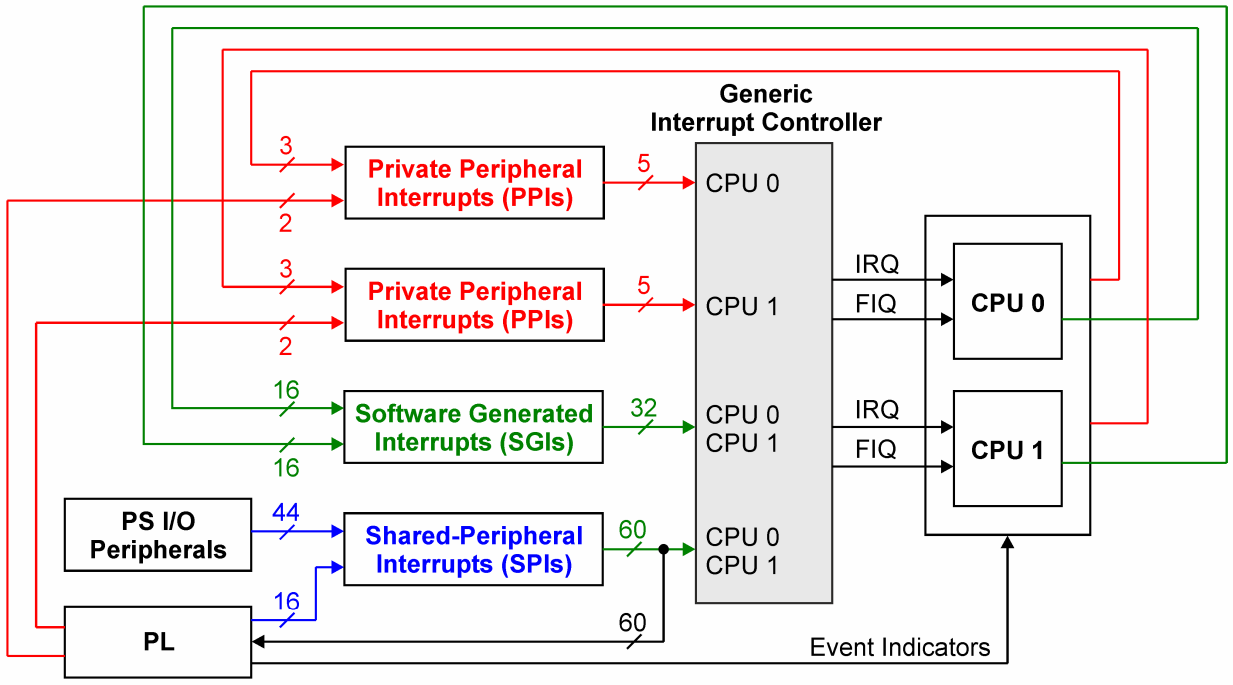
      ⇒SRAM ne usporava rad sustava

3.2.1.3 L2 priručna memorija

- L2 priručna memorija
  - veličina: 512 kB
  - duljina bloka: 32 B
  - *replacement policy*: *pseudo random*
  - *placement policy*: *8-way set-associative*
  - parametri rada su podesivi kroz upravljačke registre

3.2.1.4 Prekidni kontroler

- *Generic Interrupt Controller*, GIC
- prekidi s obzirom na izvor i posluživanje



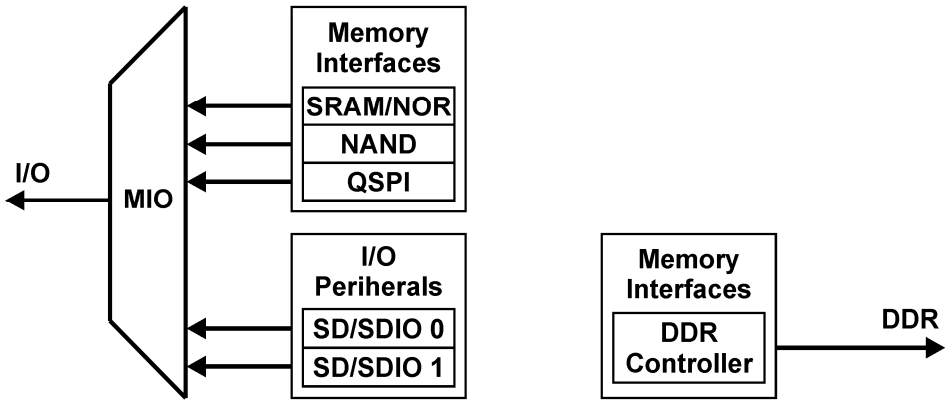
- prekidi od privatnih periferija
  - svaki procesor ima 5 izvora koji se prosljeđuju samo njemu
    - tri uz PS-a
      - globalno brojilo (GTC) (jedno za oba procesora)
      - privatno brojilo procesora (CPU *private timer*)
      - privatni *watch-dog* procesora (CPU *private* WDT)
    - dva iz PL-a
      - nFIQ (*Fast Interrupt*)
      - nIRQ (*Interrupt*)
- programski prekidi
  - svaki procesor može programski zatražiti prekid za samog sebe ili za drugi procesor
  - ukupno 16 izvora dolazi od svakog procesora
- prekidi od zajedničkih periferija
  - ovi prekidi mogu biti proslijeđeni jednom ili drugom procesoru
  - ima uh 60
    - 44 od periferija PS-a
    - 16 iz PL-a
  - uočiti
    - svih 60 zahtjeva može biti vraćeno u PL

- pojedini procesor je moguće zaustaviti i ostaviti u stanju čekanja
    - za zaustavljanje se koriste instrukcije
        WFI - *Wait For Interrupt*
        WFE - *Wait For Event*
    - signali za buđenje mogu doći i iz PL-a

3.2.2 Memorije

3.2.2.1 Izvedbe memorija

- podsjetimo se
    - u APU je smještena unutrašnja memorija (OCM)
- na Zynq se mogu spojiti i vanjske memorije



- statičke vanjske memorije
    - spojene su preko MIO
        - SRAM
        - Flash (NOR i NAND tip)
            - mogu se koristiti kao *boot* memorije
        - QSPI
            - veza preko 4 SPI sučelja
                $\Rightarrow$ brze, a traže manje priključaka nego SRAM ili FLASH
        - SD/SDIO (*Secure Digital/Secure Digital Input Output)*
            $\Rightarrow$ za spajanje flash memorijskih kartica
- dinamička vanjska memorija
    - spojena izravno na priključke
        $\Rightarrow$ za radne memorije većeg kapaciteta

3.2.2.2 Memorijska mapa

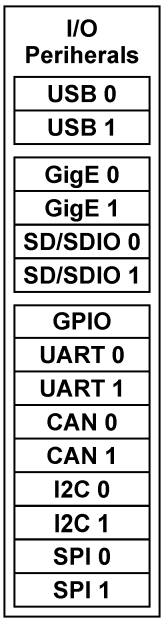
- adresna sabirnica
  - širina 32 bita
    - ⇒adrese 0x0000 0000 do 0xFFFF FFFF
    - ⇒ukupno 4 294 967 296 lokacija (4 GBytes)
- adresni prostor pojedinih podsustava

Address Range	Subsystem
0x0000_0000 - 0x3FFF_FFFF	OCM, DRAM
0x4000_0000 - 0x7FFF_FFFF	General purpose port 0 to PL
0x8000_0000 - 0xBFFF_FFFF	General purpose port 1 to PL
	Reserved
0xE000_0000 - 0xE02F_FFFF	I/O peripheral registers
	Reserved
0xE100_0000 - 0xE5FF_FFFF	Static Memories (SRAM, NAND/NOR Flash)
	Reserved
0xF800_0000 - 0xF800_0BFF	SLCR registers (upravljanje MIO i sl.)
	Reserved
0xF800_1000 - 0xF880_FFFF	PS system registers (for DRAM, OCM, TTC, etc.)
	Reserved
0xF890_0000 - 0xF8F0_2FFF	CPU private registers (for SCU, SWDT, private timers, etc.)
	Reserved
0xFC00_0000 - 0xFDFF_FFFF	QSPI memory
	Reserved
0xFFFC_0000 - 0xFFFF_FFFF	OCM (when mapped to high address space)

- početne adrese registara I/O periferija

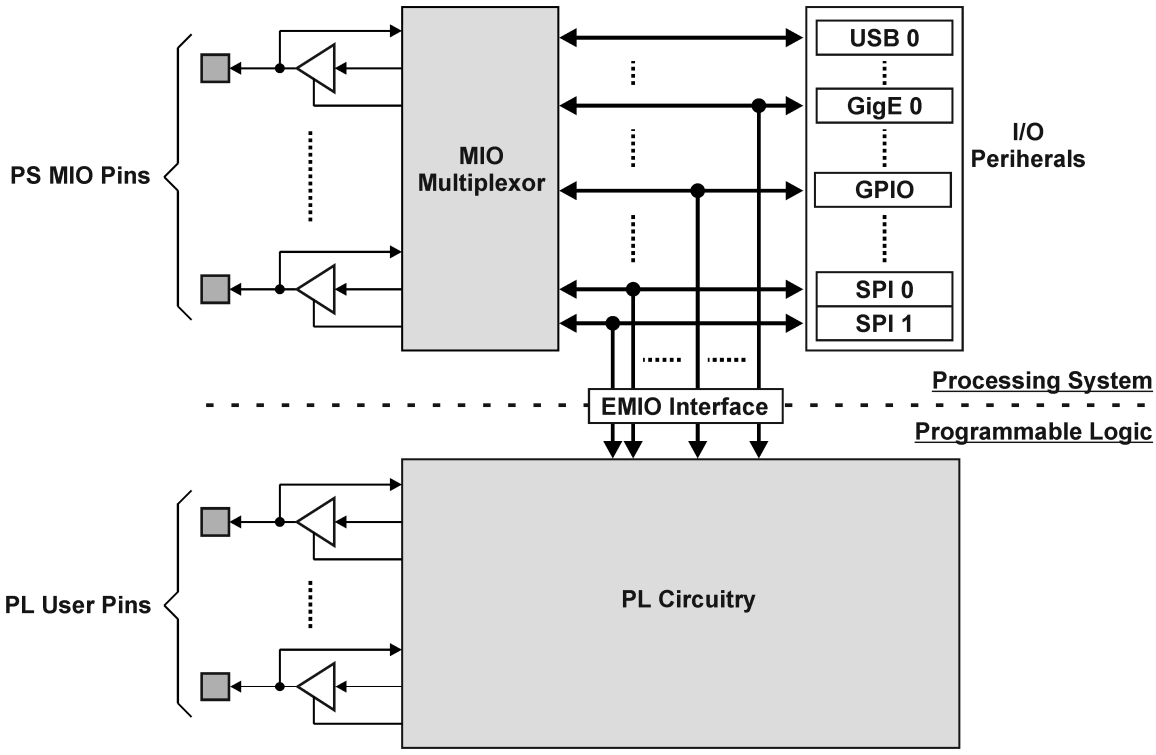
Address Range	Subsystem
0xE000_0000, 0xE000_1000	UART Controllers 0, 1
0xE000_2000, 0xE000_3000	USB Controllers 0, 1
0xE000_4000, 0xE000_5000	I2C Controllers 0, 1
0xE000_6000, 0xE000_7000	SPI Controllers 0, 1
0xE000_8000, 0xE000_9000	CAN Controllers 0, 1
0xE000_A000	GPIO Controller
0xE000_B000, 0xE000_C000	Ethernet Controllers 0, 1
0xE000_D000	Quad SPI Controller
0xE000_E000	Static Memoriy Controllers
0xE010_0000, 0xE010_1000	SDIO Controllers 0, 1

3.2.3 Periferije i veza s vanjskim svijetom



- periferije
  - brze
    - komunikacija
      - USB, GigE
    - pohrana podataka
      - SD/SDIO
  - klasične
    - komunikacije:
      - UART, CAN, I2C, SPI
    - ulazno izlazni priključci opće namjene
      - GPIO

- veza između SoC i vanjskog svijeta
  - na vanjske priključke PS-a može biti spojeno najviše 54 signala
    ⇒ MIO multipleksorom odabire koje periferije su spojene na vanjske priključke
  - na PL mogu biti spojena 192 signala (64 ulaza i 128 izlaza)
    ⇒ dovedeni su na priključke PL-a kroz EMIO sučelje
- spoj periferija s priključcima izvodi se kroz MIO i EMIO



- uočiti
  - u PL preko EMIO ulaze signali svih periferija osim USB
  - signali koji uđu u PL mogu, ali ne moraju izaći na priključke
    $\Rightarrow$ mogu se koristiti interno za upravljanje sklopovljem u PL
- uočiti
  - izlazni priključci na MIO su dvosmjerni
    $\Rightarrow$ imaju stanje visoke impedancije
  - u EMIO se spaja 64 ulazna i 64+64 izlazna signala
    $\Rightarrow$ oni se kroz PL mogu spojiti na dvosmjerne vanjske priključke
- paziti
  - unutar PL-a ne postoji stanje visoke impedancije
  - u PL-u to su 64+64+64 "obična" signala

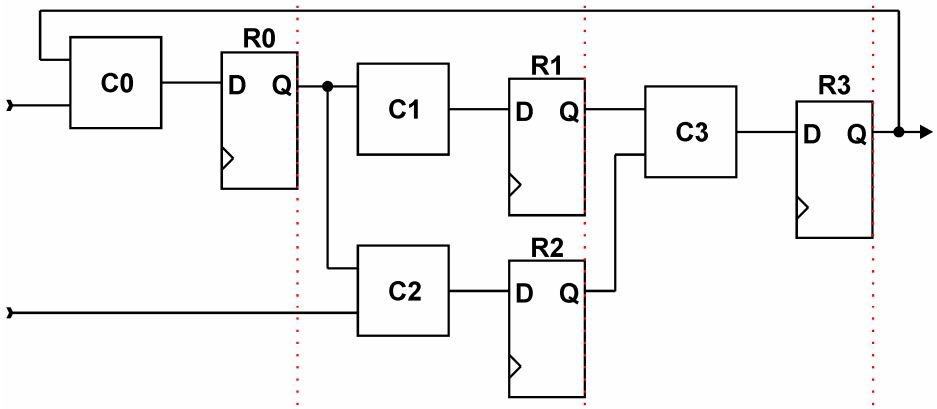
- spoj periferija s priključcima
  - primjer: UART, I2C

| Periph. | Bank 0 | | | | | | | | | | | | | | | | Bank 1 | | | | | | | | | | | | | | | |

3.3 Programabilna logika

3.3.1 Ideja programabilnog sklopovlja

- fiksno ožičenom sklopovlju shema se nakon implementacije ne može više promijeniti
  - Može li se napraviti sustav čija shema bi se mogla mijenjati?
- shema (svakog) digitalnog sustava



- uočiti
  - signal svakim periodom takta propagira od registra od registra
    ⇒ registri su slijedni (*sequential*) sklopovi
  - između registara nalaze se kombinacijske funkcije

  ⇒ ovakav model naziva se *Register Transfer Level* (RTL) model
- ako su na raspolaganju
  - kombinacijskih funkcija koje je moguće zadati po želji
  - registri (bistabili)
  - mreža za programabilno spajanje

  moguće je implementirati bilo kakvu shemu
- zbog veće učinkovitosti sklopovlja, mogu biti dodane i neke fiksno ožičene komponente
  - brza zbrajala (lance za propagaciju prijenosa)
  - množila
  - memorije
  - DSP odsječke
  - itd.

⇒ **programabilna logička polja (*Field Programmable Gate Arrays*, FPGA)**

3.3.1.1 Implementacija kombinacijske funkcije

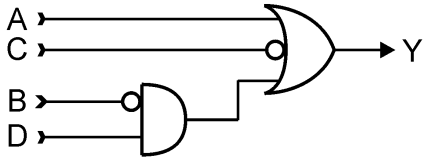
- primjer kombinacijske funkcije s 4 varijable

```
process (A,B,C,D) is
begin
   if (A = '1') then
      Y <= '1';
   elsif (B = '0') and (B/=C) then
      Y <= D;
   else
      Y <= not C;
   end if;
end process;
```

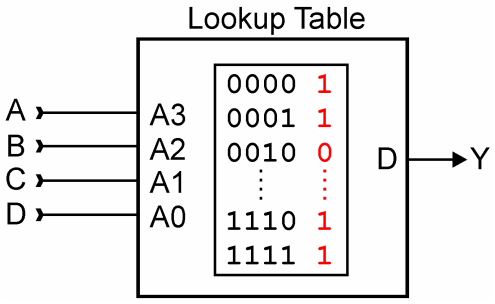
A	B	C	D	Y
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

- implementacija pomoću logičkih sklopova

$$Y=A+nC+nB\cdot D$$



- implementacija pomoću prozivne tablice (*Lookup Table*, LUT)



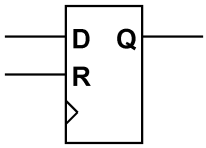
  - uočiti
    - LUT može implementirati bilo koju kombinacijsku funkciju
      ⇒potrebno je samo promijeniti njegov sadržaj
      ⇒broj lokacija LUT-a

$$2^{\text{broj varijabli}}$$

    - propagacija kroz LUT ne ovisi o složenosti funkcije

3.3.1.2 Implementacija slijedne funkcije

- primjer slijedne funkcije
  - bistabil sa sinkronim resetom

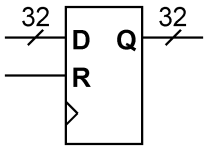



```
-- In arcitecture declaration
signal clk, R, D, Q : std_logic;

-- In architecture body
process (clk) is
begin
   if rising_edge(clk) then
      if R = '1' then
         Q <= '0';
      else
         Q <= D;
      end if;
   end if;
end process;
```

R	D	clk	Q
0	0	↑	0
0	1	↑	1
1	0	↑	0
1	1	↑	0

  - 32-bitni registar sa sinkronim resetom



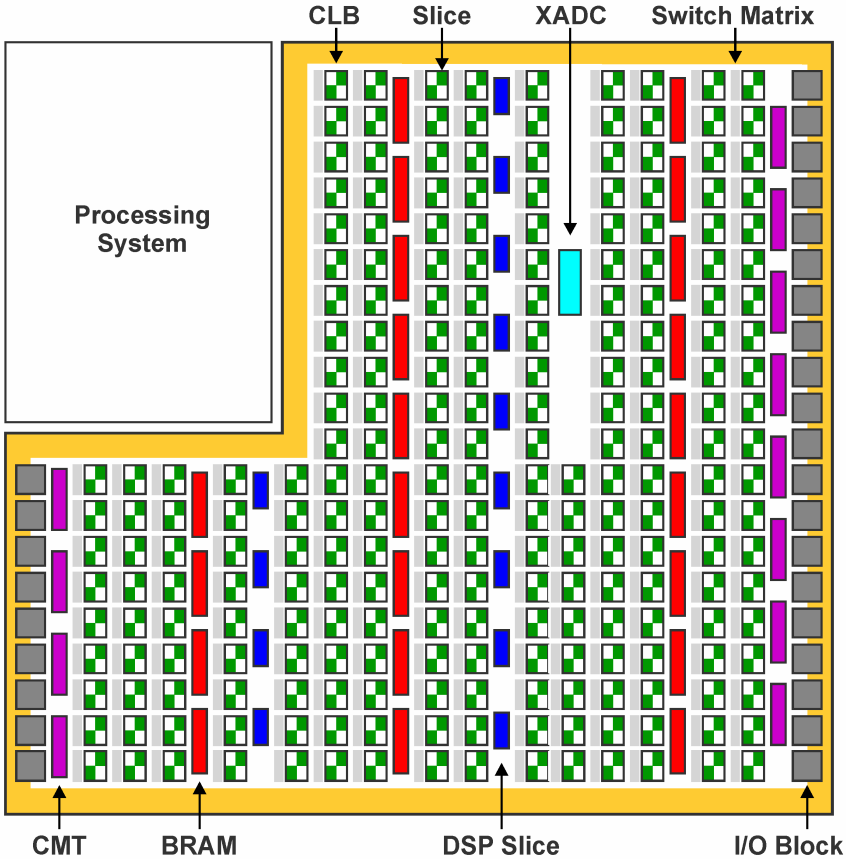

```
-- In arcitecture declaration
signal clk, R : std_logic;
signal D, Q   : std_logic_vector(31 downto 0);

-- In architecture body
process (clk) is
begin
   if rising_edge(clk) then
      if R = '1' then
         Q <= (others =>'0');
      else
         Q <= D;
      end if;
   end if;
end process;
```

3.3.2 Građa programabilne logike

- novije generacije Xilinx programabilnih sklopova
  - serija 7: Spartan-7, Virtex-7, Artix-7, Kintex-7, Zynq-7000 (28 nm)
  - UltraScale (20 nm)
  - UltraScale+ (16 nm)
  - Versal (7 nm)
- programabilna logika u Zynq-7000

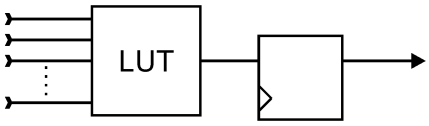
  $\Rightarrow$ implementira Artix 7 i Kintex 7 FPGA sklopove



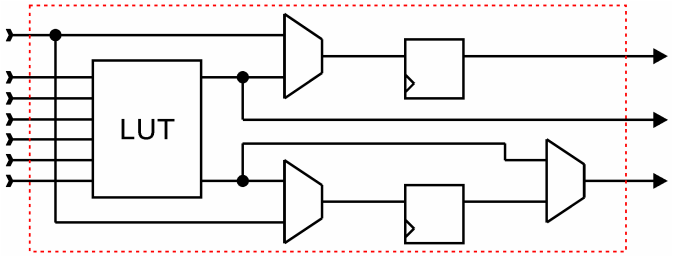
- osnovne komponente
  - univerzalne
    - konfigurabilni logički blokovi (*Configurable Logic Block*, CLB)
    - ulazno izlazni blokovi (*Input Output Block*, IOB)
    - programabilni prospoji
    - sinteza i upravljanje taktom (*Clock Management Tile*, CMT)
  - namjenske - ugrađene, fiksno ožičene
    - blokovske RAM memorije (*Block* RAM, BRAM)
    - DSP odsječci (DSP *Slice*)
    - AD pretvarač (Xilinx ADC, XADC)

3.3.2.1 Konfigurabilni logički blokovi

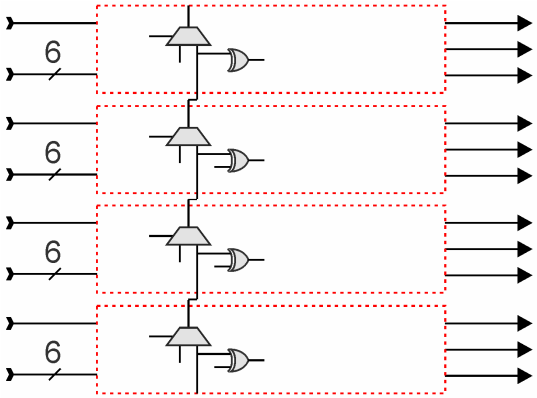
- osnovni gradivni blok



- izvedba u FPGA sklopovima serije 7

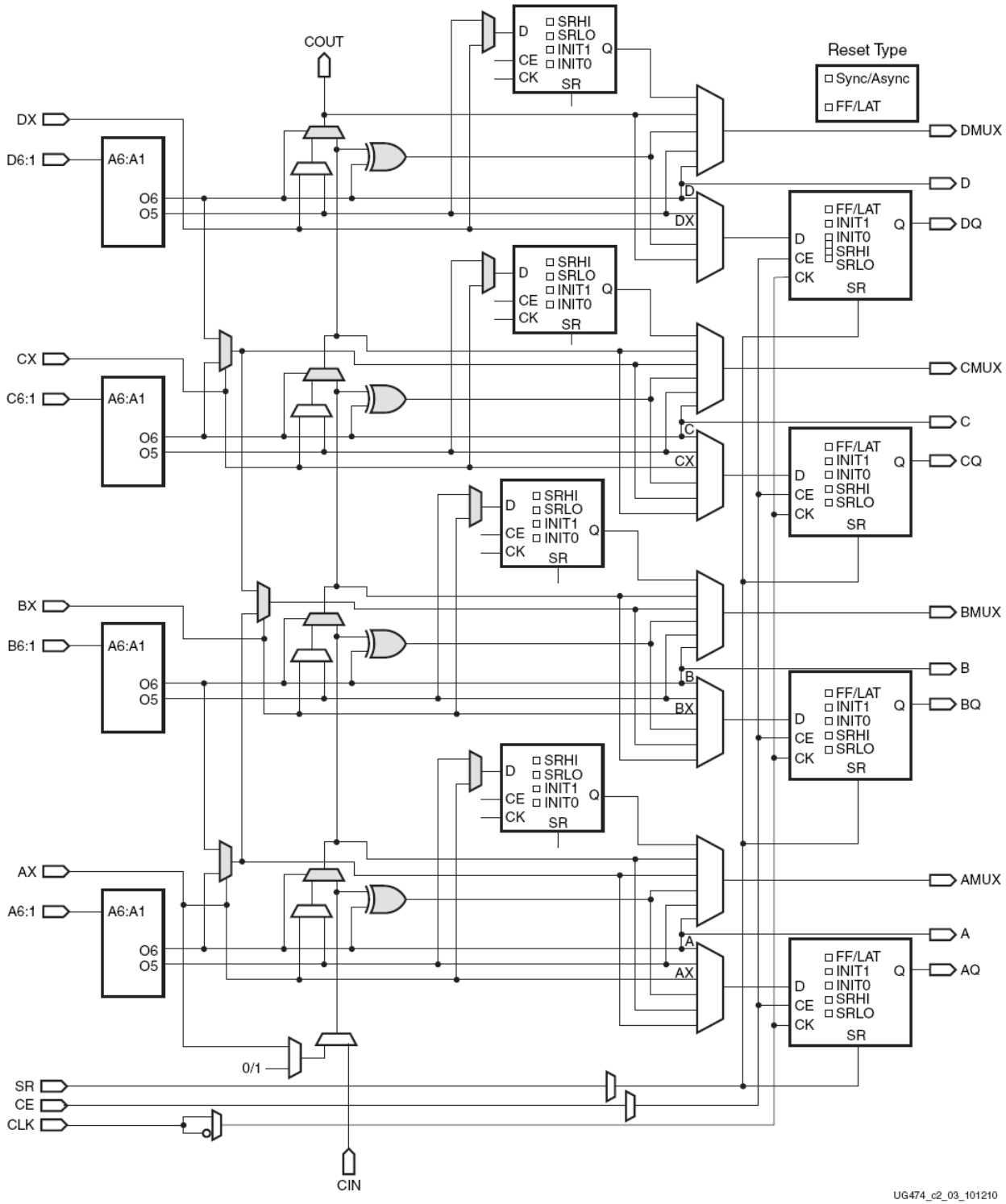


  - LUT se može koristiti
    - kao 6-ulazni
    - kao dva 5-ulazna LUT-a (šesti ulaz spojen je u '1')
- odsječak (*slice*)
  - sastoji se od
    - četiri osnovna gradivna bloka
    - lanac za propagaciju prijenosa kod zbrajanja (*carry chain*)



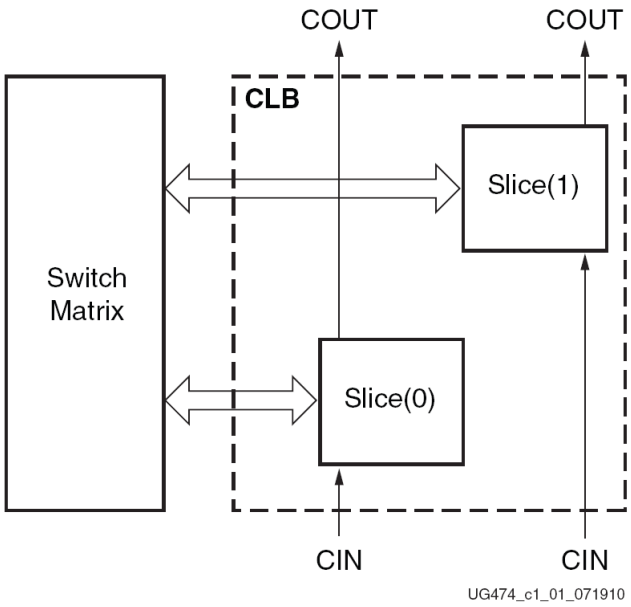
- uočiti
  - LUT je RAM memorija čiji sadržaj određuje kombinacijsku funkciju
    ⇒LUT se može koristiti i kao "obična" memorija
    - postoje dvije vrste odsječaka
      - SLICEL
        - LUT implementira isključivo logičku funkciju
      - SLICEM
        - LUT implementira logiku, memoriju ili posmačne spremnike (*shift registers*)

- blokovska shema odsječka L



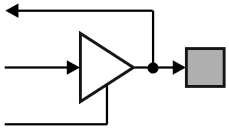
- uočiti
  - 4 LUT-a
  - 8 bistabila
  - lanac za prijenos preljeva

- konfigurabilni logički blok (*Configurable Logic Block*, CLB)
  - sadrži dva odsječka

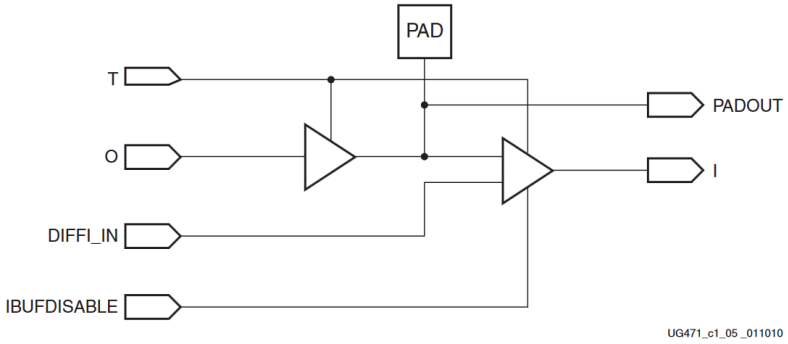


3.3.2.2 Ulazno-izlazni blokovi i vanjski priključci

- vrste ulazno-izlaznih priključaka
  - prema polaritetu
    - unipolarni (*single-ended*)
    - diferencijalni (*differential*)
  - prema svojstvima
    - priključci visokih performanci (*High Performance*, HP)
      - za spajanje memorija i sučelja prema drugim čipovima
      - I/O standardi: 1.2 V, 1.35 V, 1.5 V, 1.8 V
    - priključci za veći broj I/O standarda (*High Range*, HR)
      - I/O standardi: 1.2 V, 1.35 V, 1.5 V, 1.8 V, 2.5 V, 3.3 V
- priključci su podijeljeni u banke
  - priključci iste banke koriste isti I/O standard (napon napajanja, prag, maksimalna struja, itd.)
- sklopovlje spojeno na priključak
  - princip rada dvosmjernih priključaka



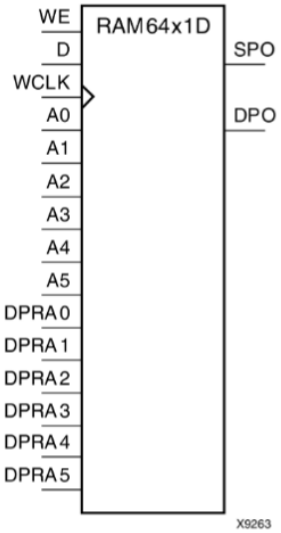
- stvarna izvedba
  - primjer - unipolarni HR priključak



3.3.2.3 Ugrađene komponente

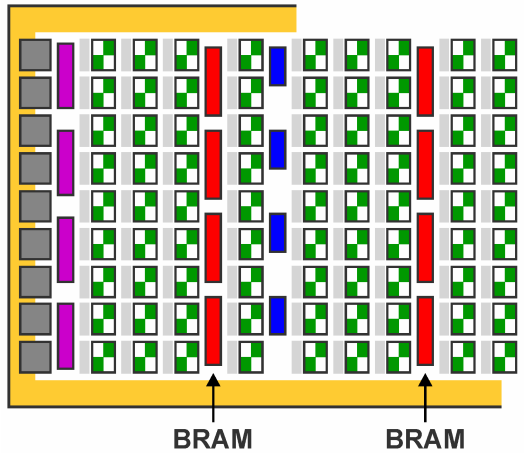
Memorije

- podsjetimo se
  - LUT-ovi u SLICEM mogu se koristiti kao RAM memorije
  - primjer
    - 6-ulazni LUT ⇒ memorija 64 x 1 bit
      - takva komponenta ima oblik



- distribuirani RAM
  - ako treba memorija veća od 64x1
    ⇒potrebno ju je složiti od više LUT-ova i kombinacijske logike
  - uočiti
    - distribuirani RAM nije ugrađena komponenta
      ⇒sastavlja se od CLB-ova (distribuiranih po čipu)

- blok RAM (BRAM)
  - ugrađena, fiksno ožičena komponenta

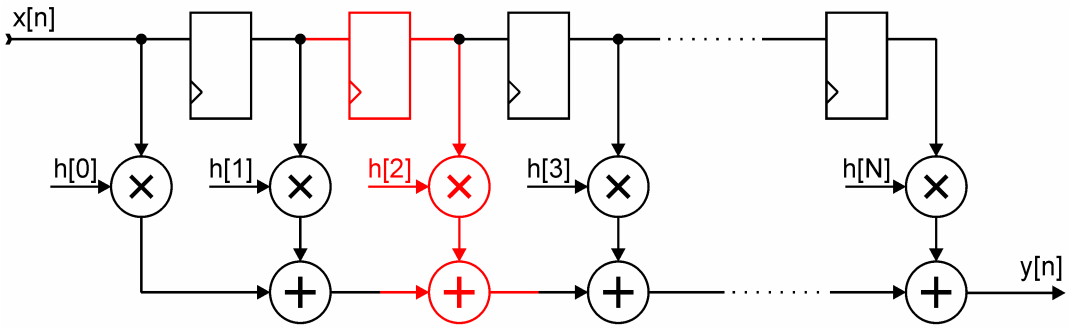


  - svojstva
    - sinkrono čitanje i pisanje
    - latencija 1 takt
    - kapacitet
      - jedna memorija kapaciteta 36kb (kb = kilo bit) ili
      - dvije memorije kapaciteta 18 kb
    - može se konfigurirati kao RAM, ROM ili FIFO
    - kao RAM i ROM može biti dvopristupan ili jednopristupan
    - podatkovna sabirnica može se podesiti na razne širine
      - primjer za memoriju kapaciteta 36 kb
        32k x 1b, 16k x 2b, 8k x 4b 4k x 8b (9b),
        2k x 16b (18b), 1k x 36b, 512k x 72b

DSP odsječci

- u obradi signala jedna od najčešće korištenih operacija je konvolucija
- primjer
  - filtracija signala x[n] FIR filtrom N-tog reda

$$y[n] = h[n] \circ x[n] = \sum_{k=0}^{N} h[k] \times x[n-k]$$



- uočiti
  - ponavlja se množenje i zbrajanje s prethodnim rezultatom
    ⇒ *multiply-accumulate*, MAC
- programska implementacija
  - *Multiply Accumulate*

```
MLA R7, R8, R9, R3      ; R7 =R8*R9+R3
```

  - *Signed Multiply Accumulate Long*

```
SMLAL R7, R8, R1, R2    ; [R8,R7] = [R8,R7]+R1*R2
```

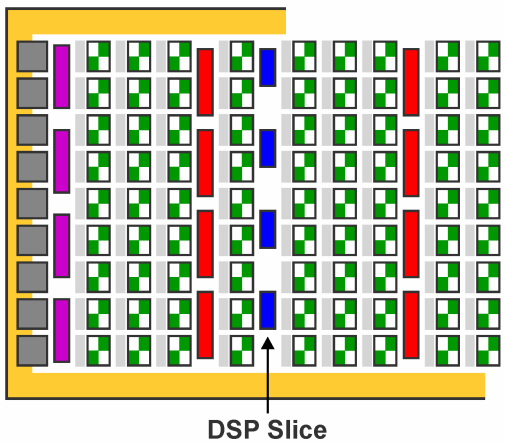
  - *Unsigned Multiply Accumulate Long*

```
UMLAL R7, R8, R1, R2    ; [R8,R7] = [R8,R7]+R1*R2
```

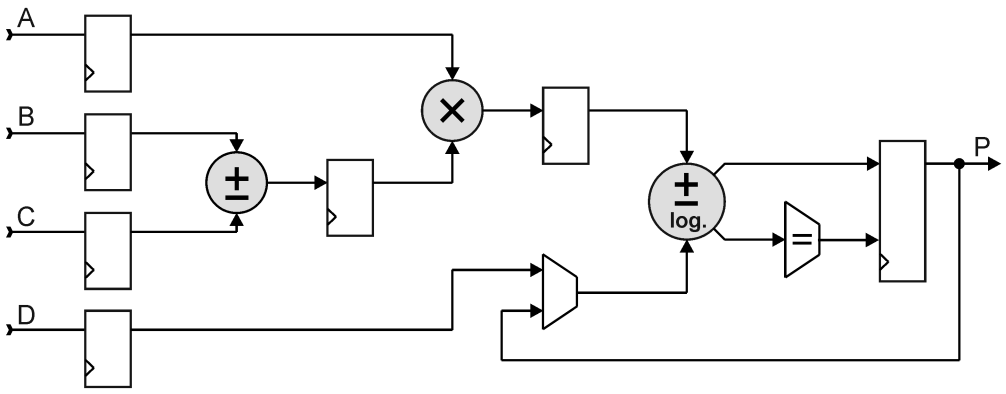
- uočiti
  - konvolucija je zahtjevna obrada signala
    - primjer
      - signal x[n] je uzorkovan frekvencijom f_s = 48 kHz
      - filtar 100. reda, N = 100

        potrebna procesna snaga $f_s \cdot (N+1) = 4\,848\,000$ MAC/s

    ⇒ implementacija mora biti učinkovita
  - 1 MAC po ciklusu
    ⇒ za velike f_s procesor ovo ne može obaviti
    ⇒ potrebno je dodatno sklopovlje
    ⇒ PL
- implementacija u programabilnoj logici
  - distribuirana
    - aritmetika se može implementirati u odsječcima
  - pomoću ugrađenih DSP odsječka



- pojednostavljena shema DSP odsječka



3.3.3 Svojstva programabilne logike u sklopu Zynq

- resursi koji su na raspolaganju u nekim sklopovima familije Zynq

	7z007s	...	7z020	...	7z100
Slices	3600		13300		69350
LUTs	14400		53200		277400
FFs	28800		106400		554800
BRAM	50		140		755
DSP48E1	66		220		2020
Banks	2		4		8

- primjer - 7z020
  - ukupna procesna moć DSP odsječaka (za simetrični FIR filtar)
    276 GMAC
  - implementacija procesora s mekom jezgrom na PL na Zynq-7020
    - MicroBlaze - 32-bitni procesor s mekom jezgrom
      - kriterij - najmanja površina
        - 602 LUT-ova
        - 224 bistabila
      - kriterij - maksimalne performance
        - 3655 LUT-ova
        - 3018 bistabila
        - 6 DSP odsječaka
        - 19 BRAM memorija

3.4 Pitanja za provjeru znanja

1. Skicirati osnovnu blokovsku shemu sustava na čipu Zynq-7000 i navesti funkcije pojedinih sklopova.
2. Nabrojiti i opisati komponente od kojih se sastoji procesorska jedinica (APU).
3. Nacrtati i opisati princip izvršavanje instrukcija kod skalarnog procesora s klasičnim protočnim sklopovljem.[(a)]
4. Nacrtati i opisati princip izvršavanje instrukcija kod superskalarnog procesora s klasičnim protočnim sklopovljem.[(a)]
5. Nacrtati i opisati princip izvršavanje instrukcija kod procesora koji sadrži diverzificirano protočno sklopovlje. Opisati diverzificirano protočno sklopovljem s varijabilnom duljinom izvršavanja.[(a)]
6. Nacrtati i opisati način na koji je kod procesora Cortex A9 izvedena superskalarnost.
7. Objasniti razliku između vektorskog i superskalarnog procesora.
8. Nacrtati i opisati način i ulogu predčitanja operacijskog koda. Pretpostaviti da obrada instrukcije sadrži sljedeće cikluse: dohvat operacijskog koda, dekodiranje i izvršavanje.
9. Nabrojiti nazive i navesti širine podataka koje koristit procesor Cortex A9. Navesti skupove instrukcija koje podržava ovaj procesor.
10. Opisati registre opće namjene procesorske jezgre Cortex-M4. Zašto su neki od registara umnoženi? Koja je uloga registara R13, R14 i R15?
11. Opisati statusne registre procesora procesorske jezgre Cortex-M4. Opisati ulogu sljedećih zastavica u registru trenutnih stanja (CPSR): N, Z, C, V, Q, J, T, E, I i F. Koja je uloga polja M.
12. Opisati razliku između virtualnog i fizičkog memorijskog prostora. Što je translacijska tablica? Opisati princip translacije ako stranica ima veličinu od 64 kB.
13. Nacrtati i opisati izvedbu jedinice za upravljanje memorijom (MMU). Koja je uloga priručne memorije za pristup tablici (TLB)? Što je TLB *hit*, TLB *miss* i TLB *walk*?
14. Skicirati blokovsku shemu koja ocrtava spoj procesora, glavne memorije, priručne memorije i kontrolera priručne memorije. Opisati način čitanja i pisanja podataka. Objasniti ulogu oznake (*tag*), bloka i zastavica (*flags*).
15. Objasniti sljedeće pojmove vezane uz priručnu memoriju: *placement policy*, *replacement policy*, potpuno pridruživanje i N-struko pridruživanje (N-*way set-associative*).
16. Što je koherencija priručne memorije i kako se ona osigurava u procesorskoj jedinici Cortex A9 MPCore.
17. Opisati ulogu NEON koprocesora kod u procesorskoj jedinici Cortex A9 MPCore. Opisati princip jednostrukog toka instrukcija s višestrukim tokom podataka (SIMD).
18. Čemu služi prekidni kontroler (GIC)? Objasniti razliku između prekida privatnih periferija, programskih prekida i prekida od zajedničkih periferija. Koja je uloga signala za buđenje (*Event Indicators*)?

19. Nacrtati blokovsku shemu i opisati princip spajanje periferija i priključcima kroz MIO i EMIO.
20. Opisati princip implementacije logičke funkcije pomoću prozivne tablice (LUT). Kako se implementira slijedno sklopovlje?
21. Opisati građu FPGA programabilne logike. Što je konfigurabilni logički blok (CLB) i od čega se sastoji? Što je odsječak (*slice*) i od čega se sastoji? Što sadrži ulazno izlazni blok (IOB)?
22. Opisati distribuirani RAM i blok RAM koji u FPGA sklopovima.
23. Nacrtati pojednostavljenu shemu DSP odsječka i opisati kako se on koristi u implementaciji FIR filtara.

(a) Pretpostaviti da obrada instrukcije sadrži sljedeće cikluse: dohvat operacijskog koda, dekodiranje, pristup memoriji, aritmetiku ili logiku, aritmetiku u pomičnom zarezu te pohrane rezultata.

3.5 Zadaci za samostalan rad

1. Potrebno je na Internetu potražiti sljedeći dokument:

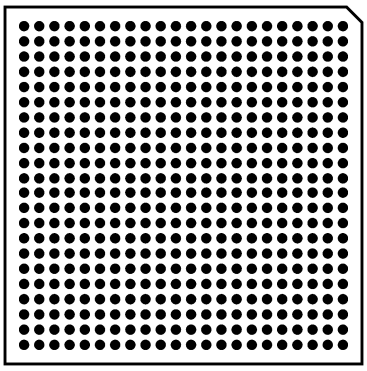
   Zynq-7000 SoC Technical Reference Manual, UG585 (v1.13), April 2, 2021

   Pregledati strukturu ovog dokumenata i uočiti dijelove koji su spomenuti na predavanju.

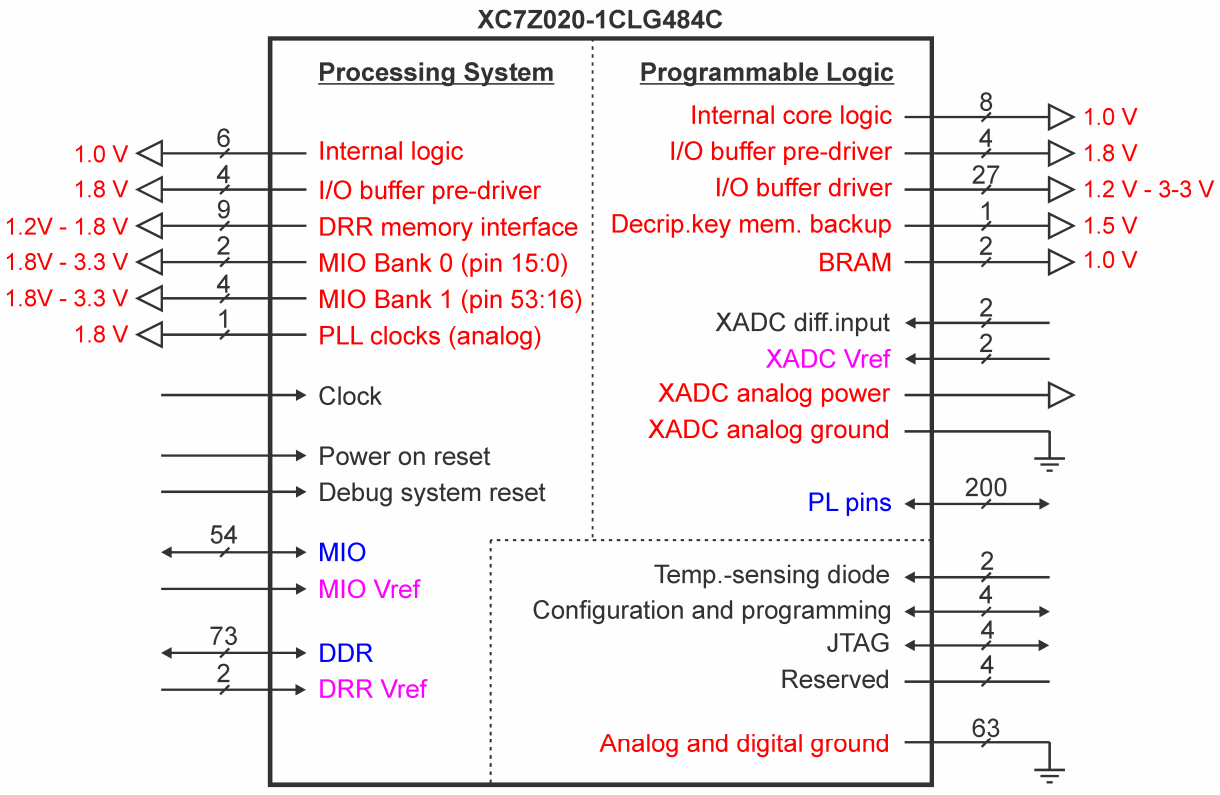
4 Električko spajanje SoC sklopa

4.1 Vanjski priključci SoC sklopa

- primjer - Zynq sklop srednje veličine
  XC7Z020-1CLG484C
- kućište CLG484
  - $22 \times 22 = 484$ priključka
  - BGA kućište
    - veličina kućišta 19 mm × 19 mm
    - razmak priključaka 0.8 mm, promjer kuglice 0.5 mm
    - označavanje priključaka (pogled odozdo)
      $\Rightarrow$ zakošen ugao - priključak A1

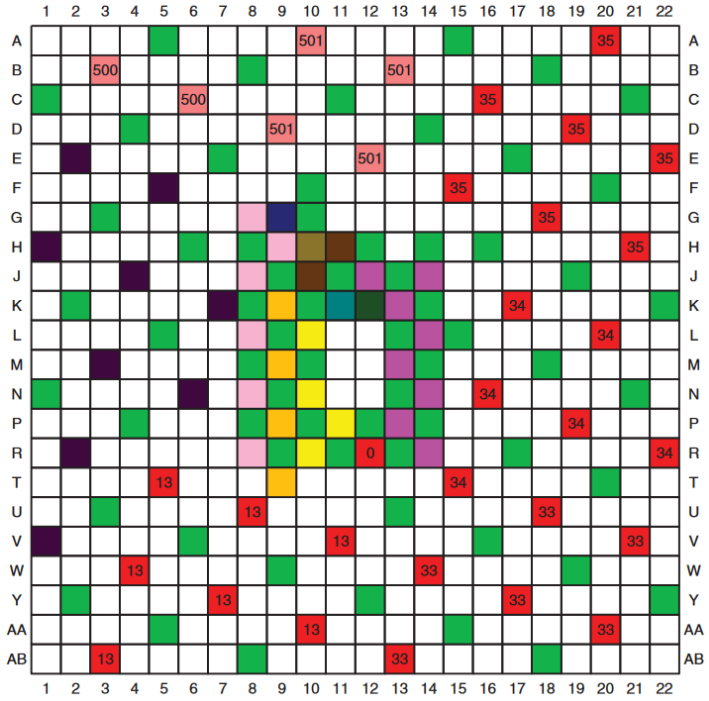


- spajanje



- funkcije priključaka
  - 131 priključak za masu i napajanje
  - 200 I/O priključaka na PL
  - 127 I/O priključaka na PS (zajedno s DDR-om)

- razmještaj priključaka za napajanje



GND
VCC I/O *buffer driver*
VCC MIO
itd.

- ulazno-izlazni priključci
  - 7z020 ne sadrži HP priključke, svi su HR
  - podijeljeni su u banke
    - PS
      - banka 0 - 16 priključaka
      - banka 1 - 38 priključaka
    - PL
      - 50 priključaka po banci
        ⇒ podržava 50 unipolarnih ili 24 diferencijalna signala
- svojstva banaka
  - vanjski priključci se povezuju s drugim komponentama
    ⇒ moraju zadovoljavati neki od standarda za spajanje
      - standard definira napone, struje, pragove i impedancije

    ⇒ banke pokrivaju mnoge standarde
    - najčešće se koriste LVCMOS I/O standard
      (*Low Voltage Complementary Metal Oxide Semiconductor*)

LVCMOS18	1.8 V
LVCMOS25	2.5 V
LVCMOS33	3.3 V

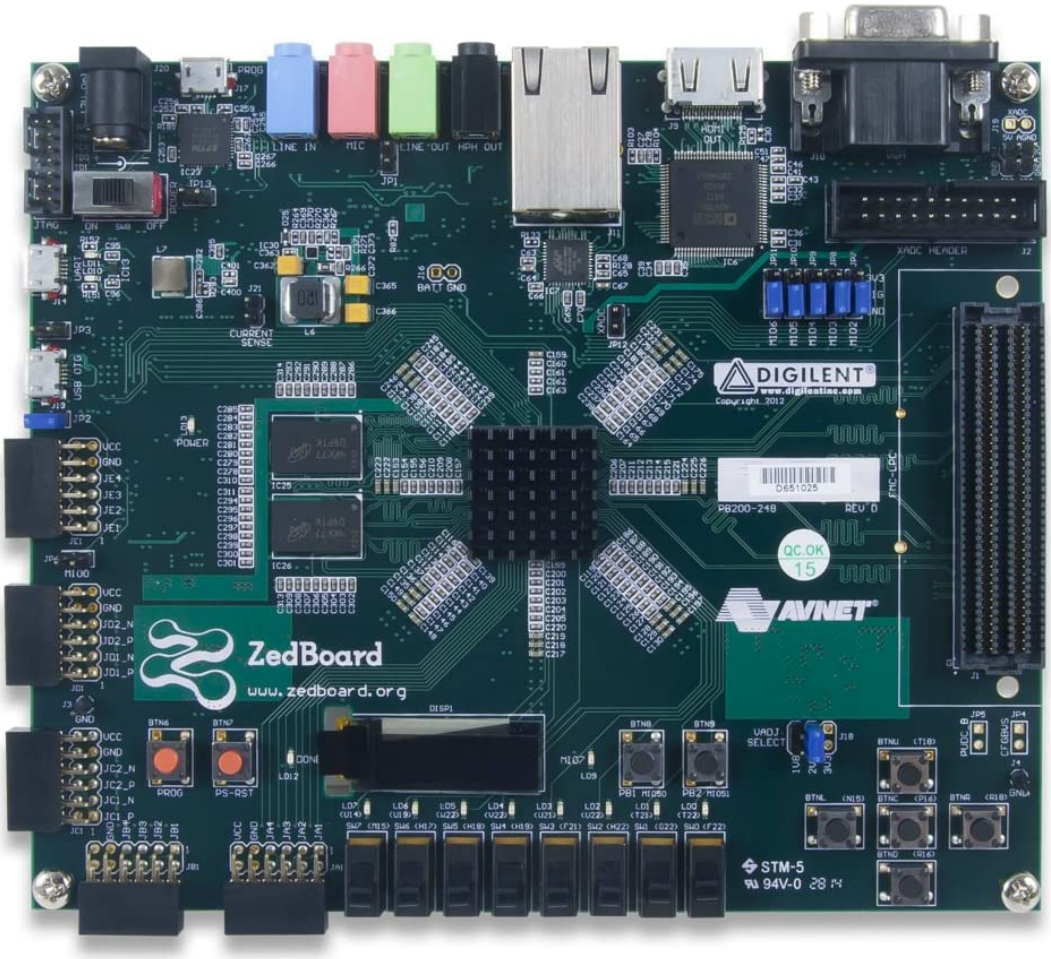
    - uočiti
      - svaka banka ima vlastito napajanje izlaznih stupnjeva
        ⇒ može raditi s drugim I/O standardom

4.2 Razvojni sustavi

- poznati razvojni sustavi SoC platformi
  - ZC706
    - XC7Z045T-2FFG900C
      ⇒dual core, 1 GHz
      ⇒218.6 k LUT-ova, 545 BRAM-ova, 900 DSP odsječaka
      - mnoštvo periferija, 2 FMC i 2 Pmod konektora
  - ZC702
    - XC7Z020-1CLG484C
      ⇒dual core, 667 MHz
      ⇒53.2 k LUT-ova, 140 BRAM-ova, 220 DSP odsječaka
    - mnoštvo periferija, 2 FMC i 3 Pmod konektora
  - ZedBoard
    - isti SoC kao na ZC702
    - razne periferije, 1 FMC i 5 Pmod konektora
    - niska cijena
  - Zybo
    - XC7Z010-CLG400
      ⇒dual core procesor, 650 MHz
      ⇒17.6 k LUT-ova, 60 BRAM-ova, 80 DSP odsječaka
    - mnoštvo periferija, 0 FMC i 6 Pmod konektora
    - vrlo niska cijena
  - MYD-C7Z015
- razvojni sustavi heterogenih platformi
  - Versal
- konektori
  - FMC - FPGA *Mezzanine Card*
    - 400 priključaka
    - za spajanje brzih vanjskih jedinica
    - podržava bipolarne signale
  - Pmod - *Peripheral module interface*
    - 12 priključaka
    - za spajanje jednostavnijih i sporijih vanjskih jedinica
- daljnji primjeri i primjeri na laboratorijskim vježbama bit će dani za ZedBoard

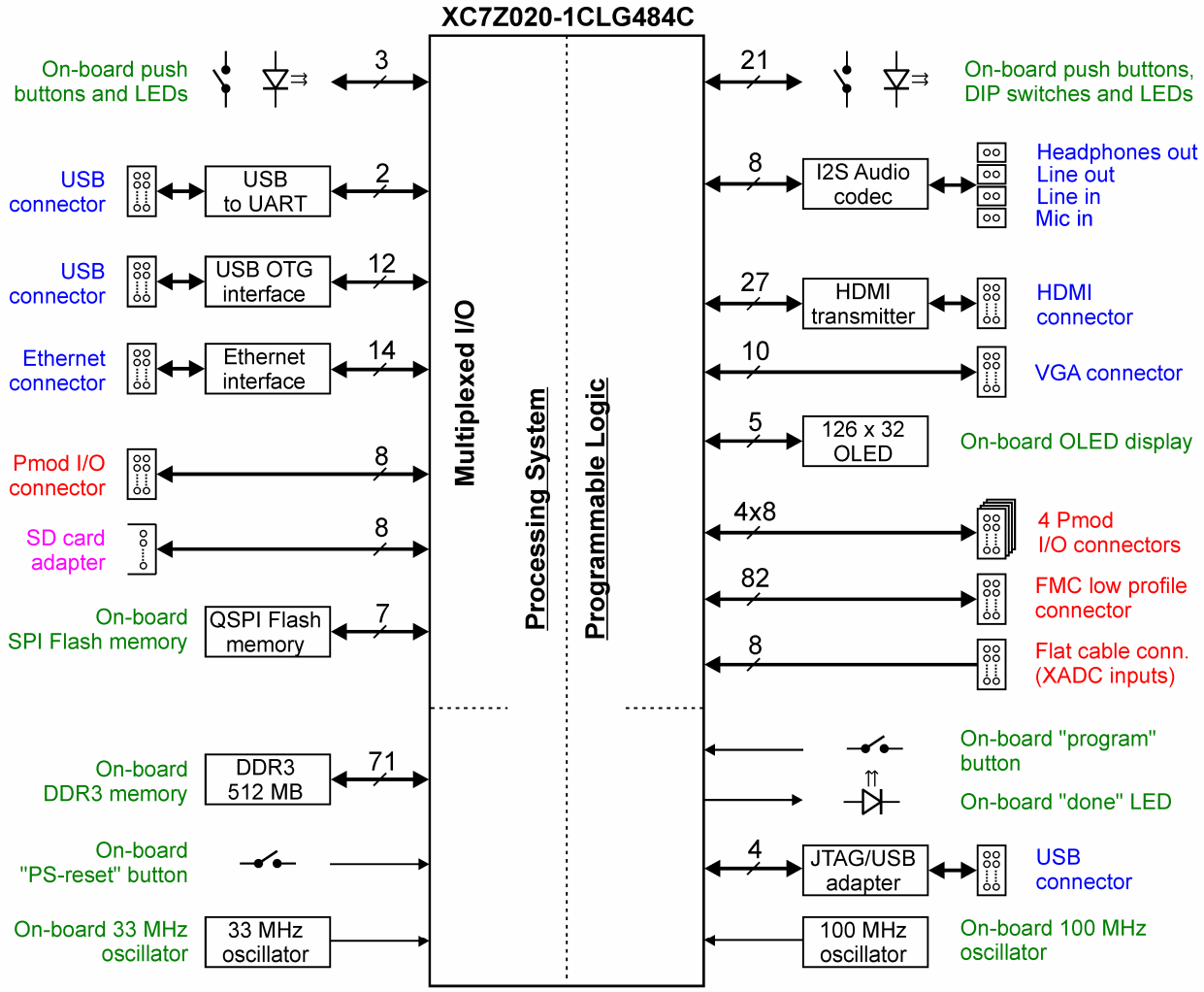
4.3 ZedBoard

- ZedBoard = *Zynq Evaluation & Development Board*
- fizički izgled



- tiskana pločica sadrži
  - Zynq sklop — XC7Z020-1CLG484C
  - memorije — DDR3, SPI Flash, SD *Card* sučelje
  - USB — OTG, UART, JTAG
  - video i audio ulaze i izlaze — HDMI, VGA, I2S audio kodek
  - digitalni pokaznik — OLED
  - Ethernet pogonski sklop
  - tipke, svjetleće diode
  - konektore za spajanje vanjskih sklopova:FMC, Pmod, AMS
  - izvore napajanja

- spoj periferija na Zynq



- uočiti
  - neki vanjski sklopovi spojeni su na PS
    ⇒ za njih je potrebno razviti programsku podršku za PS
  - neki vanjski sklopovi spojeni su na PL
    ⇒ za njih je potrebno razviti
    - sklopovlje za PL
    - programsku podršku za PS

– detaljan način spoj pojedinih periferija na sklop Zynq bit će dan u primjerima

4.4 Pitanja za provjeru znanja

1. Opisati spajanje napajanja na sklopove familije Zynq. Nabrojiti priključke s obzirom na njihovu funkciju.
2. Koje parametre obuhvaćaju naponski standardi za I/O priključke? Navesti često korištene standarde.
3. Opisati banke priključaka na procesorskom sustavu i programabilnoj logici sklopa Zynq (broj priključaka, tipovi signala, podjela u banke, napajanje i sl.).

4.5 Zadaci za samostalan rad

1. Potrebno je na Internetu potražiti sljedeći dokument:

   ZedBoard (Zynq Evaluation and Development) Hardware User's Guide, Version 2.2, 27 January 2014

   Pregledati strukturu ovog dokumenata s ciljem da se ovlada njegovim sadržajem, tj. da se u njemu brzo može pronaći tražene podatke. Proučiti dijelove koji su spomenuti na predavanju.

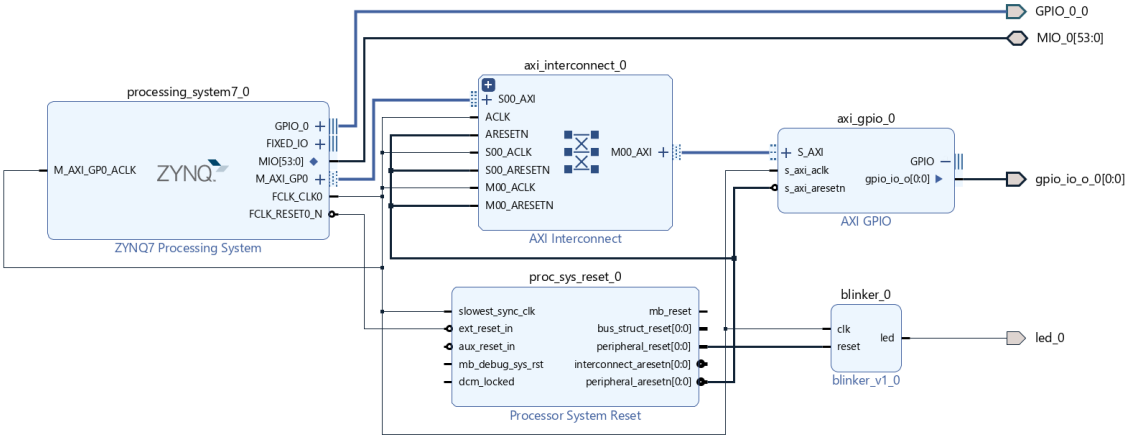
2. Potražiti na Internetu sheme razvojnog sustava ZedBoard. Proučiti kako su tipke i svjetleće diode spojene sa sklopom Zynq.

5 Dizajn temeljen na sklopovskim jezgrama

5.1 Osnovni pojmovi

- IP jezgra (*Intellectual Property Core*, IP *Core*)
    - razvijeni sklop koji se kasnije koristi kao gotova komponenta u FPGA ili ASIC dizajnu
    - primjeri:
        - procesor, UART, *Ethernet controller*
- vlasništvo nad IP jezgrom
    - proizvođač
        - npr. Xilinx Inc.
    - jezgre otvorenog koda (*open cores*)
        - promoviraju ih organizacije kao što je *Open Cores*
- vrste IP jezgri
    - meke jezgre (*soft cores*)
        - fleksibilne, konfigurabilne
        - mogu se implementirati na praktički bilo koju platformu
        - primjeri:
            - VHDL model UART sklopa
            - spojna lista (*netlist*) logičkih sklopova koja opisuje UART
    - polutvrde jezgre (*firm cores*, *semi-hard cores*)
        - u sebi sadrže i neke implementacijske aspekte
            - npr. međusobni razmještaj komponenata na čipu
        - konfigurabilne su ali u pravilu su napravljene za određenu platformu (npr. Xilinx FPGA)
    - tvrde jezgre (*hard cores*)
        - potpuno prilagođene ugradnji na neku platformu
        - podržavaju samo rudimentarno konfiguriranje
- *low level design* (LLD)
    - podrazumijeva dizajn pojedine komponente sustava
    - rezultat LLD-a je IP Core dotične komponente
    - primjer
        - sklop se razvije u VHDL-u i "spakira" u IP jezgru
    - dobivena jezgra se poslije koristi u HLD-u

- *high level design* (HLD)
  - podrazumijeva dizajn cijelog sustava
  - koristi gotove IP jezgre
  - primjer



- uočiti
  - u HLD-u ne ulazimo u detalje pojedinih jezgri
    - npr. ne zanima nas unutrašnja izvedbu procesora nego ga samo koristimo
- uočiti
  - LLD => AZRDS i vrlo malo PURS
  - HLD => PURS

5.2 Razvoj ugradbenih računalnih sustava u SoC okruženju

- razvoj URS-a u SoC okruženju obuhvaća
  - razvoj sklopovlja
  - razvoj programske podrške

$\Rightarrow$ potrebna su dva (međusobno povezana) alata

- povijesni razvoj alata
  - Xilinx ISE/EDK
    $\Rightarrow$ samo za starije FPGA platforme
  - Vivado/SDK
    $\Rightarrow$ za platforme koje sadrže SoC i novije FPGA
    – ne podržava starije FPGA platforme
  - Vivado/Vitis
    $\Rightarrow$ najnoviji skup alata
    $\Rightarrow$ podržava SoC i Versal

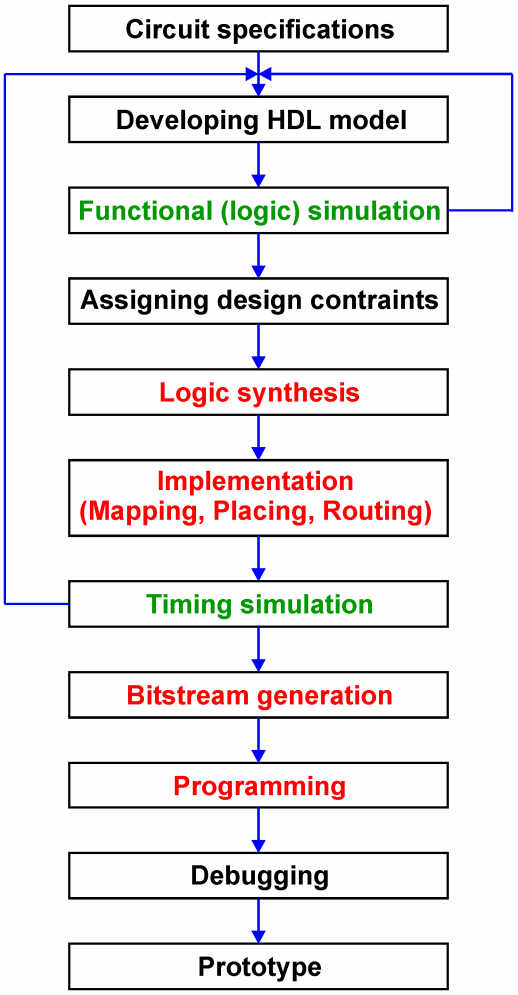
- verzije alata Vivado/Vitis
  - 2012. godina - prva verzija Vivado/SDK
  - **2020.2** - verzija koju ćemo mi koristiti
  - nova verzija izlazi dvaput godišnje
    - novije verzije su kompleksnije i trebaju biše prostora na disku
    - njihov razvoj usmjeren je na novije platforme

5.3 Vivado

- Vivado
  - alat za razvoj sklopovlja
  - namijenjen prvenstveno za HLD
  - podržava i razvoj IP jezgri, tj. LLD

5.3.1 Tijek razvoja sklopovlja

- **LLD** - faze razvoja sklopovlja (*Hardware Design Flow)*



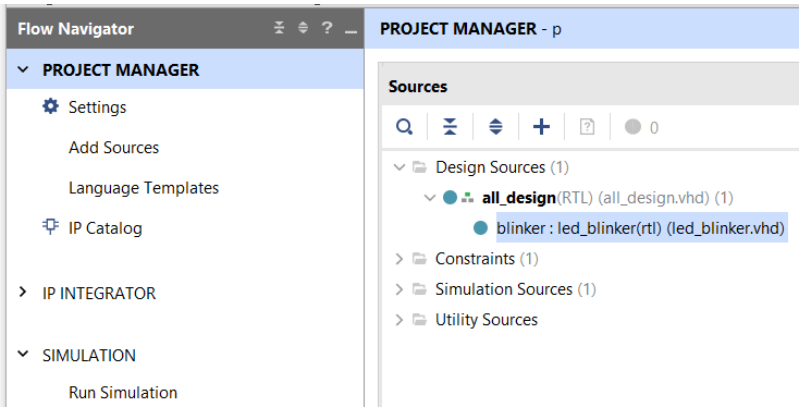
- crno označene faze
  ⇒ dizajner (čovjek)
- crveno označene faze
  ⇒ alat
- zeleno označene faze
  ⇒ alat provodi simulaciju
  ⇒ dizajner provodi analizu

- razvoj HDL modela (*developing HDL model*)
  - pojedini entiteti (komponente, podsustavi) modeliraju se pomoću jezika za opis sklopovlja
  - entiteti su povezani hijerarhijski
    ⇒jednostavnije komponente su ugrađene u složenije
  - krovni model
    - najviši entitet u hijerarhiji
    - predstavlja cijeli digitalni sustav
    - obično ima poseban naziv
      wrapper.vhd, all_design.vhd i sl.
- funkcijska (logička) simulacija (*functional (logic) simulation*)
  - provodi se za svaku novu komponentu u hijerarhiji
  - ne uključuje kašnjenja, komponente su idealizirane
- definiranje logičkih i fizičkih ograničenja (*assigning design constr.*)
  - postavljanje zahtjeva na frekvenciju takta, propagacije i sl.
  - pridruživanje fizičkih priključaka signalima krovnog entiteta
  - rezultat
    XDC datoteka
- logička sinteza (*logic synthesis*)
  - pretvaranje RTL modela u listu povezanih standardnih logičkih sklopova
  - koristi se baza sklopova koju daje proizvođač
    invertori, I vrata, bistabili, multipleksori, i sl.
  - rezultat
    "gate-level" lista (*gate-level netlist*)
- implementacija (*implementation*)
  - sadrži tri faze
    - mapiranje liste na ciljanu platformu (*mapping*)
      ⇒prilagođavanje liste koju daje sinteza konkretnom sklopovlju na kojem se radi implementacija
    - razmještanje elemenata iz liste na čip (*placing*)
      ⇒kod razmještanja se optimira duljina vodova, propagacije, utrošena površina itd.
    - električko povezivanje elemenata (*routing*)
- vremenska simulacija (*timing simulation*)
  - uključuje stvarna kašnjenja modela implementiranog sklopa
- generiranje datoteke za programiranje (*bitstream generation*)
  - rezultat
    datoteka za programiranje FPGA sklopa (*bitstream*)

- **generiranje sklopovske jezgra** (*IP core*)
  - krovni ili neki u hijerarhiji niži entitet može se "spakirati" u IP jezgru
- **HLD** - u Vivadu se koristi i naziv blokovski dizajn (*block design*)
  - koristi gotove jezgre kao komponente na shemi
  - faze razvoja sklopovlja
    - umjesto razvoja RTL modela crta se shema
    - sve ostale faze su iste kao i kod LLD
  - jezgra se može kreirati i iz blokovskog dizajna

5.3.2 Načini rada

- tijekom rada s Vivadom korisnik zadaje komande
  - sve komande napisane su u jeziku koji se naziva
    Tool Command Language (Tcl)
  - Tcl datira iz 1990. godine
  - danas se koristi kao standardno sučelje raznih alata za dizajn
- razlikujemo dva načina rada
  - projektni način rada (*project mode*)
    - lakše korištenje
      $\Rightarrow$primjereno početniku
    - komande se unose pomoću grafičkog sučelja
      (*Vivado Integrated Design Environment, Vivado IDE*)



  - dizajn proces je automatiziran
    - generiraju se mape i podmape koje sadrže datoteke projekta
    - svi rezultati pojedinih operacija pohranjeni su u datoteke
    - ako korisnik nakon sinteze promijeni dizajn, alat će upozoriti da treba ponoviti sintezu
    - itd.

- izravan način rada (*non-project mode*)
  - zahtijeva više iskustva
  - komande (i skripte) se unose "ručno", preko TCL konzole
  - rezultati pojedinih operacija nalaze se u memoriji
    ⇒ brže izvođenje
    ⇒ traži ručnu pohranu i sl.
  - primjeri komandi
    read_vhdl ...
    read_xdc ...
    synth_design ...

- napomena
  - i u projektnom načinu rada ponekad je potrebno koristiti komande
  - primjer
    reset_project
    ⇒ resetira projekt na početne postavke
    ⇒ briše rezultate sinteze, simulacije i implementacije

- uočiti
  - bez obzira koji se način rada koristi, alat uvijek u pozadini izvodi Tcl komande
    - komande koje je Vivado izvršio mogu se pronaći u datoteci **vivado.jou** (*journal file*)

- u PURS-u ćemo koristi pretežno projektni način rada

- u daljnjem tekstu obrađeni su sljedeći primjeri
  - razvoj sklopovlja u HDL okruženju
  - kreiranje jezgri
  - razvoj sklopovlja temeljen na jezgrama

5.4 Razvoj sklopovlja u HDL okruženju

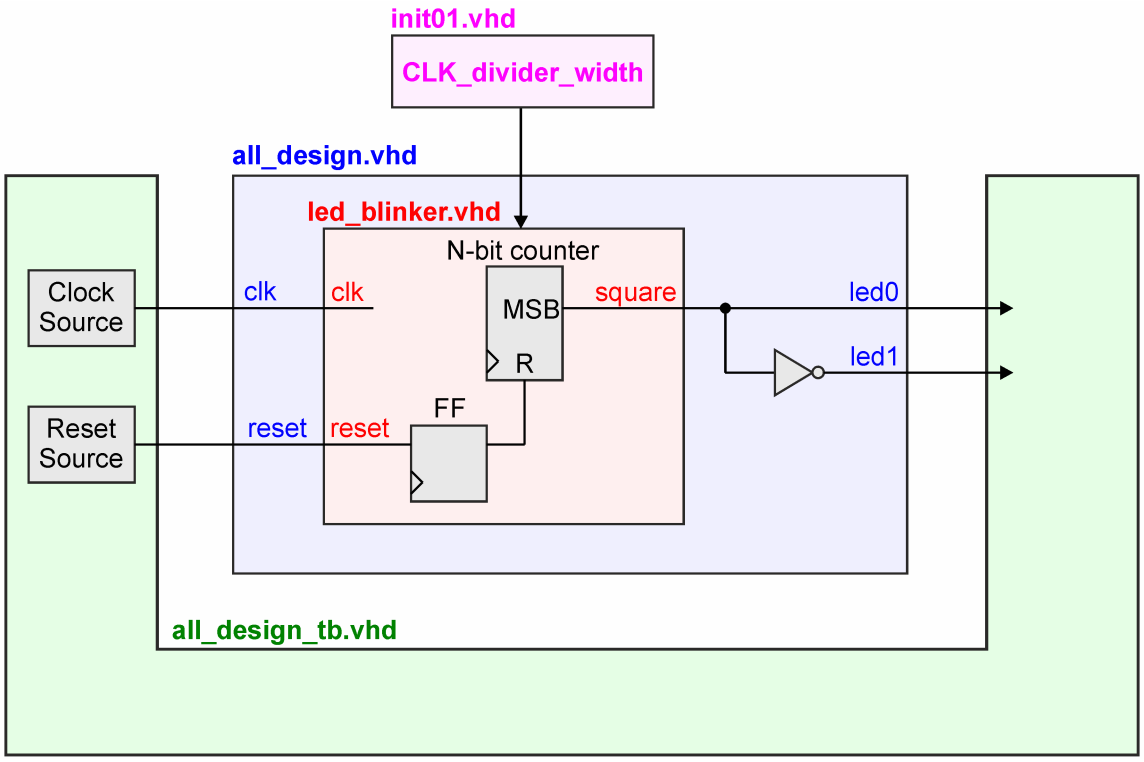
Primjer

Potrebno je razviti VHDL model sklopa koji na svom izlazu daje pravokutni signal perioda između 1 s i 2 s. Pretpostaviti da sklop radi na taktu frekvencije 100 MHz. Sklop mora imati reset ulaz čije aktivno stanje na izlazu daje logičku 0. Osigurati ispravan reset pod pretpostavkom da do njega ne dolazi sinkrono s taktom.

Razvijeni sklop upotrijebiti u krovnom modelu koji u protufazi pali i gasi dvije svjetleće diode. Reset signal dovesti s tipke. Implementirati krovni model na sustavu ZedBoard.

Rješenje

- uočiti
  - radi se isključivo dizajn sklopovlja u PL
  - ne koristi se PS
  - bit će korištene sljedeće vanjske komponente
    - 2 svjetleće diode
    - 1 tipka
    - izvor takta
- hijerarhija entiteta



- entiteti i paketi

led_blinker.vhd

- generator pravokutnog signala period između 1 s i 2 s
  - generički entitet
    - ⇒broj bitova brojila, N, je generička konstanta

all_design.vhd

- krovni model
- predstavlja sadržaj cijelog FPGA (ili SoC) sklopa

all_design_tb.vhd

- predstavlja ispitno okruženje
- sadrži
  - generiranje pobudnih signala
  - jedinku komponente koju se ispituje
    - ⇒UUT, DUT (*Unit Under Test*, *Device Under Test*)
  - analizu odziva komponente

init01.vhd

- sadrži paket s generičkim parametrima sklopova

⇒konfigurira sve komponente u entitetu all_design

⇒u ovom primjeru sadrži širinu brojila u led_blinker-u

5.4.1 Izvor pulsova

- izvor pulsova
  - zadano

f_{clk} = 100 MHz
T = 1 s do 2 s

⇒N-bitno sinkrono brojilo

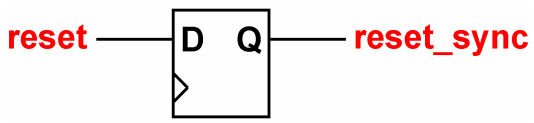
$f_{square} = f_{clk}/2^N \Rightarrow N = \log_2(f_{clk}/f_{square})$

$N_{1s} = \log2(100e6/1) = 26.58$
$N_{2s} = \log2(100e6/2) = 25.57$

⇒N = 26 (cijeli broj)

- sinkronizacija reseta
  - signali na ulazima bistabila moraju biti stabilni u intervalu [t_{setup}, t_{hold}]
  - ako reset postane aktivan unutar ovog intervala, neki bistabili u brojilu će biti resetirani, a neki neće

⇒ulazni reset signal potrebno je prije daljnjeg korištenja sinkronizirati s taktom na zasebnom (jednom) bistabilu



- VHDL model sklopa

```
-- LED BLINKER

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity led_blinker is
   generic (CLK_divider_width : integer range 2 to 32 := 32);
            -- CLK_divider_width =  3 for simulation
            -- CLK_divider_width = 26 for implementation on ZedBoard
   port(
        clk    : in  std_logic;
        reset  : in  std_logic;
        square : out std_logic
        );
end led_blinker;

architecture RTL of led_blinker is

   signal clk_divider: std_logic_vector(CLK_divider_width-1 downto 0);
   signal reset_sync : std_logic;

begin

   -- Synchronization of input signals
   process (clk) is
   begin
      if rising_edge(clk) then
         reset_sync <= reset;
      end if;
   end process;

   -- Synchonous counter
   process (clk) is
   begin
      if rising_edge(clk) then
         if reset_sync='1' then
            clk_divider <= (others => '0');
         else
            clk_divider <= clk_divider + 1;
         end if;
      end if;
   end process;

   square <= clk_divider(CLK_divider_width-1);

end rtl;
```

5.4.2 Krovni model

```
-- ALL DESIGN

library ieee;
use ieee.std_logic_1164.all;

library work;
use work.init01.all;

entity all_design is
   port (clk   : in  std_logic;
         reset : in  std_logic;
         led0  : out std_logic;
         led1  : out std_logic);
end all_design;


architecture RTL of all_design is

   component led_blinker is
   generic (CLK_divider_width : integer range 2 to 32);
   port(clk    : in  std_logic;
        reset  : in  std_logic;
        square : out std_logic);
   end component;

   signal led_blikner_out : std_logic;

begin

   blinker: component led_blinker
      generic map(CLK_divider_width => CLK_divider_width)
      port map (clk    => clk,
                reset  => reset,
                square => led_blikner_out);

   led0 <= led_blikner_out;
   led1 <= not led_blikner_out;

end RTL;
```

5.4.3 Parametri sklopa

```
package init01 is

   -- CLK_divider_width =  3 for simulation
   -- CLK_divider_width = 26 for implementation on ZedBoard
   constant CLK_divider_width : integer range 2 to 26 := 26;

end package init01;
```

5.4.4 Ispitno okruženje

```
-- ALL DESIGN_TB

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity all_design_tb is
end all_design_tb;

architecture Behavioral of all_design_tb is

   constant Tclk : time := 10 ns;  -- Period of 100 MHz clock

   component all_design is
      port (clk    : in  std_logic;
            reset  : in  std_logic;
            led0   : out std_logic;
            led1   : out std_logic);
   end component;

   signal clk   : std_logic := '0';
   signal reset : std_logic;
   signal led0  : std_logic;
   signal led1  : std_logic;

begin

   -- generation f input signals
   clk <= not clk after Tclk/2;
   reset <= '1', '0' after 5.25*Tclk;

   -- instance od unit under test
uut: component all_design
      port map (clk   => clk,
                reset => reset,
                led0  => led0,
                led1  => led1);

   -- analysis of output signals
   --    => nor needed here because of circuit's simplicity
   --    => visual check of waveforms is perfomed after simulation

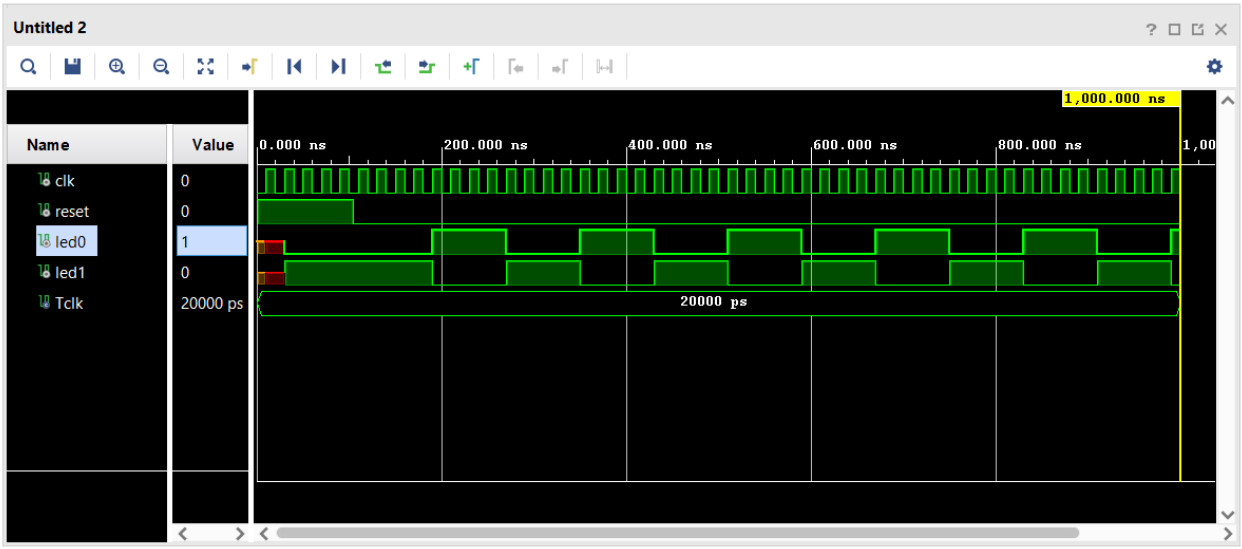
end Behavioral;
```

5.5 Simulacija

- kod simulacije praktično je smanjiti širinu brojila
    - paket init01

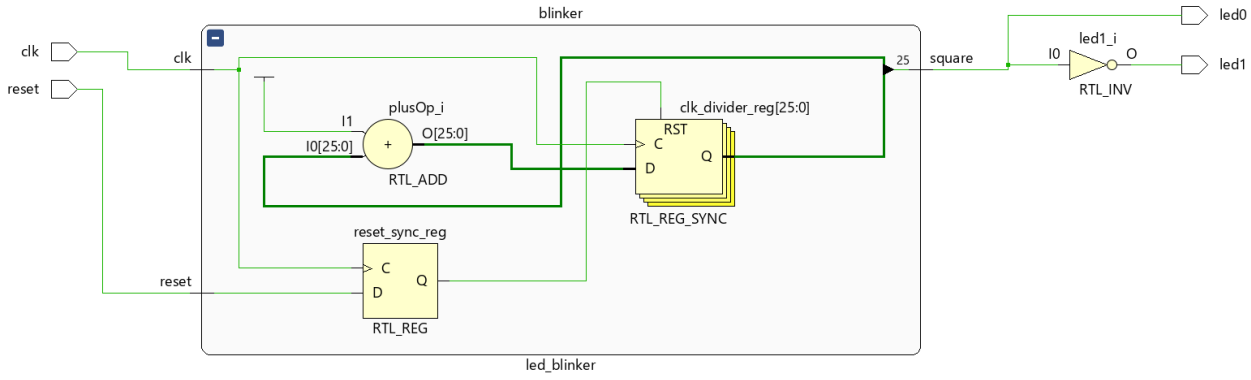
```
-- CLK_divider_width =  3 for simulation
-- CLK_divider_width = 26 for implementation on ZedBoard
constant CLK_divider_width : integer range 2 to 26 := 3;
```

- primjer
    - simulacija krovnog entiteta (all_design)



5.6 RTL analiza

- neki alati mogu prikazati shemu koju predstavlja RTL model
- primjer
    - shema krovnog entiteta (all_design)



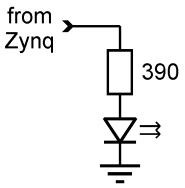
5.7 Implementacija na sustavu ZedBoard

- vidi poglavlje "4.3 ZedBoard"
  ⇒u programabilnu logiku ulazi takt frekvencije 100 MHz
  ⇒na Zynq su spojene svjetleće diode i tipke
- detalji se nalaze u
  - ***ZedBoard Hardware User's Manual***
  - *Zynq-7000 SoC Packaging and Pinout Product Specif., UG685*
  - spoj takta na Zynq

Subsystem	Signal Name	Zynq pin	Bank
PL	GCLK	Y9	13
PS	PS_CLK	F7	500

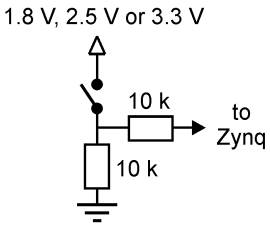
  - spoj svjetlećih dioda na Zynq

Subsystem	Signal Name	Zynq pin	Bank
PL	LD0	T22	33
	LD1	T21	33
	LD2	U22	33
	LD3	U21	33
	LD4	V22	33
	LD5	W22	33
	LD6	U19	33
	LD7	U14	33
PS	LD9	D5 (MIO 7)	500



  - spoj tipki na Zynq

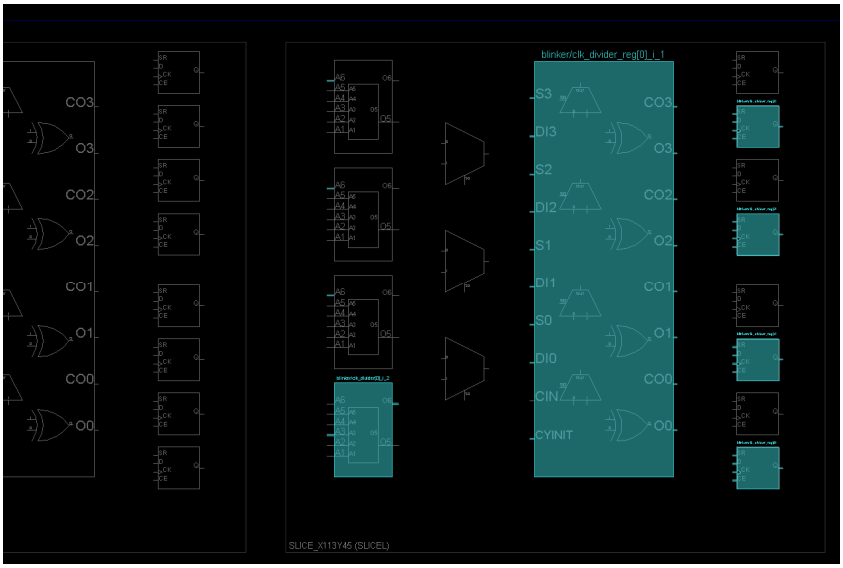
Subsystem	Signal Name	Zynq pin	Bank
PL	BTNU	T18	34
	BTNR	R18	34
	BTND	R16	34
	BTNC	P16	34
	BTNL	N15	34
PS	PB1	D13 (MIO 50)	501
	PB2	C10 (MIO 51)	501



  - naponi na bankama priključaka Zynqa

Subsystem	Bank	Connected voltage	Jumper
PS	500 (MIO Bank 0)	3.3 V	-
	501 (MIO Bank 1)	1.8 V	-
	DDR	1.5 V	-
PL	0	3.3 V	-
	13	3.3 V	-
	33	3.3 V	-
	34	1.8 V, 2.5 V, 3.3 V	J18
	35	1.8 V, 2.5 V, 3.3 V	J18

- spoj signala na priključke Zynqa
  - takt
    - ⇒takt od 100 MHz spojen je na priključak Y9 banke 13
    - ⇒banka 13 je fiksno spojena na 3.3 V
      - ⇒I/O standard LVCMOS33
  - svjetleće diode
    - bit će korištene diode LD0 i LD1
      - ⇒LD0 i LD1 spojene su na priključke T22 i T21 banke 33
      - ⇒banka 33 je fiksno spojena na 3.3 V
        - ⇒I/O standard LVCMOS33
      - ⇒(3.3 V – 1.8 V) / 390 Ω = 3.8 mA
        - ⇒Drive Strength = 4
          (ne smeta ako ostane *default* vrijednost 12)
      - ⇒brzina porasta
        - ⇒svjetleća dioda ⇒ SLOW (NONE)
      - ⇒pritezni otpornici i zaključenja
        - ⇒NONE
  - tipka
    - bit će korištena tipka BTND
      - ⇒tipka BTND spojena je na priključak R16 banke 34
      - ⇒banka 33 spojena je preko J18 1.8 V, 2.5 V ili 3.3 V
        - ⇒spojiti premosnik J18 u srednji položaj (2.5 V)
          - ⇒I/O standard LVCMOS25
- implementacija u Vivadu je automatiziran postupak
- primjer - pogled na programabilnu logiku
  - dio resursa utrošenih za brojilo u entitetu led_blinker (donja 4 bita)



5.8 Kreiranje jezgri

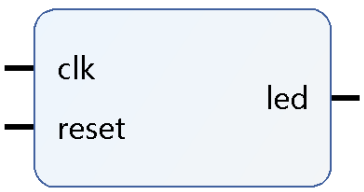
- podsjetimo se
  - IP jezgra
    - razvijeni sklop koji se kasnije koristi kao gotova komponenta u FPGA ili ASIC dizajnu
- za opis jezgri koristi se IP-XACT standard
  - razvio ga je SPIRIT Consortium
    - čine ga kompanije koje se bave razvojem SoC komponenti i alata
      - ARM, Cadance, Mentor Graphics, STM, TI, ...
  - jezgra se opisuje u XML formatu
    - XML (*Extensible Markup Language*)
      - jezik za opis raznih oblika podataka
      - prilagođen je čovjeku i stroju

Primjer

Sklop led_blinker razvijen u prethodnom primjeru potrebno je realizirati u obliku IP jezgre.

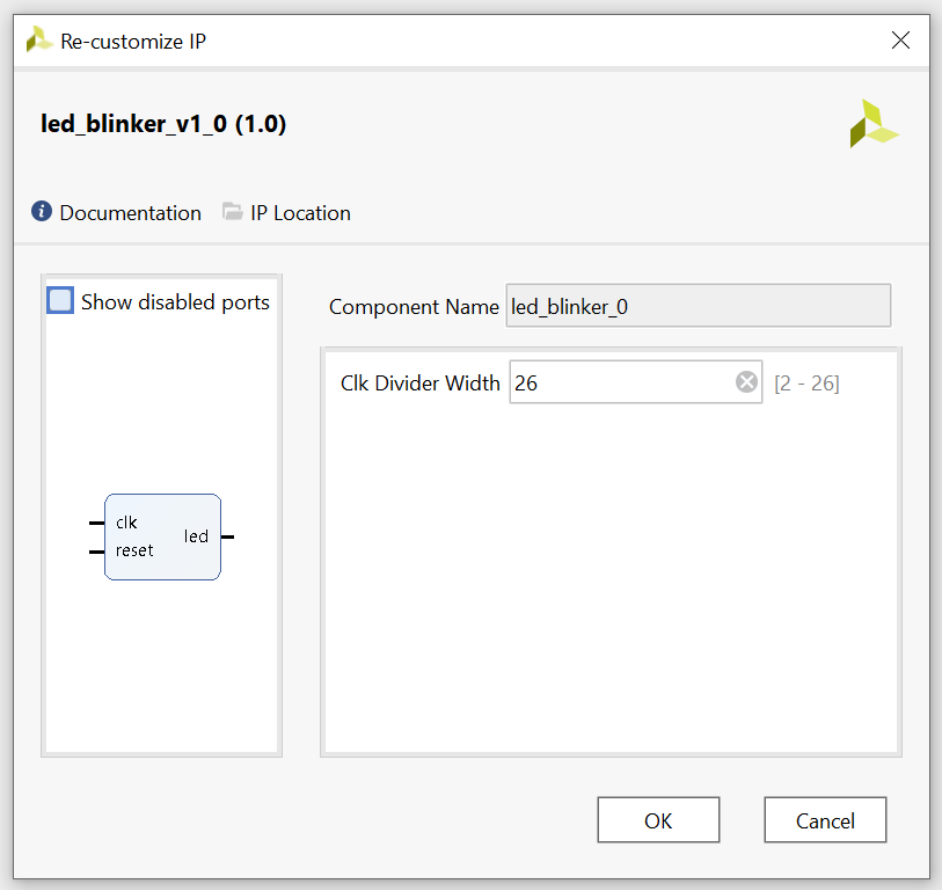
Rješenje

- Vivado sadrži alate koji razvijeni entitet pakiraju u IP jezgru
  - rezultat pakiranja je jezgra
    - .zip datoteka ⇒ ESDL_user_led_blinker_1.0 (ili slično)
- sadržaj jezgre (.zip datoteke)
  - opis komponente prema IP-XACT ⇒ component.xml
    - sadrži opis jezgre
      - ime, proizvođač, opis, ...
      - platforme s kojima je kompatibilna (Artix, Zynq, ...)
      - opis priključaka i generičkih konstanti
      - grafički izgled



      - itd.

- VHDL kod komponente ⇒ led_blinker.vhd
- Tcl datoteke za podešavanje parametara jezgre (*customization*)
  ⇒ xgui (mapa)
  - sadrži komande koje generiraju prozor za konfiguriranje
    - prozor za konfiguriranje dobiva se kad se jezgra koristi u nekom dizajnu (dvostruki klik miša na jezgru)



- dodatne informacije
  - dokumentacija, podaci za simulaciju i sl.
  – u našem primjeru nema dodatnih informacija

- jezgra (.zip datoteka) sada se može isporučiti korisniku
  ⇒korisnik ju kao gotovu komponentu ugrađuje u svoj dizajn
  ⇒vidi daljnji tekst

5.9 Razvoj sklopovlja temeljen na jezgrama

- HLD

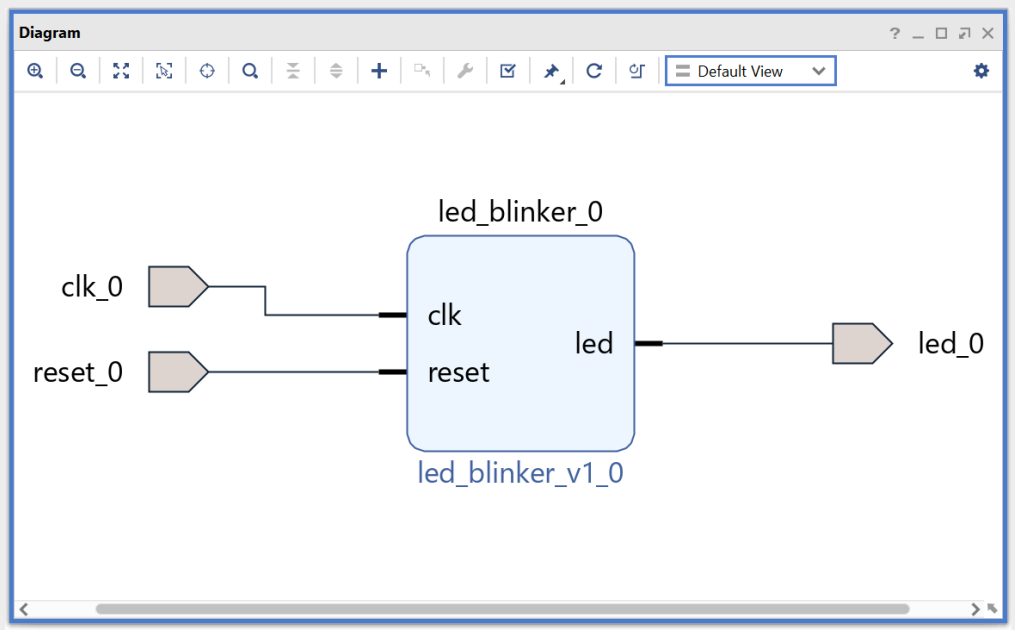
  ⇒blokovski dizajn

Primjer

Korištenjem jezgre razvijene u prethodnom primjeru, razviti sklop koji na razvojnom sustavu ZedBoard pali i gasi svjetleću diodu. Koristiti vanjski izvor takta frekvencije 100 MHz. Reset signal generirati pomoću tipke.

Rješenje

- uključivanje jezgre u dizajn
  - novu jezgru treba staviti u odgovarajuću mapu (IP *Repository*)
  - jezgru treba uključiti u katalog jezgri (IP *Catalog*)
- izrada sheme
  - crtanje sheme



  - podešavanje parametara jezgre ⇒ CLK *Divider Width*
- generiranje krovnog modela
  - shema obično sadrži veći broj komponenata (jezgri)

    ⇒sve komponente (cijelu shemu) potrebno je obuhvatiti zajedničkim HDL krovnim modelom (*wrapper*)
    - ovaj postupak je automatiziran

- krovni model u našem primjeru

```
-- led_blinker_wrapper

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
library UNISIM;
use UNISIM.VCOMPONENTS.ALL;

entity led_blinker_wrapper is
  port (clk_0 : in STD_LOGIC;
        led_0 : out STD_LOGIC;
        reset_0 : in STD_LOGIC);
end led_blinker_wrapper;

architecture STRUCTURE of led_blinker_wrapper is

  component led_blinker is
  port (led_0 : out STD_LOGIC;
        clk_0 : in STD_LOGIC;
        reset_0 : in STD_LOGIC);
  end component led_blinker;

begin

led_blinker_i: component led_blinker
     port map (clk_0 => clk_0,
               led_0 => led_0,
               reset_0 => reset_0);

end STRUCTURE;
```

  - uočiti
    - korisnik razvija sklop na visokoj razini apstrakcije (shema)
    - u pozadini se sustav opisan jezikom za opis sklopovlja
    - biblioteka *unisim*
      - dizajn koristi komponente u FPGA LUT, BRAM, ...)

      $\Rightarrow$ *unisim* sadrži VHDL modele za njihovu simulaciju

- simulacija
  - izvodi se na sličan način kao u poglavlju 5.5
- RTL analiza
  - izvodi se na opisan u poglavlju 5.6
- implementacija
  - izvodi se na sličan način kao u poglavlju 5.7

5.10 Komercijalno dostupne jezgre

- proizvođači na tržištu nude svoje jezgre
  - Xilinx, ARM, ..., Xylon
- jezgre za Zynq okruženje
  - jezgra procesorskog sustava
    - ZYNQ7 Processing System
  - namijenjen implementaciji u PL
    - Processor System Reset
    - AXI Interconnect
    - MicroBlaze
    - FFT
    - FIR filtar
    - led_blinker :)
    - itd.
- uočiti
  - PS je fiksno ožičena komponenta
    ⇒uključuje se u dizajna kao blok
    - može se konfigurirati
  - plavo označene jezgre koriste se u radu s procesorom
    ⇒vidi sljedeća poglavlja

5.11 Pitanja za provjeru znanja

1. Objasniti pojmove: IP jezgra, *high level design* (HLD), *low level design* (LLD). Nabrojiti i opisati faze razvoje razvoja digitalnog sustava u slučaju kad se koristi *low level design* (LLD).
2. Objasniti razliku između projektnog i izravnog načina rada u alatima za razvoj sklopovlja. Što je *Tool Command Language*?

5.12 Zadaci za samostalan rad

1. Napraviti pripremu za laboratorijske vježbe 1, 2 i 3.

6 Računalni sustav u SoC okruženju

6.1 Tijek razvoja računalnog sustava na čipu

- 1. faza
  - razvoj sklopovlja
  - primjer alata: **Vivado**
- 2. faza
  - prijenos podataka o sklopovlju u okruženje za razvoj programske podrške
  - primjer: **XSA (*Xilinx Support Archive*) datoteka**
- 3. faza
  - razvoj programske podrške
  - primjer alata: **Vitis**

- **XSA** (*Xilinx Support Archive*) datoteka
  - datoteka koja sadrži informacije o sklopovlju koje su potrebne da se generira (programska) platforma
  - XSA generira Vivado
  - uočiti
    - XSA datoteka sadrži sve informacije koke Vitisu trebaju za razvoj programske podrške

    $\Rightarrow$ to je jedina veza Vivada prema Vitisu

Primjer

Potrebno je razviti sklopovlje i programsku podršku koji pali i gasi svjetleću diodu LD9 na razvojnom sustavu ZedBoard. Pristup periferijama realizirati izravno upisom u registre, bez pomoći upravljačkih programa.

Rješenje

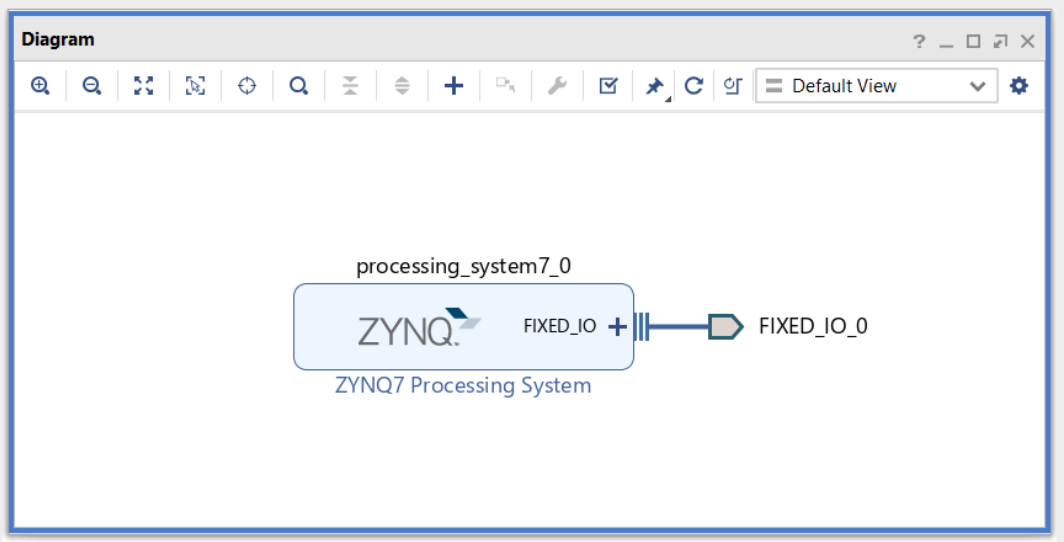
- potrebno je
  - razviti sklopovlje $\Rightarrow$ Vivado
  - prenijeti konfiguraciju sklopovlja u programsko okruženje

    $\Rightarrow$ XSA datoteke
  - razviti programsku podršku $\Rightarrow$ Vitis
  - ispitati rad na stvarnom sklopovlju $\Rightarrow$ ZedBoard

- sklopovlje
  - vidi poglavlje 5.7 Implementacija na sustavu ZedBoard
    ⇒svjetleća dioda LD9 spojena je preko MIO sučelja na priključak 7 sklopa GPIO
    ⇒potreban je samo PS
    ⇒treba konfigurirati MIO
    ⇒treba konfigurirati GPIO
- programska podrška
  - pisana bez korištenja upravljačkih programa
    ⇒izravan pristup registrima ⇒ *bare machine*
    ⇒treba napisati funkciju main()

6.2 Sklopovlje

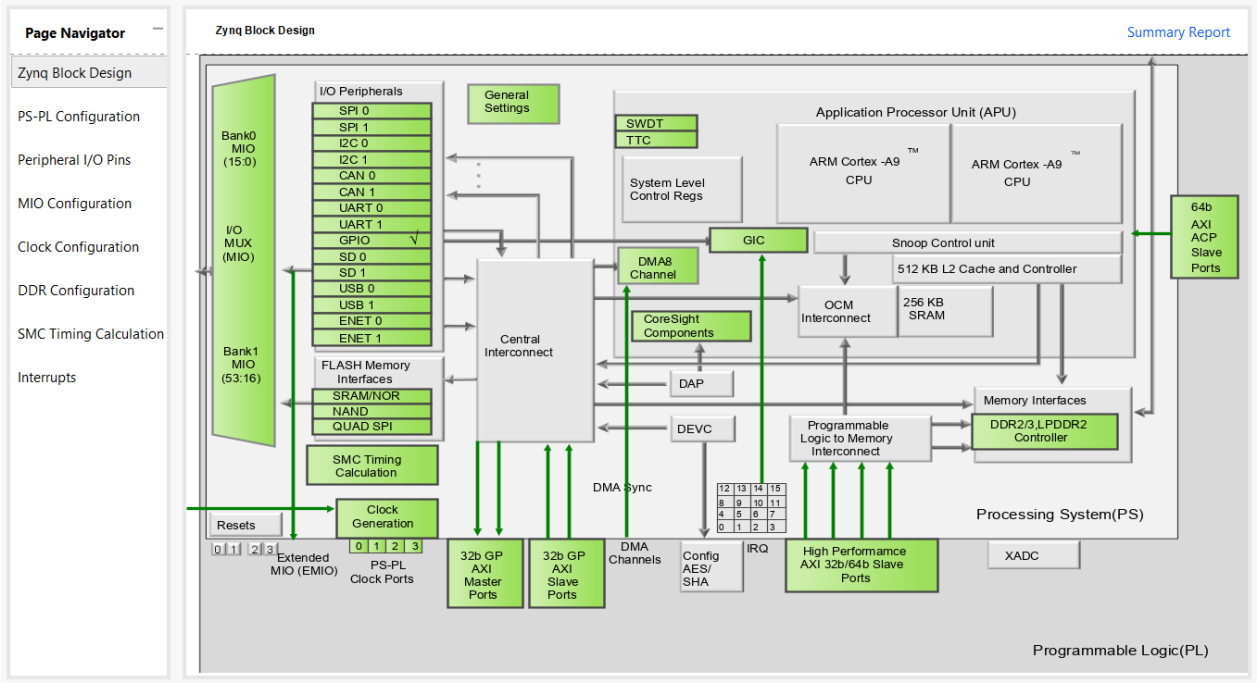
- bit će korišten HLD (blokovski dizajn)
- u ovom primjeru bit će korištena samo jedna jezgra
  ⇒ime jezgre: ZYNQ7 Processing System
  ⇒predstavlja cijeli procesorski sustav
- shema sklopovlja



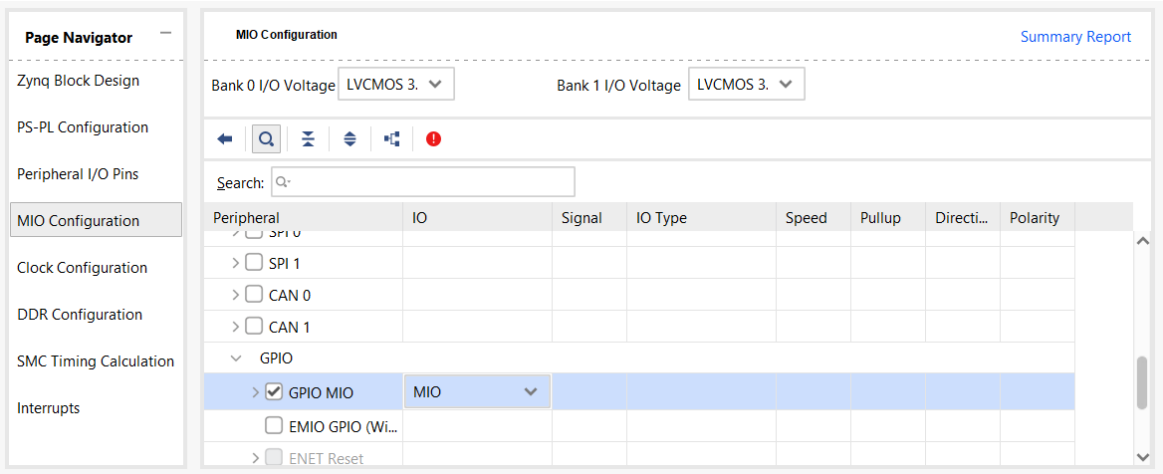
- uočiti
  - skup priključaka FIXED_IO
    ⇒obuhvaća linije koje su fizički uvijek spojene na vanjske priključke sklopa Zynq
    - na shemi ih je svejedno potrebno spojiti jer je takva filozofija rada s jezgrama

6.2.1 Procesorski sustav

- procesorski sustav dan je kao jezgra
  - pojedini dijelovi se mogu konfigurirati
    ⇒"dvostruki klik" na komponentu ZYNQ7 Processing System



- uočiti
  - ovo sučelje opisano je u XGUI (vidi poglavlje "5.8 Kreiranje jezgri")
  - ovo je pojednostavljena shema iz poglavlja 3.1
  - zelene komponente mogu se konfigurirati
    ⇒tome služi lijevi izbornik
    - primjer



– konfiguriranje će biti ilustrirano na laboratorijskim vježbama

6.2.2 Upravljačko sklopovlje

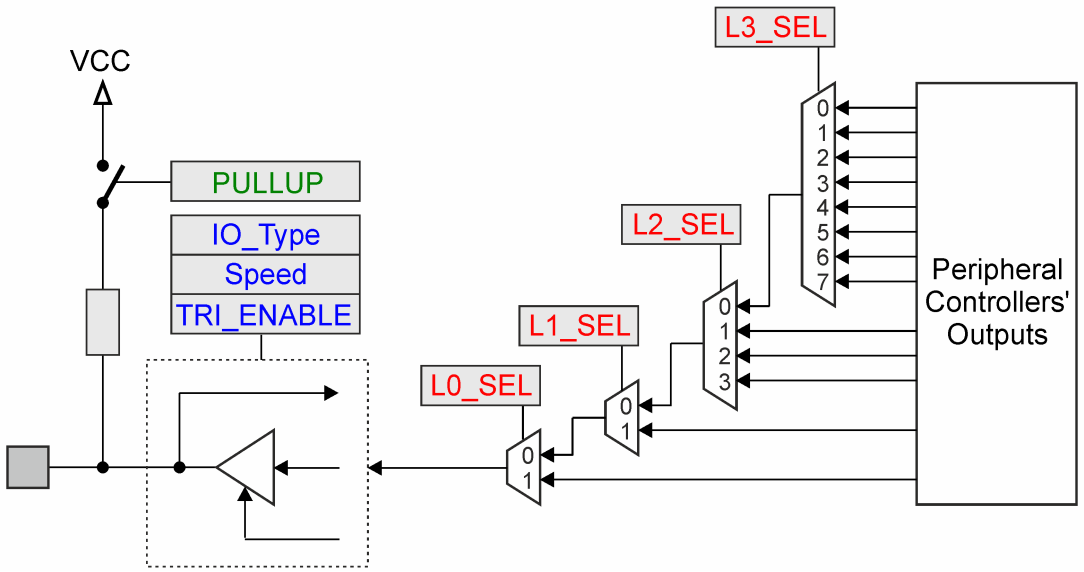
- SLCR

  ⇒služi za konfiguraciju pojedinih sklopova procesorskog sustava

- temeljne adrese i funkcije SLCR registara

Register Base Address	Description
F800_0000	SLCR write protection lock and security
F800_0100	Clock control and status
F800_0200	Reset control and status
F800_0300	APU control
F800_0400	TrustZone control
F800_0500	CoreSight SoC debug control
F800_0600	DDR DRAM controller
F800_0700	MIO pin configuration
F800_0800	MIO parallel access
F800_0900	Miscellaneous control
F800_0A00	On-chip memory (OCM) control
F800_0B00	I/O buffers for MIO pins (GPIOB) and DDR pins (DDRIOB)

- sklopovlje za odabir funkcija izlaznog priključka



- uočiti
  - broj MUX-ova koje signal prolazi od periferije do priključka ovisi o brzini periferije
    - signali najbržih periferija prolaze kroz jedan MUX
      - primjer - Ethernet kontroler
    - signali najsporijih periferija prolaze kroz četiri MUX-a
      - primjer - UART, CAN

- registri za upravljanje

  MIO_PIN_53 ..... MIO_PIN_00

  - polja u registrima

Bits	Field Name	Reset Value	Description
31:14	reserved	0x0	
13	DisableRcvr	0x0	Receiver status (if IO_Type is HSTL) 0: enable 1: disable
12	PULLUP	0x0	Enables Pullup on IO Buffer pin 0: disable 1: enable
11:9	IO_Type	0x3	Select the IO Buffer Type. 000: Reserved 001: LVCMOS18 010: LVCMOS25 011: LVCMOS33 100: HSTL 101: Reserved 110: Reserved 111: Reserved
8	Speed	0x0	IO Buffer Edge Rate (if IO_Type is LVCMOSxx) 0: Slow CMOS edge 1: Fast CMOS edge
7:5	L3_SEL	0x0	Level 3 Mux Select
4:3	L2_SEL	0x0	Level 2 Mux Select
2	L1_SEL	0x0	Level 1 Mux Select
1	L0_SEL	0x0	Level 0 Mux Select
0	TRI_ENABLE	0x1	Tri-state enable 0: disable (pin is on) 1: enable (peripheral controls three state)

  - uočiti
    - upravljanje svojstvima priključka
      ⇒DisableRcvr, PULLUP, IO_Type, Speed, TRI_ENABLE
    - odabir funkcije priključka
      ⇒L3_SEL, L2_SEL, L1_SEL, L0_SEL

- podsjetimo se
  - LD9 spojena je preko MIO na GPIO7

- odabir funkcije priključka GPIO7

  ⇒registar MIO_PIN_07

Bits	Field Name	Reset Value	Description
	...		
7:5	L3_SEL	0x0	000: GPIO 7 (bank 0), Output-only others: reserved
4:3	L2_SEL	0x0	00: Level 3 Mux 01: SRAM/NOR OE_B, Output 10: NAND Flash CLE_B, Output 11: SDIO 1 Power Control, Output
2	L1_SEL	0x0	0: Level 2 Mux 1: Trace Port Data Bit 13, Output
1	L0_SEL	0x0	0: Level 1 Mux 1: reserved
	...		

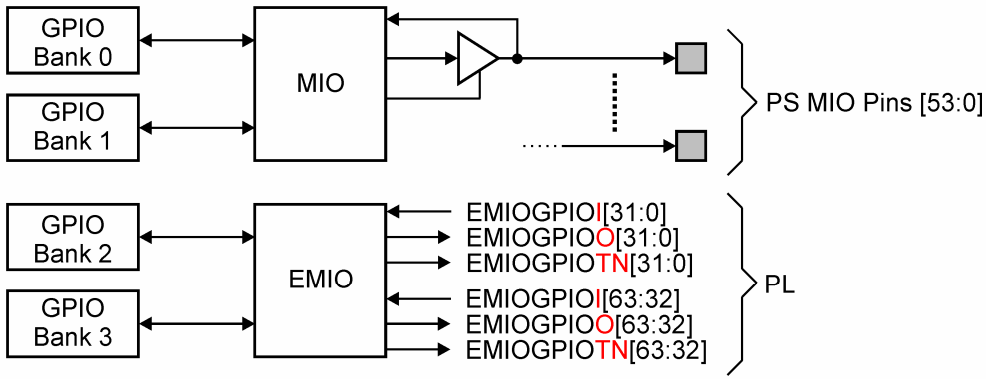
- uočiti
  - Lx_SEL linije
    - nakon reseta sve četiri SEL linije su u stanju 0

      ⇒odabran je GPIO
  - stanje polja TRI_ENABLE
    - nakon reseta linija je u stanju 1

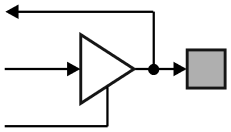
      ⇒periferija može upravljati stanjem visoke impedancije
  - ovako se ponašaju i preostali GPIO priključci

6.2.3 GPIO

- GPIO služi za upravljanje priključcima (signalima) opće namjene
- spoj GPIO s MIO i EMIO

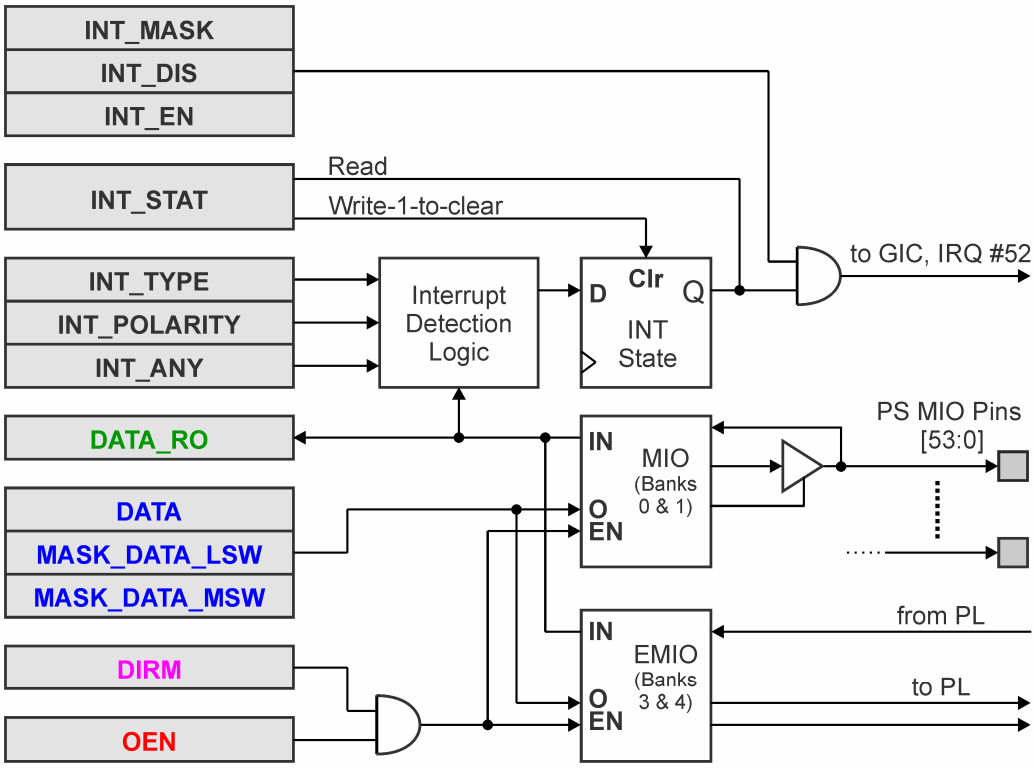


- uočiti
  - GPIO se sastoji od 4 (logičke) banke
    - banka 0 i banka 1
      - spojene su na vanjske priključke Zynqa preko MIO
        - banka 0 (upravlja vanjskim priključcima 31:0)
        - banka 1 (upravlja vanjskim priključcima 53:32)
      - svaki priključak može biti
        - ulazni (*input*)
        - izlazni (*output*)
        - izlazni s tri stanja (*three state*)
        - dvosmjerni (*bidirectional*)



    - banka 2 i banka 3
      - spojene su na PL preko EMIO
        - ⇒ unutar PL tretiraju se kao obični signali
        - po želji mogu biti spojeni i na izlazni priključak
- vanjski MIO priključci pripadaju dvama MIO naponskim bankama
  - MIO naponska banka 0 (*MIO Voltage Bank 0*)
    - vanjski priključci 15:0
  - MIO naponska banka 1(*MIO Voltage Bank 1*)
    - vanjski priključci 53:16
- paziti
  - na ZedBoardu je potrebno provjeriti ne koje priključke su spojene pojedine svjetleće diode i prema potrebi podesiti premosnike koji na priključke dovode napone napajanja (2.5 V ili 3.3 V)
  - u Vivadu ovi priključci moraju biti konfigurirani za odabrani naponski standard (LVCMOS25 ili LVCMOS33)

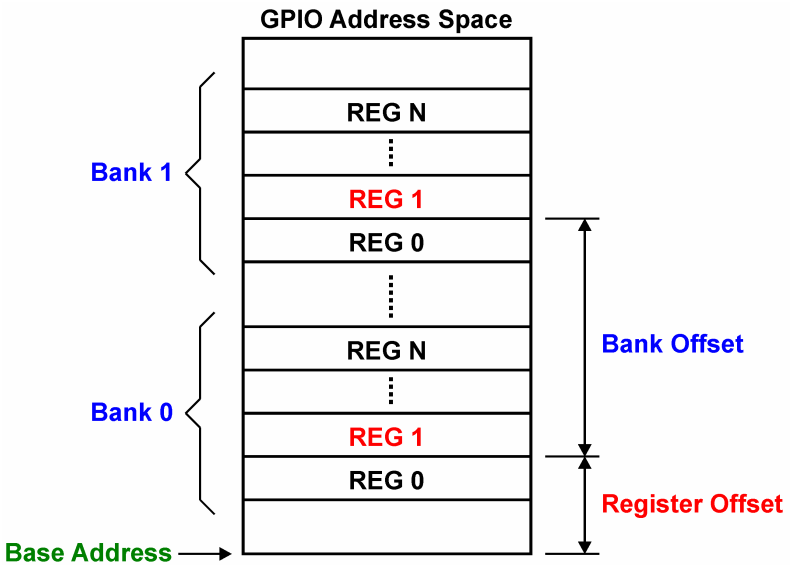
- upravljačko sklopovlje GPIO



- adrese registara

osnovna adresa + pomak registra + pomak banke

(*Base Address* + *Register Offset* + *Bank Offset*)



- osnovna adresa GPIO sklopa: 0xE000A000
  - u zaglavlju *xparameters.h* definirana je konstanta

```
#define XPAR_PS7_GPIO_0_BASEADDR 0xE000A000
```

- konfiguracijski registri GPIO sklopa (*configuration registers*)

Register Name	Type	Reg. Offset (for Bank 0)	Register Offset Constant (in *xgpiops_hw.h*)	Bank Offset
INT_MASK	ro	0x0000020C	XGPIOPS_INTMASK_OFFSET	0x40
INT_DIS	wo	0x00000214	XGPIOPS_INTDIS_OFFSET	0x40
INT_EN	wo	0x00000210	XGPIOPS_INTEN_OFFSET	0x40
INT_STAT	wtc	0x00000218	XGPIOPS_INTSTS_OFFSET	0x40
INT_TYPE	rw	0x0000021C	XGPIOPS_INTTYPE_OFFSET	0x40
INT_POLARITY	rw	0x00000220	XGPIOPS_INTPOL_OFFSET	0x40
INT_ANY	rw	0x00000224	XGPIOPS_INTANY_OFFSET	0x40
DATA_RO	ro	0x00000060	XGPIOPS_DATA_RO_OFFSET	0x4
DATA	rw	0x00000040	XGPIOPS_DATA_OFFSET	0x4
MASK_DATA_LSW	mixed	0x00000000	XGPIOPS_DATA_LSW_OFFSET	0x8
MASK_DATA_MSW	mixed	0x00000004	XGPIOPS_DATA_MSW_OFFSET	0x8
DIRM	rw	0x00000204	XGPIOPS_DIRM_OFFSET	0x40
OEN	rw	0x00000208	XGPIOPS_OUTEN_OFFSET	0x40

  - uočiti

    ⇒pomak banke nije isti za sve registre
  - napomene
    - konstante za pomake registara definirane su u zaglavlju *xgpiops_hw.h*
      - u istom zaglavlju definirane su i konstante za pomak banke, ali one ovdje nisu korištene
    - značenja kratica: ro=read only, wo=write only, rw=read/write, wtc - readable, write a one to clear

- čitanje stanja priključaka

  ⇒DATA_RO - sadrži stanja priključaka pojedine banke
    - čitanje banke 0

```
//   GPIO base address + Reg. offset + Bank offset
b0=*(       0xE000A000 +  0x00000060 +       0*0x4);
// ili korištenje predefiniranih konstanti
b0=*(XPAR_PS7_GPIO_0_BASEADDR + XGPIOPS_DATA_RO_OFFSET+0*0x4);
```

    - čitanje banke 1

```
//   GPIO base address + Reg. offset + Bank offset
b1=*(       0xE000A000 +  0x00000060 +         1*0x4);
```

  - sklopovlje za čitanje je uvijek aktivno, bez obzira je li priključak ulazni ili izlazni

- odabir je li priključak ulazni ili izlazni

  ⇒DIRM - pojedini bitovi određuju je li priključak ulazni ili izlazni

  0 ⇒ ulazni (*input*)

  1 ⇒ izlazni (*output*) ili izlazni/dvosmjerni

  - konfiguriranje priključaka banke 1
    (bitovi 0, 1 i 3 kao izlazni, a preostali bitovi kao ulazni

```
// 0b 0000 0000 0000 0000 0000 0000 0000 1011 = 0x 0000 000B
//GPIO base address + Reg. offset + Bank offset
*(       0xE000A000 +  0x00000204 +      1*0x40) = 0x0000000B;
```

  - uočiti
    - DIRM određuje je li omogućen pristup izlaznom pojačalu priključka

- upis u izlazni registar

  ⇒DATA - određuje stanja priključaka (ako su priključci izlazni)

  - odjednom se upisuju sva 32 bita
  - upis u izlazni registar banke 0 i banke 1

```
//GPIO base address + Reg. offset + Bank offset
*(       0xE000A000 +  0x00000040 +       0*0x4) = 0x00000000;
*(       0xE000A000 +  0x00000040 +       1*0x4) = 0x00123456;
```

  ⇒MASK_DATA_LSW

  - upisuje u registar DATA, ali samo u zadane priključke u donjih 16 bitova
    - bitovi 32:16 predstavljaju masku (MASK)
    - bitovi 15:0 predstavljaju podatak (DATA)

    ⇒DATA se upisuje u izlazni registar tamo gdje je MASK=0
  - upis vrijednosti 0 i 1 u bitove 1 i 3 banke banke 1

```
// 0b 1111 1111 1111 0101 0000 0000 0000 1000 = 0x FFF5 0008
//GPIO base address + Reg. offset + Bank offset
*(       0xE000A000 +  0x00000000 +       1*0x8) = 0xFFF50008;
```

  ⇒MASK_DATA_MSW

  - upisuje u registar DATA, ali samo u zadane priključke u gornjih 16 bitova

    ⇒isti princip kao kod MASK_DATA_LSW
  - uočiti
    - čitanje iz DATA daje podatak iz registra, a ne podatak iz izlaznog priključka

- upravljanje izlaznim pojačalom

  ⇒ OEN - pojedini bitovi određuju je li izlaz omogućen ili onemogućen

  0 ⇒ onemogućen (*disabled*), tj. u stanju visoke impedancije

  1 ⇒ omogućen (*enabled*)

  - omogućavanje izlaza priključaka 0, 1 i 3 banke 1

```
//GPIO base address + Reg. offset + Bank offset
*(      0xE000A000 +  0x00000208 +      1*0x40) = 0x0000000B;
```

6.3 Programska podrška za ugradbene računalne sustave

- programska podrška namijenjena oformljenoj platformi
  - primjer
    - razvoj aplikacije koja se izvršava na Windows oper. sustavu
  - programska podrška "naslanja se" na operacijskim sustavom

    ⇒ aplikacija komunicira sa sklopovljem preko upravljačkih programa
- programska podrške za ugradbeni računalni sustav
  - obuhvaća sve razine
    - pokretački program (*boot loader*)
    - osnovne inicijalizacije
      - tablice prekidnih vektora
      - izvori takta
      - konfiguracija priručne memorije
      - sučelja prema memorijama, stanja čekanja, i sl.
      - itd.
    - posluživanje posebnog sklopovlja
      - sklopovlje ja nadzor rada (*watch dog*)
      - režim smanjene potrošnje (*idle*, *power down*)
    - upravljački programi (*device drivers*) PURS
    - operacijski sustav .
    - korisnikova aplikacija .
  - rezultat povezivanja (*linking*) je cjelovita programska podrška
    - neke komponente mogu biti izostavljene
    - primjer
      - ako nema upravljačkih programa i OS ⇒ *bare machine*
- poglavlja 6 (ovo poglavlje) ⇒ *bare machine*

  poglavlje 7 ⇒ upravljački programi i jednostavan OS

6.4 Vitis

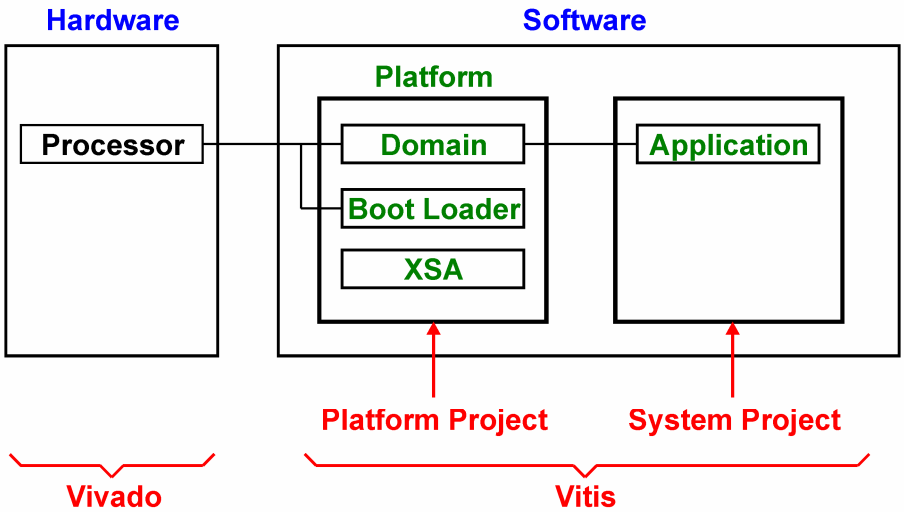
- Xilinxov alat za razvoj programske podrške

6.4.1 Radna okolina

- radna okolina (*workspace*)
  - mapa (direktorij) u kojoj Vitis čuva sve datoteke i podmape vezane uz programsku podršku projekta
  - lokacija ove mape daje se kod pokretanja Vitisa
    File/Switch Workspace
  - paziti
    - nakon pokretanja Vitisa uvijek provjeriti jesmo li u ispravnoj mapi (posebno je to važno ako je radna mapa premještana)
- napomena
  - ponekad se ista mapa koristi kao radna okolina za Vitis i Vivado
    $\Rightarrow$ tako ćemo raditi na vježbama zbog lakšeg prenošenja datoteka na računala u laboratoriju

6.4.2 Programsko okruženje

- okruženje za razvoj programske podrške
  - platforma
    - specifikacija sklopovlja
      $\Rightarrow$ iz XSA datoteke
    - programske komponente
      - domena
      - programi za inicijalizaciju (*boot loader*)
  - aplikacija

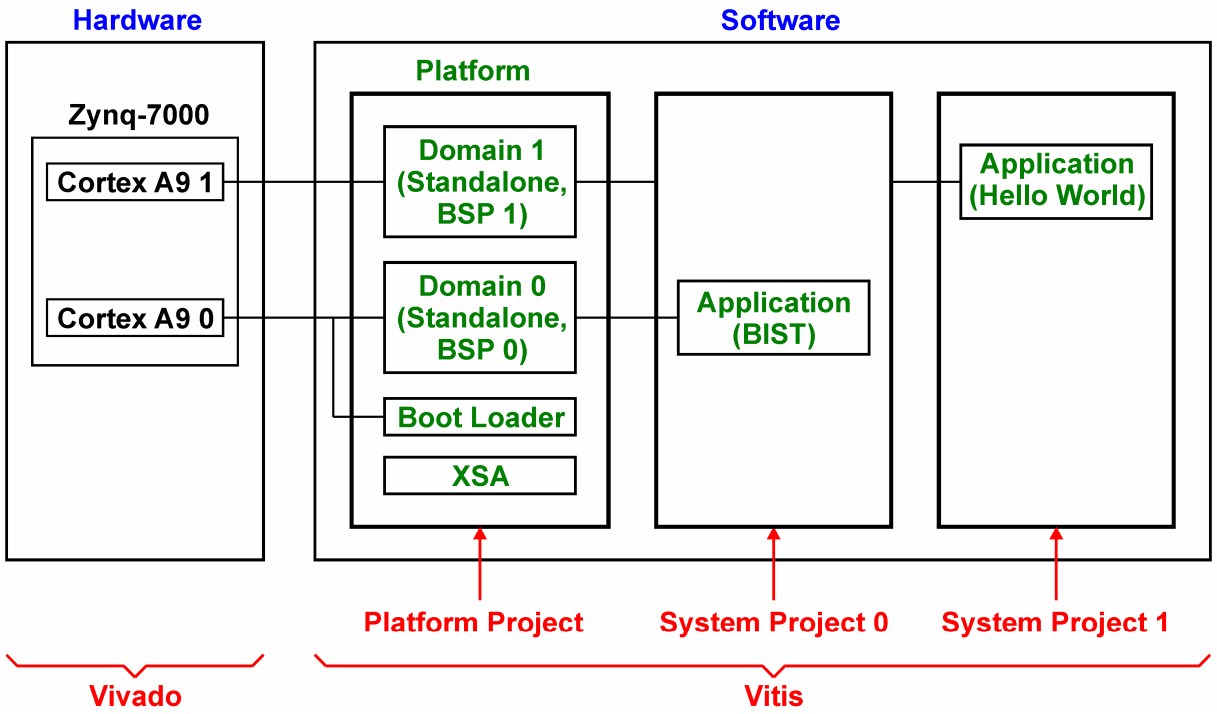


- domena
  - sadrži osnovno programsko okruženje, npr.
    - upravljački programi i operacijski sustav
      - Standalone OS
      - FreeRTOS
      - Linux
  - vezana je uz pojedini procesor ili grupu procesora
  - na jednoj platformi može postojati više domena

- aplikacija
  - programi - sadrže izvorni kod i zaglavlja
  - razvija se za danu platformu i jednu domenu
  - na jednoj platformi može postojati više aplikacija

- u Vitisu
  - platforma s pripadajućim domenama čini Platform Project
    $\Rightarrow$ tu se dodaju domene
    $\Rightarrow$ tu se konfigurira program za inicijalizaciju (*Boot Loader*)
  - aplikacije koje rade istovremeno (simultano) čine System Project

- primjer dizajna s više sistemskih projekata (Zynq-7000)



- operacijski sustavi sadrže upravljačke programe za sklopovlje (*device drivers*)
    - Standalone OS
        ⇒ *Board Support Package*, BSP
    - FreeRTOS
        ⇒ *Board Support Package*, BSP
    - Linux
        ⇒ cjelovito Linux okruženje

- *Board Support Package*, BSP
    - programsko okruženje u kojem razvijamo aplikaciju
    - sadrži
        - standardne C biblioteke
        - upravljačke programe (*device drivers*)
    - Vitis ga automatski generira
    - korisnik ga može mijenjati
    – BSP je detaljnije obrađen u poglavlju "7 Standalone OS"

6.4.3 Tijek razvoja programske podrške

- *Vitis Software Development Workflow*
- 1. faza: kreiranje platforme (Platform Project)
    - odabir imena platforme
    - učitavanje informacija o sklopovlju
        $\Rightarrow$ iz XSA datoteka
    - kreiranje domene 1
        - odabir operacijskog sustava
            - Standalone OS
            - FreeRTOS
            - Linux
        - odabir procesora
            - primjer: Zynq-7000, na raspolaganju je
                - ps7_cortex9_0 (fiksno ožičen)
                - ps7_cortex9_1 (fiksno ožičen)
                - MicroBlaze (ako je implementiran u PL-u)
    - kreiranje domene 2 (ako je potrebna)
    - itd.
- 2. faza: kreiranje sistemskog projekta (System Project)
- 3. faza: kreiranje aplikacije (Application Project)
    - aplikacija se pridružuje napravljenom System Project-u
    - napomena
        - u verziji alata 2020.2, 2. i 3. faza obavljaju se istovremeno
        - kreiranje obaju projekata pokreće se pomoću komande
            File/New/Application Project
- 4. faza: razvoj aplikacije
- 5. faza: uhodavanje aplikacije
- 6. faza razvoj vlastitog ili modifikacija postojećeg *boot loader*-a
- 7. integracija ia ispitivanje cijelog sustava

6.5 Programska podrška

6.5.1 Pokretački program

- cjelokupna programska podrška računalnog sustava pohranjena je u vanjskoj nebrisivoj memoriji
    - primjeri
        - PC ⇒ tvrdi disk
        - Zynq ⇒ Flash memorija (NAND, NOR, QSPI, SD, SDIO)
- pokretački program (*boot loader*)
    - program koji nakon pokretanja sustava (reseta) učitava programsku podršku iz vanjske nebrisive memorije u RAM
    - pohranjen je u ROM-u

6.5.1.1 Učitavanje konfiguracije u Zynq

- podsjetimo se (poglavlje "3.2.1.2 Ugrađena memorija (OCM)")
    - OCM sadrži RAM i ROM
        - ROM nije dostupan korisniku
        - u ROM-u se nalazi pokretački program
- učitava se cijela konfiguracija
    - sklopovlje
    - programska podrška
- načini rada pokretačkog programa
    - pokretanje iz Flash memorije (*PS Master Boot Mode*)
        ⇒ koristi se tijekom normalnog rada sustava (u eksploataciji)
        - pokretački program inicijalizira PS za pristup Flash memoriji
            ⇒ inicijalizacija takta, reset sklopovlja, periferija itd.
        - učitava konfiguraciju iz Flash memorije u OCM
        - pokreće učitanu programsku podršku učitanu u OCM
    - pokretanje preko JTAG sučelja (*JTAG Slave Boot Mode*)
        ⇒ koristi se tijekom razvoja i uhodavanja
        - pokretački program inicijalizira PS za pristup JTAG sučelju
        - postavlja PS u stanje čekanja
        - PC šalje konfiguraciju preko JTAG sučelja

- odabir načina rada pokretačkog programa

Boot Device	MIO[5]	MIO[4]	MIO[3]
JTAG	0	0	0
NOR	0	0	1
NAND	0	1	0
Quad SPI	1	0	0
SD Card	1	1	0

  - ZedBoard podržava crveno označene načine rada
    ⇒za odabir se koriste premosnici



    – uloga MIO[2] i MIO[6]
      ⇒vidi daljnja poglavlja

- uočiti
  - programska podrška koja se nalazi u Flash memoriji može biti
    - cjelovita programska podrška koja se izvršava za Zynqu
      ili
    - program za učitavanje (*First Stage Boot Loader*, FSBL)

6.5.2 Osnovne inicijalizacije

- Cortex A9 ⇒ reset vektor nalazi se na lokaciji 0
  ⇒nakon reseta skače se na adresu koja je zapisana na lokaciji 0
- prije poziva korisnikove aplikacije, treba napraviti osnovne inicijalizacije
  - nalaze se u datotekama
    asm_vectors.s
    boot.s
- osnovne inicijalizacije obično se pišu u asembleru

- na adresi 0 smješten je kod *asm_vectors.s*

```
.org 0

.globl _vector_table

.section .vectors
_vector_table:
   B   _boot
   B   Undefined
   B   SVCHandler
   B   PrefetchAbortHandler
   B   DataAbortHandler
   NOP /* Placeholder for address exception vector*/
   B   IRQHandler
   B   FIQHandler

IRQHandler:                         /* IRQ vector handler */
   stmdb   sp!,{r0-r3,r12,lr}       /* state save ...*/
     ⋮
   ldmia   sp!,{r0-r3,r12,lr}       /* state restore ... */
   subs    pc, lr, #4               /* adjust return */

FIQHandler:                         /* FIQ vector handler */
   stmdb   sp!,{r0-r3,r12,lr}
     ⋮

Undefined:                          /* Undefined handler */
     ⋮

SVCHandler:                         /* SWI handler */
     ⋮

DataAbortHandler:                   /* Data Abort handler */
     ⋮

PrefetchAbortHandler:               /* Prefetch Abort handler */
     ⋮
```

  - uočiti
    - nakon reseta skače se na adresu (labelu) _boot
      - labela _boot nalazi se u datoteci boot.s

- program boot.s sadrži sljedeće inicijalizacije
  - brisanje instrukcijske priručne memorije (*instr. cache invalidation*)
  - inicijalizacija pokazivača na stog za sve načine rada
    ⇒ IRQ, FIQ, supervisor, undefine, abort, system
  - konfiguriranje MMU
  - uključivanje priručne memorije, MMU i FPU
    itd.
  - poziv funkcije main()

6.5.3 Korisnikova aplikacija

- glavni program

```
#define N_Pause 10000000

int main(){
   // LD9 => MIO7 => GPIO bank 0
   // Bit 7 => 0b 0000 0000 0000 0000 0000 0000 1000 0000 => 0x00000080;
   // DIRM[7]=1 (pin 7 is output pin)
   //                  GPIO base address+Reg. offset+Bank offset
   *(volatile unsigned int *)(0xE000A000+0x00000204 +0*0x40)=0x00000080;
   // OEN[7]=1  (pin 7 is enabled)
   *(volatile unsigned int *)(0xE000A000+0x00000208 +0*0x40)=0x00000080;
   while(1){
      int k;
      // writing into pin via MASK_DATA_0_LSW
      //    <---  --MASK- ----> <---  --DATA--- -->
      // 0b 1111 1111 0111 1111 0000 0000 1000 0000  =>  0xFF7F0080;
      *(volatile unsigned int *)(0xE000A000+0x00000000+0*0x8)=0xFF7F0080;
      for (k=0;k<N_Pause;k++);
      //    <---  --MASK- ----> <---  --DATA--- -->
      // 0b 1111 1111 0111 1111 0000 0000 0000 0000 => 0x FF7F 0000;
      *(volatile unsigned int *)(0xE000A000+0x00000000+0*0x8)=0xFF7F0000;
      for (k=0;k<N_Pause;k++);
   }
   return 0;   // never reached
}
```

6.6 Prevođenje, povezivanje i punjenje

- provodi se u sklopu alata Vivado

⇒ vidi laboratorijske vježbe

6.7 Učitavanje konfiguracije u sklop Zynq

- tijekom razvoja programske podrške, konfiguracija se u Zynq učitava preko JTAG sučelja

⇒ vidi laboratorijske vježbe

6.8 Uhodavanje programske podrške

- Vitis podržava dva alata za uhodavanje (*debugger*)
  - Xilinx System Debugger
    - ugrađen je u Vitis
  - GNU Debugger (GDB)
    - vanjski *debugger* koji se poziva iz Vitis sučelja
    - popularan je kod inženjera koji razvijaju Linux aplikacije
- podržana su dva načina uhodavanja (*debugging*)
  - na stvarnom sklopovlju
  - na QEMU (*Quick Emulator platform*)
- razlikovati
  - debug
    - u Vitisu pokreće sučelje za uhodavanje
    - postoje točke prekida i sl.
  - run
    - program se samo pokrene na sklopovlju i on dalje radi
- napomena
  - debug i run se pozivaju iz dvaju različitih izbornika, ali koriste istu konfiguraciju (prividno izgleda kao isti poziv)

$\Rightarrow$ vidi laboratorijske vježbe

6.9 Pitanja za provjeru znanja

1. Objasniti pojmove: domena, platforma, sistemski projekt, aplikacijski projekt (u Vitis programskom okruženju). Opisati ulogu XSA datoteke.

6.10 Zadaci za samostalan rad

1. Napraviti pripremu za laboratorijsku vježbu 4.

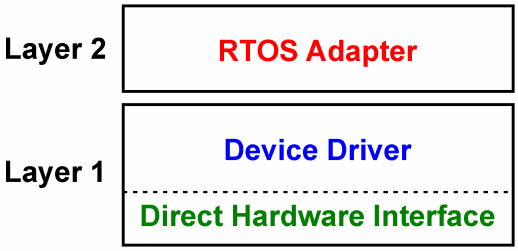
7 Standalone OS

- operacijski sustavi koje trenutno podržava platforma Zynq-7000
  - Standalone OS
    - ⇒Board Support Package, BSP
  - FreeRTOS
    - ⇒Board Support Package, BSP
  - Linux
    - cjelovito Linux okruženje
- Standalone OS sadrži upravljačke programe za sljedeće jedinice
  - procesor i upravljačko sklopovlje
    - CPU (I/O funkcije specifične za procesor)
    - DMA
  - komunikacijsko sklopovlje
    - GPIO, UART, UART lite, SPI, CAN, USB, PCI Express
  - sistemska brojila
    - Timer/Counter, Watchdog Timer/Timebase
  - sistemske komponente
    - DDR memorijsko sučelje
    - Ethernet 10/100 MAC Lite
    - Gigabit Ethernet MAC
    - Hardware ICAP
    - Interrupt Controller
    - LL FIFO
    - System Monitor

7.1 Upravljački programi

- upravljački program (*device driver*)
  - program koji izravno upravlja pojedinom komponentom računalnog sustava (*device*)
  - najčešće ga piše proizvođač sklopovlja
    - ⇒dobro poznaje sklopovlje pa može napisati učinkovit upravljački program

7.1.1 Arhitektura upravljačkih programa



- sučelje prema sklopovlju
  - nalazi se na najnižoj razini u hijerarhiji programske podrške
    - npr. pristup registrima
  - ne usporavati ostalu programsku podršku
    ⇒izvodi se kao skup konstanti i makroa
- sloj 1
  - predstavlja upravljački program (*Device Driver*) u užem smislu riječi
    ⇒njegovo sučelje koristi programer
    ⇒programer ne mora detaljno poznavati sklopovlje
    - sadrži sučelje prema sklopovlju (*Direct Hardware Interface*)
      ⇒preko njega programira odgovarajuće registre sklopovlja
  - ako bi proizvođač promijenio sklopovlje
    ⇒sučelje upravljačkog programa ne bi se promijenilo
- sloj 2
  - prilagodba sučelju RTOS-a (RTOS *Adapter*)
    - RTOS je OS i zahtijeva *driver*-e
    - specifikacija RTOS-ovih *driver*-e ne odgovara izravno *driver*-ima iz sloja 1

    ⇒na sloju 2 nalaze se programi za prilagodbu sučelja između sloja 1 i RTOS-a

7.1.2 Nazivi komponenata upravljačke programske podrške

- *device driver naming convention*
- tijekom rada programer koristi pojedine komponente upravljačkog programa
  - identifikatore (konstante, varijable, tipovi i funkcije)
  - datoteke
- da se olakša snalaženje, uvedene su konvencije u imenovanje

  ⇒sva imena koja pripadaju upravljačkom programu, a na raspolaganju su programeru počinju slovom X ili x

7.1.2.1 Naziv upravljačkog programa

- svaki upravljački program ima svoj naziv (*driver name*)
  - ime asocira na komponentu kojom upravlja
- primjeri
  - **GpioPs** *driver* za GPIO koji je dio procesorskog sustava
  - **UartPs** *driver* za UART koji je dio procesorskog sustava
  - **SpiPs** *driver* za SPI koji je dio procesorskog sustava

7.1.2.2 Vanjski identifikatori

- vanjski identifikatori (*external identifier*)
  - identifikatori koje koristi programer
    (za razliku od npr. unutrašnjih varijabli pojedinih funkcija *driver*-a koje programer ne koristi)
- oblik vanjskih identifikatora

```
X<DRIVER_NAME>_CONSTANT_NAME;    // konstante (velika slova)
X<driver_name>_VariableName;     // varijable
X<driver_name>_TypeName;         // tipovi
X<driver_name>_FunctionName(ArgumentType Argument)
                                 // funkcije
```

- primjeri

```
XGpioPs_ConfigTable // varijabla
XGpioPs_ReadPin     // funkcija
XGPIOPS_Handler     // tip
```

7.1.2.3 Nazivi datoteka

- datoteke upravljačkih programa obuhvaćaju
  - C kod
    - sadrži implementaciju upravljačkog programa
  - zaglavlja
    - sadrži sučelja upravljačkog programa

- nazivi datoteka
  - izravan pristup sklopovlju

```
x<driver_name>_hw.h   ili   x<driver_name>_l.h
x<driver_name>_hw.c   ili   x<driver_name>_l.c
```

  - sloj 1

```
x<driver_name>.h            // vanjsko sučelje
x<driver_name>.c            // glavna datoteka

x<driver_name>_i.h          // unutrašnje sučelje
x<driver_name>_func1.c      // 1. pomoćna datoteka
x<driver_name>_func2.c      // 2. pomoćna datoteka
```

  - sloj 2

```
x<driver_name>_adapter.h
x<driver_name>_adapter.c
```

- uočiti
  - **_hw** i **_l** označavaju implementaciju specifičnu za sklopovlje
    ⇒ nazivi potječu od ***hardware*** odnosno ***low***
- smještaj datoteka
  - datoteka komponenata za izravan pristup sklopovlju i datoteka sloja 1 smještene su u isti direktorij
- primjeri
  - izravan pristup sklopovlju

```
xgpiops_hw.h
xgpiops_hw.c
```

  - sloj 1

```
xgpiops.h               // vanjsko sučelje
xgpiops.c               // glavna datoteka

-                       // unutrašnje sučelje
xgpiops_intr.c          // 1. pomoćna datoteka
xgpiops_selftest.c      // 2. pomoćna datoteka
xgpiops_sinit.c         // 3. pomoćna datoteka
xgpiops_g.c             // 4. pomoćna datoteka
```

  - razno

```
xgpiops_polled_example.c  // primjer projekta
xgpiops_intr_example.c    // primjer projekta
```

7.2 Sučelje upravljačkih programa

- *Device Driver Application Programming Interface, Device Driver* API

7.2.1 Upravljački program za GPIO procesorskog sustava

Primjer

Iz biblioteke operacijskog sustava Standalone OS izdvojiti komponente programske podrške za upravljanje sklopom GPIO koji pripada procesorskom sustavu. Opisati princip inicijalizacije i korištenja upravljačkog programa.

Rješenje

- upravljački program služi za rad s GPIO PS
  ⇒upravljački program je nazvan **GpioPs**
  ⇒vanjski identifikatori počinju s **XGpioPs_**
  ⇒pripadajuće zaglavlje **xgpiops.h**
- upravljački program **GpioPs** opisan je strukturom sljedećeg tipa

```
typedef struct {
   XGpioPs_Config GpioConfig; // Device configuration
   u32 IsReady;               // Device is initialized and ready
   XGpioPs_Handler Handler;   // Status handlers for all banks
   void *CallBackRef;         // Callback ref for bank handlers
   u32 Platform;              // Platform data
   u32 MaxPinNum;             // Max pins in the GPIO device
   u8 MaxBanks;               // Max banks in a GPIO device
   u32 PmcGpio;               // Flag for access PSGPIO for Versal
} XGpioPs;
```

- Što ako u sustavu postoji više jedinki neke vanjske jedinice (npr. dva GPIO sklopa)?
  - svaka jedinka ima zasebne vanjske priključke i upravljačke registre
    ⇒svaka treba svoj vlastiti upravljački program
  - međutim, jedinke sklopa su slične i nema razloga multiplicirati upravljački program
    ⇒koristit će se dvije jedinke istog upravljačkog programa

- Kako razlikovati različite jedinke istog upravljačkog programa?

  ⇒prvi element gornje strukture je struktura sljedećeg tipa

```
typedef struct {
   u16 DeviceId;   // Unique ID of device
   u32 BaseAddr;   // Register base address
} XGpioPs_Config;
```

  - uočiti
    - `DeviceId` govori o kojoj je jedinci riječ
    - `BaseAddr` je adresu počevši od koje su smješteni upravljački registri jedinke
  - napomena
    - GPIO je jednostavan
    - složenije jedinice sadrže u `XGpioPs_Config` više elemenata koji su razlikuju pojedine jedinke

- uočiti
  - u sklopu Zynq postoji samo jedna jedinka GPIO sklopa (vidi poglavlje "6.2.3 GPIO")

    ⇒postojat će samo jedna jedinka upravljačkog programa

- **rad s upravljačkim programom**
  - kreiranja jedinke upravljačkog programa
  - konfiguriranje jedinke
  - rad sa sklopom

- primjer kreiranja jedinke upravljačkog programa (*driver instantiation*)

```
XGpioPs Gpio;
```

- sada je potrebno konfigurirati ovu jedinku upravljačkog programa

  ⇒treba "popuniti" polja u strukturi `Gpio`
  - biblioteke sadrže tablicu s predefiniranim konfiguracijama za sve periferije
    - upisuje se u `Gpio->GpioConfig`
  - pronalaženje tablice izvodi se pomoću funkcije

```
XGpioPs_Config *XGpioPs_LookupConfig(u16 DeviceId)
```

    - uočiti
      - argument `DeviceId` predstavlja jedinku
      - funkcija vraća pokazivač na strukturu tipa `XGpioPs_Config`

- primjer određivanja konfiguracije

```
XGpioPs_Config *GpioConfigPtr;

GpioConfigPtr = XGpioPs_LookupConfig (XPAR_PS7_GPIO_0_DEVICE_ID);
```

  - konstanta `XPAR_PS7_GPIO_0_DEVICE_ID` definirana je u **xparameters.h**

```
#define XPAR_PS7_GPIO_0_DEVICE_ID 0
```

- konfiguriranje upravljačkog programa (popunjavanje strukture `Gpio`)
  - na raspolaganju je funkcija

```
s32 XGpioPs_CfgInitialize(XGpioPs *InstancePtr,
                          const XGpioPs_Config *ConfigPtr,
                          u32 EffectiveAddr);
```

- primjer konfiguriranja upravljačkog programa

```
Status = XGpioPs_CfgInitialize(&Gpio,
                               GpioConfigPtr,
                               GpioConfigPtr->BaseAddr);
```

- uočiti:
  - do ovog trenutka napravljena je inicijalizacija upravljačkog programa
  - sada je potrebno koristiti GPIO (inicijalizirati, pisati, čitati i sl.)
    ⇒na raspolaganju su funkcije
- funkcije za rad sa sklopom GPIO PS
  - ima ih 30

```
        ⋮
u32 XGpioPs_Read(const XGpioPs *InstancePtr, u8 Bank);
u32 XGpioPs_ReadPin(const XGpioPs *InstancePtr, u32 Pin);
        ⋮
void XGpioPs_SetDirection(const XGpioPs *InstancePtr,
                          u8 Bank, u32 Direction);
void XGpioPs_SetDirectionPin(const XGpioPs *InstancePtr,
                             u32 Pin, u32 Direction);
        ⋮
void XGpioPs_SetOutputEnable(const XGpioPs *InstancePtr,
                             u8 Bank, u32 OpEnable);
void XGpioPs_SetOutputEnablePin(const XGpioPs *InstancePtr,
                                u32 Pin, u32 OpEnable);

void XGpioPs_Write(const XGpioPs *InstancePtr,
                   u8 Bank, u32 Data);
void XGpioPs_WritePin(const XGpioPs *InstancePtr,
                      u32 Pin, u32 Data);
```

- uočiti
  - prvi argument svih funkcija je pokazivač na jedinku upravljačkog programa
  - postoje funkcije koje rade s cijelom skupom priključaka i one koje rade s pojedinačnim priključkom
- napomena
  - funkcije koje rade s pojedinim priključkom adresiraju sve priključke u rasponu [117:0] (32+22+32+32 priključka)

- primjer upisa u vanjski priključak

```
XGpioPs_WritePin(&Gpio, 7, 1);    // Pin 7 => '1'
```

7.2.2 Standardna i specifična sučelja

- cjelokupno sučelje upravljačkog programa (*device driver* API) sadrži dva dijela
  - standardno sučelje (*standard* API)
  - funkcije koje su specifične za dotičnu jedinicu

- podsjetimo se primjera u poglavlje 7.2.1
  - korištene su funkcije
    - `XGpioPs_CfgInitialize()`, `XGpioPs_LookupConfig()`
      - ove funkcije pripadaju standardnom sučelju
        ⇒ sadrži ju svaki upravljački program u Standalone OS
  - preostale funkcije (`XGpioPs_Read()`, `XGpioPs_ReadPin()`, itd.)
    - specifične su za GPIO

- funkcije standardnog sučelja
  - postoje za svaku jedinicu

```
..._CfgInitialize(), ..._Reset(), ..._SelfTest(),
..._LookupConfig()
```

  - opcionalne (primjer za GPIO)

```
..._Start(), ..._Stop(), ..._GetStats(), ..._ClearStats(),
..._InterruptHandler()
```

7.3 Rad sa sklopom GPIO pomoću upravljačkih programa

- **GpioPs** - upravljački program za GPIO koji je dio procesorskog sustava

Primjer

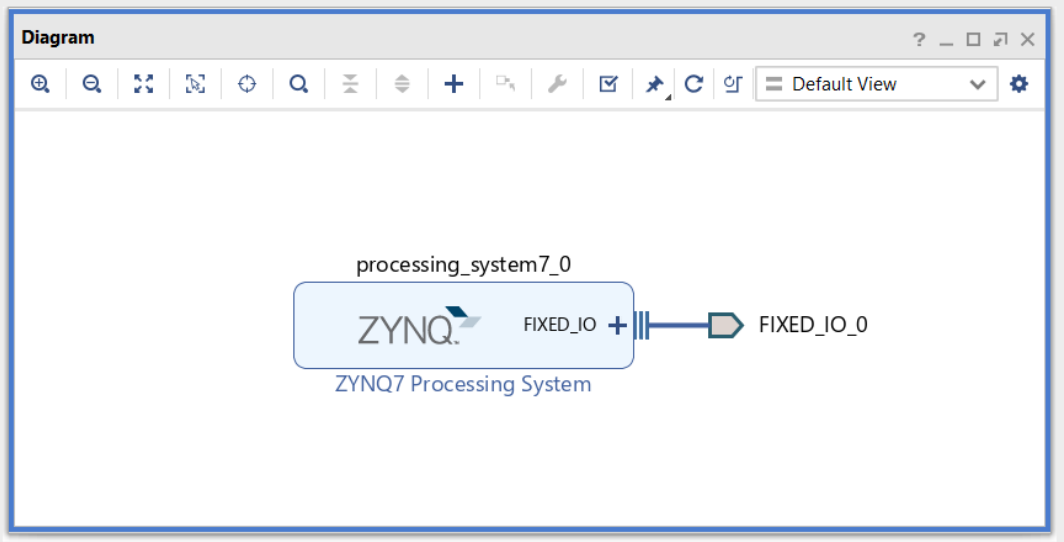
Potrebno je razviti sklopovlje i programsku podršku koji pali i gasi svjetleću diodu LD9 na razvojnom sustavu ZedBoard. Koristiti upravljački program koji je na raspolaganju u operacijskom sustavu Standalone OS.

Rješenje

- potrebno je
  - razviti sklopovlje ⇒ Vivado
  - prenijeti konfiguraciju sklopovlja u programsko okruženje
    ⇒ XSA datoteke
  - razviti programsku podršku ⇒ Vitis
  - ispitati rad na stvarnom sklopovlju ⇒ ZedBoard

7.3.1 Sklopovlje

- shema sklopovlja
  - jednako je sklopovlju iz prethodnog primjera
    ⇒ poglavlje 6.2



  - svjetleća dioda LD9
    ⇒ spojena je preko MIO sučelja na priključak 7 sklopa GPIO

7.3.2 Programska podrška

```
#include "xparameters.h"
#include "xgpiops.h"

#define LD9 7                  // LD9 is connected to MIO7
#define N_Pause 10000000       // For dummy counter

XGpioPs Gpio;                  // Driver instance for GPIO Device

int main(){
   // ============== Driver Initialization ================

   XGpioPs_Config *GpioConfigPtr;
   int Status;

   // XGpioPs_Config *XGpioPs_LookupConfig(u16 DeviceId);
   GpioConfigPtr = XGpioPs_LookupConfig (XPAR_PS7_GPIO_0_DEVICE_ID);

   Status = XGpioPs_CfgInitialize(&Gpio,
                                  GpioConfigPtr,
                                  GpioConfigPtr->BaseAddr);

   if (Status != XST_SUCCESS) {    //  XST_SUCCESS=0L; (xstatus.h)
      return XST_FAILURE;          //  XST_FAILURE=1L; (xstatus.h)
   }

   // =============== GPIO Initialization =================

   // void XGpioPs_SetDirectionPin(const XGpioPs *InstancePtr,
   //                              u32 Pin, u32 Direction);
   XGpioPs_SetDirectionPin(&Gpio, LD9, 1);        // OUT

   // void XGpioPs_SetOutputEnablePin(const XGpioPs *InstancePtr,
   //                                 u32 Pin, u32 OpEnable);
   XGpioPs_SetOutputEnablePin(&Gpio, LD9, 1);     // ENABLED

   // ===============      GPIO Write     =================

   while(1){

      int k;

      // void XGpioPs_WritePin(const XGpioPs *InstancePtr,
      //                       u32 Pin, u32 Data);
      XGpioPs_WritePin(&Gpio, LD9, 0x01);              // LED ON

      for (k=0;k<N_Pause;k++);

      XGpioPs_WritePin(&Gpio, LD9, 0x00);              // LED OFF

      for (k=0;k<N_Pause;k++);
   }

   return XST_SUCCESS;
}
```

7.4 Hello World!

Primjer

Potrebno je razviti sklopovlje i programsku podršku koja preko asinkronog serijskog sučelja ispisuje poruku "Hello World!".

Serijsko sučelje treba raditi brzinom od 9600 bitova/s, prenositi poruke duljine 8 bitova koje sadrže jedan stop bit, a ne sadrže bit za parnost.

Koristiti upravljačke programe koji su na raspolaganju u operacijskom sustavu Standalone OS. Ne koristiti funkcije iz standardnih C biblioteka.

Uhodati sklopovlje i programsku podršku na razvojnom sustavu ZedBoard.

Rješenje

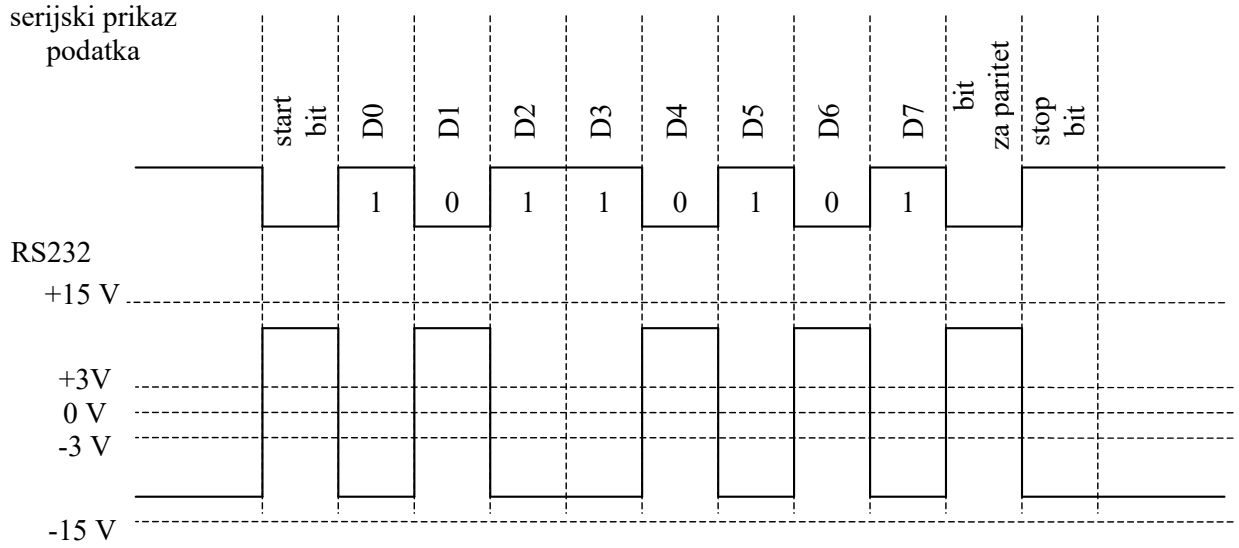
- potrebno
  - razviti sklopovlje ⇒ Vivado
    - procesorski sustav
      - omogućiti UART
      - dodijeliti ga odgovarajućim priključcima preko MIO
      - uočiti
        - na shemi se ovo samo konfigurira
        - dodjela se izvodi programski, preko MIO_PIN_XX
  - prenijeti konfiguraciju sklopovlja u programsko okruženje ⇒ XSA datoteke
  - razviti programska podršku ⇒ Vitis
    - konfigurirati MIO (upisom u odgovarajući MIO_PIN_XX
    - inicijalizirati upravljački program
    - inicijalizirati serijsko sučelje
    - napisati odsječak koji ispisuje *string* "*Hello World!*"
  - ispitati rad na sklopovlju ⇒ ZedBoard

7.4.1 Sklopovlje

7.4.1.1 Asinkrono serijsko sučelje

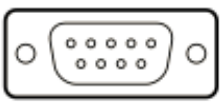
- *Universal Asynchronous Receiver-Transmitter*, UART
- praktički neizostavno sučelje računalnih sustava
- postoje dvije linije za komunikaciju
  - odašiljač - odašilje podatke
  - prijamnik - prima podatke

- asinkrona serijska komunikacija



  - uočiti
    - podaci se prenose serijski
    - ne postoji takt za sinkronizaciju
      ⇒svaki bit mora imati jednako, unaprijed dogovoreno trajanja (između odašiljača i prijamnika)
      ⇒broj bitova za start, stop i parnost mora biti dogovoren
      - standardne brzine prijenosa (bit/s, baud)
        110, 300, 1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200
    - naponske razine na kabelu su reda veličine ±10 V

- serijsko sučelje na osobnim računalima
  - izvorno
    - konektor: DE 9



    - sučelje: COM 1, COM 2, ...
  - današnja osobna računala rijetko imaju DE 9 konektor
    ⇒koristi se USB priključak
    ⇒potreban je adapter UART/USB
    - konverzija signala (naponi, struje, ...)
    - konverzija protokola

- spoj serijskog sučelja u našem slučaju
  - Zynq - ima 2 UART-a
  - Zedboard - ima jedan UART/USB adapter
- vanjsko sklopovlje serijskog sučelja na ZedBoardu
  - spoj vanjskog serijskog sklopovlja na Zynq
    - **ZedBoard Hardware User's Manual**

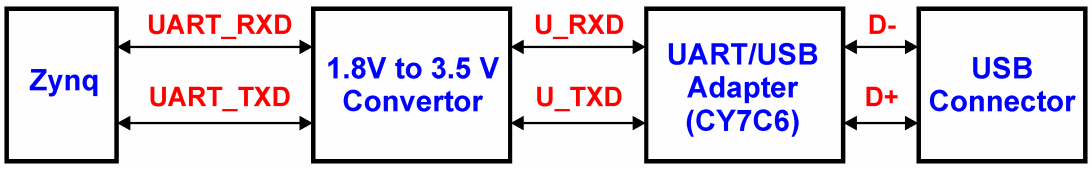
Subsystem	Signal Name	Zynq pin	Bank
PS	UART_TXD	D11 (MIO 48)	501
PS	UART_RDX	C14 (MIO 49)	501

  - podsjetimo se (poglavlje "5.7 Implementacija na sustavu ZedBoard")
    - naponi na bankama priključaka Zynqa

Subsystem	Bank	Connected voltage	Jumper
PS	500 (MIO Bank 0) 501 (MIO Bank 1) DDR	3.3 V 1.8 V 1.5 V	- - -
PL	0 13 33 34 35	3.3 V 3.3 V 3.3 V 1.8 V, 2.5 V, 3.3 V 1.8 V, 2.5 V, 3.3 V	- - - J18 J18

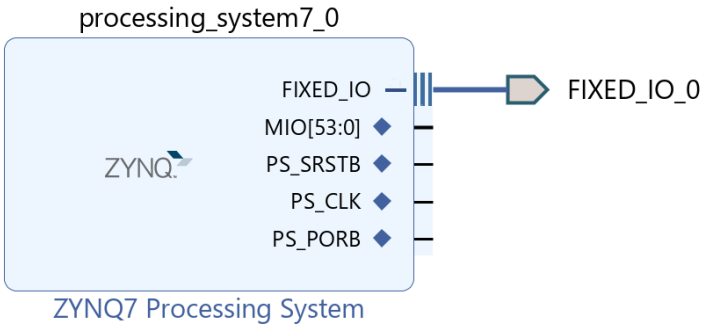
  - uočiti
    - MIO Bank 1 spojen je na napajanje 1.8 V
    - UART/USB adapteri obično rade na 3.3 V

$\Rightarrow$spoj adaptera na Zynq

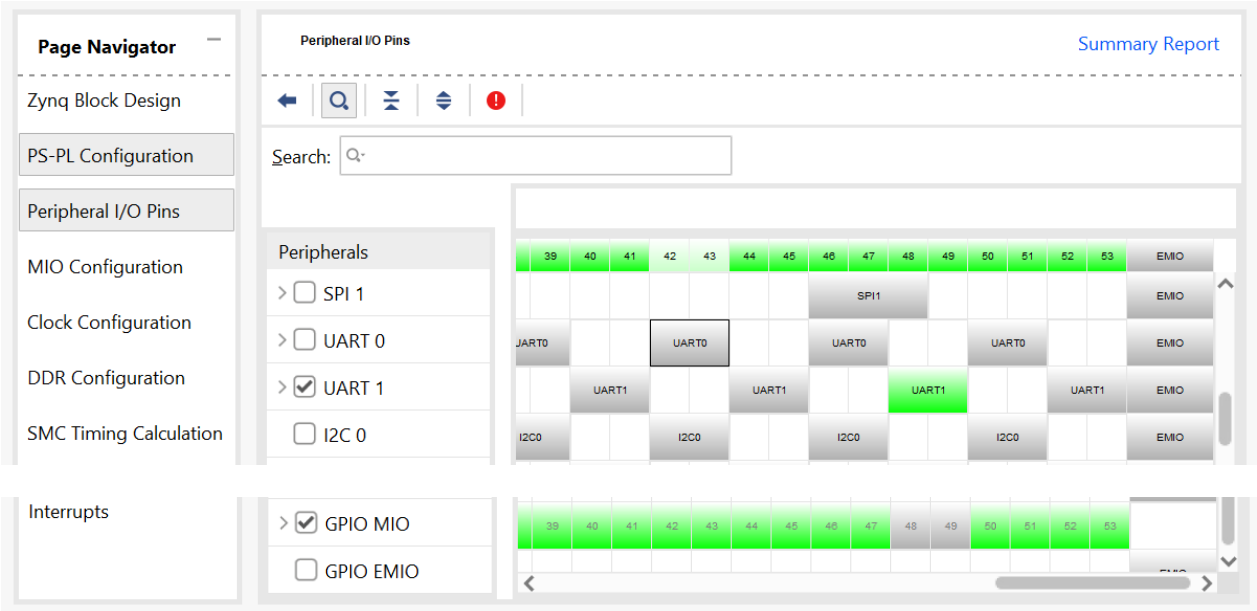


7.4.1.2 Procesorski sustav

- koristit ćemo UART1
  - UART1 može biti spojen na priključke MIO [49:48] (tablica u 3.2.3)
- shema



- konfiguracija jezgre procesorskog sustava
  - uključiti UART1 i spojiti ga na MIO[49:48]

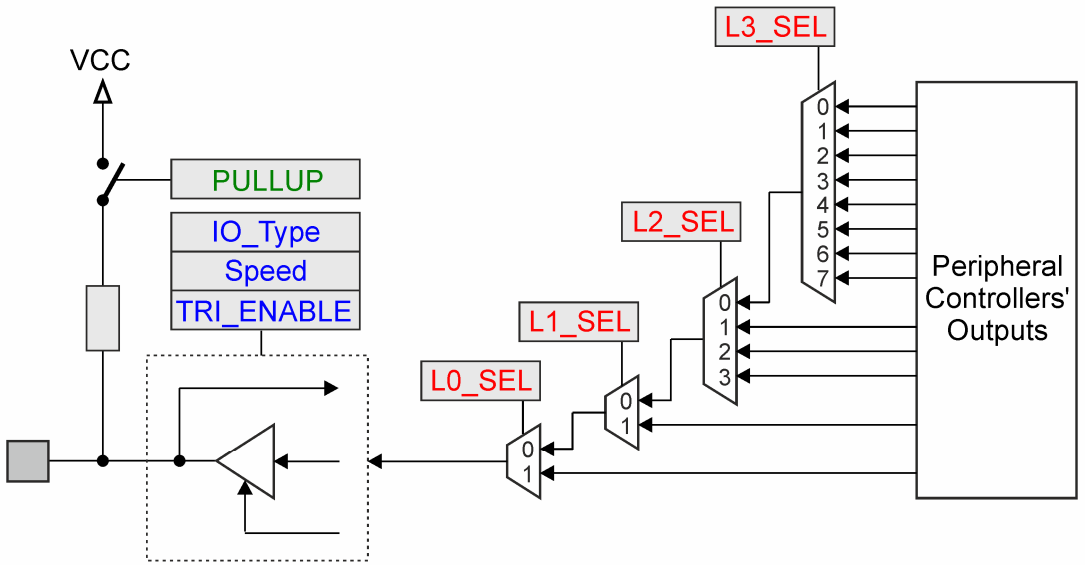


  - uočiti
    - ako je GPIO i dalje uključen, bit će automatski odspojen od MIO[49:48]
- uočiti
  - spajanje UART1 na MIO[49:48] uneseno je u parametre jezgre
    - u ovom trenutku spoj još nije ostvaren
      $\Rightarrow$ za to treba programska podrška

7.4.2 Programska podrška

7.4.2.1 Upravljačko sklopovlje

- podsjetimo se
  - sklopovlje za odabir funkcija izlaznog priključka (poglavlje 6.2.2)



  - registri za konfiguraciju MIO_PIN_53 ..... MIO_PIN_00
    $\Rightarrow$ nalaze se u SLCR
- u ovom primjeru moramo spojiti UART1 na MIO[49:48]
  - registar MIO_PIN_48

Bits	Field Name	Reset Value	Description
	...		
7:5	L3_SEL	0x0	000: GPIO 48 (bank 1), Input/Output 001: CAN 1 Tx, Output ⋮ 110: reserved 111: UART 1 TxD, Output
4:3	L2_SEL	0x0	00: Level 3 Mux 01: reserved 10: reserved 11: SDIO 0 Power Control, Output
2	L1_SEL	0x0	0: Level 2 Mux 1: USB 1 ULPI Clock, Input/Output
1	L0_SEL	0x0	0: Level 1 Mux 1: reserved
	...		

  - registar MIO_PIN_49
    - slična tablica

- **Gdje se izvodi programiranje registara u SLCR?**
  - podsjetimo se
    - kod crtanja sheme u Vivadu konfigurirana je jezgra
      - sklopu UART1 dodijeljeni su priključcima MIO[49:48]
      - ova informacija je pohranjena u XSA datoteku
    - u Vitisu je napravljen Platform Project
      - **generira se automatski** (pomoću informacija is XSA)
      - sadrži programsko okruženje
        - programi za inicijalizaciju sklopovlja
        - domene - sadrže BSP
- datoteke za inicijalizaciju sklopovlja
  - nalazi se u dijelu **hw**
  - programski odsječci i zaglavlja
    ps7_init.c, ps7_init.h
    ps7_init_gpl.c, ps7_init_gpl.h
  - opis platforme u HTML formatu
    ⇒ služi za pregledan prikazu
  - itd.
- datoteka ps7_init.c
  - sadrži kod za opis MIO priključaka (te mnoge druge opise)

```
// .. ⋮
// EMIT_MASKWRITE(      addr,        mask,         val)
EMIT_MASKWRITE   (0XF80007C0, 0x00003FFFU, 0x000016E0U)
// .. ⋮
// .. L0_SEL = 0
// .. ==> 0XF80007C4[1:1] = 0x00000000U
// ..     ==> MASK : 0x00000002U    VAL : 0x00000000U
// .. L1_SEL = 0
// .. ==> 0XF80007C4[2:2] = 0x00000000U
// ..     ==> MASK : 0x00000004U    VAL : 0x00000000U
// .. L2_SEL = 0
// .. ==> 0XF80007C4[4:3] = 0x00000000U
// ..     ==> MASK : 0x00000018U    VAL : 0x00000000U
// .. L3_SEL = 7
// .. ==> 0XF80007C4[7:5] = 0x00000007U
// ..     ==> MASK : 0x000000E0U    VAL : 0x000000E0U
// .. ⋮
```

  - uočiti

```
0x000016E0U = 0b0000 0000 0000 0000 0001 0110 1110 0000
```

- na ovakav način definirani su svi parametri Zynqa koji su preneseni sa sheme

7.4.2.2 Upravljački program za UART

- upravljački program služi za rad s UART PS
  - ⇒upravljački program je nazvan **UartPs**
  - ⇒vanjski identifikatori počinju s **XUartPs_**
  - ⇒pripadajuće zaglavlje **xuartps.h**

- upravljački program **UartPs** opisan je strukturom sljedećeg tipa

```
typedef struct {
   XUartPs_Config Config; // Configuration data structure
   u32 InputClockHz;      // Input clock frequency
   u32 IsReady;           // Device is initialized and ready
   u32 BaudRate;          // Current baud rate

   XUartPsBuffer SendBuffer;
   XUartPsBuffer ReceiveBuffer;

   XUartPs_Handler Handler;
   void *CallBackRef;     // Callback reference for event handler
   u32 Platform;
   u8 is_rxbs_error;
} XUartPs;
```

- prvi element gornje strukture
  - ⇒razlikuje jedinke upravljačkih programa (pridružuje jedinkama različito sklopovlje)

```
typedef struct {
   u16 DeviceId;           // Unique ID of device
   u32 BaseAddress;        // Base address of device (IPIF)
   u32 InputClockHz;       // Input clock frequency
   s32 ModemPinsConnected; // For modem pins
} XUartPs_Config;
```

  - uočiti
    - `DeviceId` govori o kojoj je jedinci riječ
    - `BaseAddr` je adresu počevši od koje su smješteni upravljački registri jedinke
    - `InputClockHz` sadrži frekvenciju takta iz kojeg se izvodi željenu brzinu prijenosa podataka

- napomena
  - unatoč postojanju dvaju serijskih sučelja, ovaj upravljački program dopušta kreiranje samo jedne jedinke
  - ta jedinka imat će `DeviceId` = 0, a bit će vezana (preko `BaseAddress`) za UART1

  ⇒upravljački program **UartPs** dopušta kreiranje samo jedne jedinke, a ona je vezana uz sklopovlje UART1

- standardna sučelja upravljačkog programa
  - pronalaženje tablice s predefiniranom konfiguracijom

```
XUartPs_Config *XUartPs_LookupConfig(u16 DeviceId);
```

  - konfiguriranje upravljačkog programa

```
s32 XUartPs_CfgInitialize(XUartPs *InstancePtr,
                          XUartPs_Config * Config,
                          u32 EffectiveAddr);
```

- specifična sučelja upravljačkog programa
  - ima ih 22

```
u32 XUartPs_Send(XUartPs *InstancePtr,u8 *BufferPtr,
                 u32 NumBytes);

u32 XUartPs_Recv(XUartPs *InstancePtr,u8 *BufferPtr,
                 u32 NumBytes);

s32 XUartPs_SetBaudRate(XUartPs *InstancePtr,
                        u32 BaudRate);

        ⋮

u8 XUartPs_GetRecvTimeout(XUartPs *InstancePtr);

void XUartPs_SetRecvTimeout(XUartPs *InstancePtr,
                            u8 RecvTimeout);

        ⋮

s32 XUartPs_SelfTest(XUartPs *InstancePtr);
```

- dokumentacija upravljačkih programa
  - Platform Project sadrži datoteku s opisom platforme
    platform.spr
    - platforma sadrži domene, a domena uključuje BSP
      - U BSP-u se nalaze poveznice na dokumentaciju upravljačkih programa

7.4.2.3 Aplikacija

- aplikacija (funkcija **main()**)

```
#include "xparameters.h"
#include "xuartps.h"

// Only one driver instance is allowed. It is related to UART1.
#define UART_DEVICE_ID   0

XUartPs Uart_Ps;          // Driver instance for UART Device

int main() {

   u8 SendString[] = "Hello World!\n\r";
   int SentCount;

   // ============== Driver Initialization ================

   XUartPs_Config *Config;
   int Status;

   // XUartPs_Config *XUartPs_LookupConfig(u16 DeviceId);
   Config = XUartPs_LookupConfig(UART_DEVICE_ID);

   if (NULL == Config) {
      return XST_FAILURE;
   }

   // s32   XUartPs_CfgInitialize(XUartPs *InstancePtr,
   //                             XUartPs_Config * Config,
   //                             u32 EffectiveAddr);
   Status = XUartPs_CfgInitialize(&Uart_Ps,
                                  Config,
                                  Config->BaseAddress);

   if (Status != XST_SUCCESS) {   //  XST_SUCCESS=0L; (xstatus.h)
      return XST_FAILURE;         //  XST_FAILURE=1L; (xstatus.h)
   }

   // ================ UART Initialization ================

   // s32 XUartPs_SetBaudRate(XUartPs *InstancePtr, u32 BaudRate);
   XUartPs_SetBaudRate       (&Uart_Ps            , 9600);

   // ===============    Transmit data    ================

   SentCount = 0;
   while (SentCount < (sizeof(SendString) - 1)) {
      SentCount += XUartPs_Send(&Uart_Ps, &SendString[SentCount], 1);
   }

   return XST_SUCCESS;
}
```

7.5 Hello again!

... ali ovoga puta bez kamene sjekire :)

Primjer

Potrebno je razviti sklopovlje i programsku podršku koja preko asinkronog serijskog sučelja ispisuje poruku "Hello again!".

Serijsko sučelje treba raditi brzinom od 9600 bitova/s, prenositi poruke duljine 8 bitova koje sadrže jedan stop bit, a ne sadrže bit za parnost.

Koristiti upravljačke programe koji su na raspolaganju u operacijskom sustavu Standalone OS. Za ispis poruke koristiti funkcije iz Xilinxovih i standardnih C biblioteka.

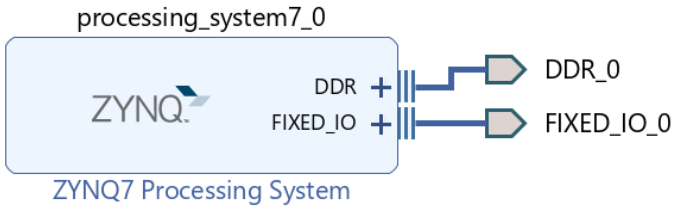
Uhodati sklopovlje i programsku podršku na razvojnom sustavu ZedBoard.

Rješenje

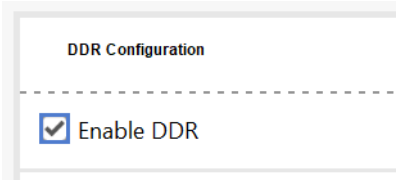
- uočiti
  - koristit ćemo funkcije iz biblioteka
    ⇒ potrošnja memorije će porasti
    ⇒ potrebno je na Zynq spojiti vanjski RAM

7.5.1 Sklopovlje

- shema sklopovlja



- konfiguracija jezgre procesorskog sustava
  - uključiti UART1 i spojiti ga na MIO[49:48]
    ⇒ kao i u prethodnom primjeru
  - omogućiti memorijski kontroler



  - podesiti parametre memorijskog kontrolera
    ⇒ bit će ilustrirano na laboratorijskim vježbama

7.5.2 Programska podrška

7.5.2.1 Ulazno-izlazne funkcije

- funkcije iz Xilinxovih standardnih C biblioteka

```
void print(const char8 *ptr);
```

  - ispisuje string na konzolu

```
void putnum(u32 num);
```

  - pretvara integer u heksadecimalni string i šalje ga na konzolu

```
void xil_printf( const char8 *ctrl1, ...);
```

  - reducirana verzija funkcije **printf**
    ⇒troši svega 1 kB memorije
  - podržava sljedeće formate ispisa
    %d, %l, %x, %c, %s

```
int printf (const char *__restrict, ...);
```

  - ponaša se kao klasična funkcija **printf**

- adresa konzole definirana je u xparameters.h

```
#define STDIN_BASEADDRESS  0xE0001000
#define STDOUT_BASEADDRESS 0xE0001000
```

  - uočiti
    - osnovne adrese sklopova UART sučelje
      0xE0000000 UART0
      0xE0001000 UART1

7.5.2.2 Aplikacija

```
#include "xparameters.h"
#include "xuartps.h"

#include <stdio.h>                                    // <---
#include "xil_printf.h"                               // <---

#define UART_DEVICE_ID   0

XUartPs Uart_Ps;          // Driver instance for UART Device

int main() {

   // ============== Driver Initialization ================

   XUartPs_Config *Config;
   int Status;

   Config = XUartPs_LookupConfig(UART_DEVICE_ID);

   if (NULL == Config) {
      return XST_FAILURE;
   }

   Status = XUartPs_CfgInitialize(&Uart_Ps,
                                  Config,
                                  Config->BaseAddress);

   if (Status != XST_SUCCESS) {
      return XST_FAILURE;
   }

   // =============== UART Initialization =================

   XUartPs_SetBaudRate (&Uart_Ps, 9600);

   // ===============    Transmit data    =================

   print      ("print:      Hello again!\n\r");       // <---
   xil_printf ("xil_printf: Hello again!\n\r");       // <---
   printf     ("printf:     Hello again!\n\r");       // <---

   return 0;
}
```

- napomena
  - UART1 je nakon što se izvrši pokretački program iz ROM-a inicijaliziran na brzinu prijenosa 115200 bitova/s
    ⇒ako se UART1 koristi s tom brzinom prijenosa, u teoriji se može preskočiti inicijalizacija
  - nije dobra praksa oslanjati se na to
    ⇒treba napraviti inicijalizaciju u vlastitoj aplikaciji, kao što je napravljeno u gornjim primjerima

7.6 Pitanja za provjeru znanja

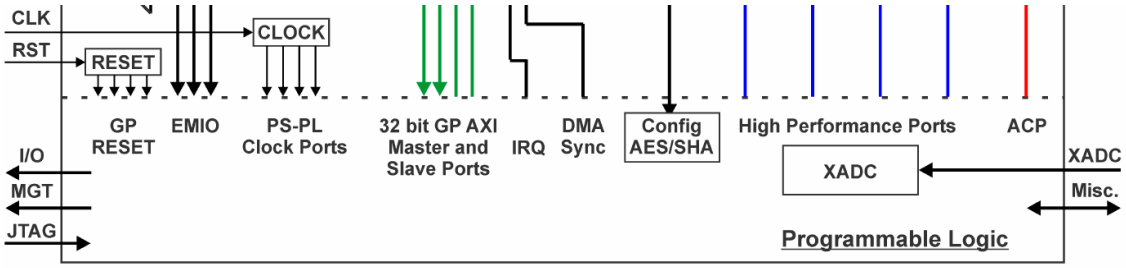
1. Što je upravljački program? Opisati sloj 1 upravljačkih programa koji pripadaju Standalone operacijskom sustavu.
2. Opisati razliku između standardnih i specifičnih sučelja upravljačkih programa. Navesti primjere funkcija koje pripadaju ovim sučeljima.
3. Nacrtati valne oblike i opisati princip asinkrone serijske komunikacije. Navesti standardne brzine prijenosa.

7.7 Zadaci za samostalan rad

1. Napraviti pripremu za laboratorijsku vježbu 5.

8 Povezivanje procesorskog sustava i programabilne logike

- podsjetimo se - poglavlje 3.1
  - osnovna blokovska shema



  - AXI - *Advanced eXtensible Interface*

- uočiti vezu između PS i PL
  - EMIO
  - AXI sučelja opće namjene (*General Purpose AXI Ports*)
  - AXI sučelja visokih performanci (*High Performance AXI Ports*)
  - DMA
  - prekidi (IRQ) i događaji (*events*)

8.1 Povezivanje PS i PL preko EMIO

Primjer

Na sklop Zynq potrebno je preko sučelja EMIO spojiti dvije izlazne linije sklopa GPIO te serijsko sučelje UART0.

Izlazne linije sklopa GPIO spojiti na priključke U22 i U21 (diode LD2 i LD3), a Rx i Tx sklopa UART na priključke V8 i V9 (pod pretpostavkom da je na njih preko Pmod konektora spojena serijska vanjska jedinica).

Napisati odgovarajuću programsku podršku te ju uhodati na razvojnom sustavu ZedBoard (svjetleće diode).

Rješenje

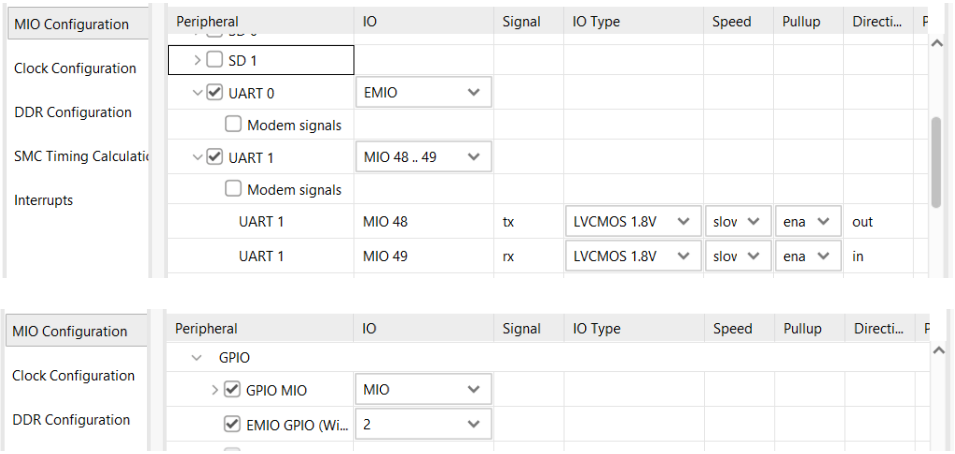
- uočiti
  - EMIO

    $\Rightarrow$ konfigurira se u jezgri
  - programabilna logika

    $\Rightarrow$ u ovom slučaju samo 4 "žice" $\Rightarrow$ prospoj na priključke

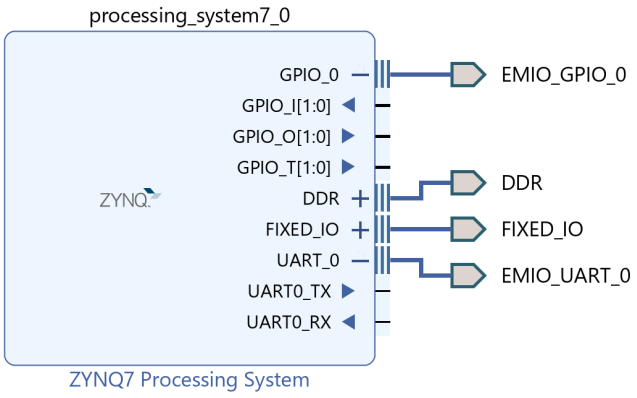
8.1.1 Sklopovlje

- konfiguracija procesorskog sustava



- uočiti
  - EMIO GPIO
    - iskorištena dva bita (od ukupno 64)
    - ove signale možemo ih koristiti za upravljanje nekom jezgrom
      - npr. spojiti GPIO_O[0] na reset jezgre led_blinker
    - u ovom primjeru izravno su spojeni na izlazne priključke PL-a
  - UART0
    - signali TX i RX ušli su u PL
      ⇒odatle spojeni izravno na izlazne priključke PL-a
    - ovakvo prospajanje kroz EMIO se koristi kad u MIO nema dovoljno priključaka za sve željene periferije
  - UART1 je također uključen (nije traženo u zadatku)

- shema sklopovlja



- uočiti
  - za svaki GPIO priključak na raspolaganju su tri signala
    GPIO_O, GPIO_I i GPIO_T

- dodjela priključaka (XDC datoteka)

```
set_property PACKAGE_PIN U22     [get_ports {EMIO_GPIO_0_tri_io[1]}]
set_property PACKAGE_PIN U21     [get_ports {EMIO_GPIO_0_tri_io[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {EMIO_GPIO_0_tri_io[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {EMIO_GPIO_0_tri_io[0]}]

set_property PACKAGE_PIN V8      [get_ports EMIO_UART_0_rxd]
set_property PACKAGE_PIN V9      [get_ports EMIO_UART_0_txd]
set_property IOSTANDARD LVCMOS33 [get_ports EMIO_UART_0_rxd]
set_property IOSTANDARD LVCMOS33 [get_ports EMIO_UART_0_txd]
```

8.1.2 Programska podrška

- podsjetimo se
  - u poglavlju 7.3.2 pokazana je programska podrška koja pali i gasi diodu LD9 koja je spojena na MIO7
  - ovdje će u tu programska podršku biti dodano paljenje i gašenje dioda LD2 i LD3 (u protufazi)
  - novi dio koda prikazan je crvenom bojom
- aplikacija

```
#include "xparameters.h"
#include "xgpiops.h"

#define N_Pause 10000000

XGpioPs Gpio;                  // Driver instance for GPIO Device

int main(){

   // ============== Driver Initialization ================

   XGpioPs_Config *GpioConfigPtr;
   int Status;

   // XGpioPs_Config *XGpioPs_LookupConfig(u16 DeviceId);
   GpioConfigPtr = XGpioPs_LookupConfig (XPAR_PS7_GPIO_0_DEVICE_ID);

   Status = XGpioPs_CfgInitialize(&Gpio,
                                  GpioConfigPtr,
                                  GpioConfigPtr->BaseAddr);

   if (Status != XST_SUCCESS) {    //  XST_SUCCESS=0L; (xstatus.h)
      return XST_FAILURE;          //  XST_FAILURE=1L; (xstatus.h)
   }
```

```
    // =============== GPIO Initialization =================

    // void XGpioPs_SetDirectionPin(const XGpioPs *InstancePtr,
    //                              u32 Pin, u32 Direction);
    XGpioPs_SetDirectionPin(&Gpio,  7, 1);  // LD9 (MIO7)  => OUT
    XGpioPs_SetDirectionPin(&Gpio, 54, 1);  // LD2 (MIO54) => OUT
    XGpioPs_SetDirectionPin(&Gpio, 55, 1);  // LD3 (MIO55) => OUT

    // void XGpioPs_SetOutputEnablePin(const XGpioPs *InstancePtr,
    //                                 u32 Pin, u32 OpEnable);
    XGpioPs_SetOutputEnablePin(&Gpio,  7, 1);  // LD9 (MIO7)  => ENABLE
    XGpioPs_SetOutputEnablePin(&Gpio, 54, 1);  // LD2 (MIO54) => ENABLE
    XGpioPs_SetOutputEnablePin(&Gpio, 55, 1);  // LD3 (MIO55) => ENABLE

    // ===============      GPIO Write     =================

    while(1){

       int k;

       // void XGpioPs_WritePin(const XGpioPs *InstancePtr,
       //                       u32 Pin, u32 Data);
       XGpioPs_WritePin(&Gpio,  7, 0x01);  // LD9 (MIO7)  = 1
       XGpioPs_WritePin(&Gpio, 54, 0x01);  // LD2 (MIO54) = 1
       XGpioPs_WritePin(&Gpio, 55, 0x00);  // LD2 (MIO55) = 0

       for (k=0;k<N_Pause;k++);

       XGpioPs_WritePin(&Gpio,  7, 0x00);  // LD9 (MIO7)  = 0
       XGpioPs_WritePin(&Gpio, 54, 0x00);  // LD2 (MIO54) = 0
       XGpioPs_WritePin(&Gpio, 55, 0x01);  // LD2 (MIO55) = 1

       for (k=0;k<N_Pause;k++);
    }

    return XST_SUCCESS;
}
```

- uočiti
  - za MIO i EMIO koristi se isti upravljački program, **GpioPs**
  - u gornjem programu registrima se pristupa preko funkcija koje pišu ili čitaju jedan bit
    - ⇒ bitovi su označeni [117:0]
      - ⇒ banke 3, 2, 1 i 0 imaju po 32, 32, 22 i 32 priključka
      - ⇒ prva dva bita u banci 2 (EMIO) su 54 i 55

- registrima pristupiti i pomoću funkcija koje upisuju cijele riječi

```
// =============== GPIO Initialization ================

// void XGpioPs_SetDirection(const XGpioPs *InstancePtr,
//                           u8 Bank, u32 Direction);
XGpioPs_SetDirection(&Gpio, 0, 0x00000080);   // LD9 (MIO7)         => OUT
XGpioPs_SetDirection(&Gpio, 2, 0x00000003);   // LD 2:3 (MIO 54:55) => OUT

// void XGpioPs_SetOutputEnable(const XGpioPs *InstancePtr,
//                              u8 Bank, u32 OpEnable);
XGpioPs_SetOutputEnable(&Gpio, 0, 0x00000080); // LD9 (MIO7)         => EN
XGpioPs_SetOutputEnable(&Gpio, 2, 0x00000003); // LD 2:3 (MIO 54:55) => EN

// etc.
```

- na sličan način radi se i s ostalim periferijama koje se spajaju preko EMIO

8.2 Povezivanje PS i PL preko AXI sučelja

- podsjetimo se - poglavlje 3.1
  - osnovna blokovska shema
    - komunikacija između pojedinih podsustava



    - uočiti
      - APB, AHB, AXI
        $\Rightarrow$ koriste se AMBA sabirnice

- AMBA - *Advanced Microcontroller Bus Architecture*
  - neslužbeni ali široko prihvaćeni otvoreni standard sabirnica za SoC koji koriste ARM procesore
  - razvijene u kompaniji ARM, 1996. -
  - postoji više generacija ovih sabirnica

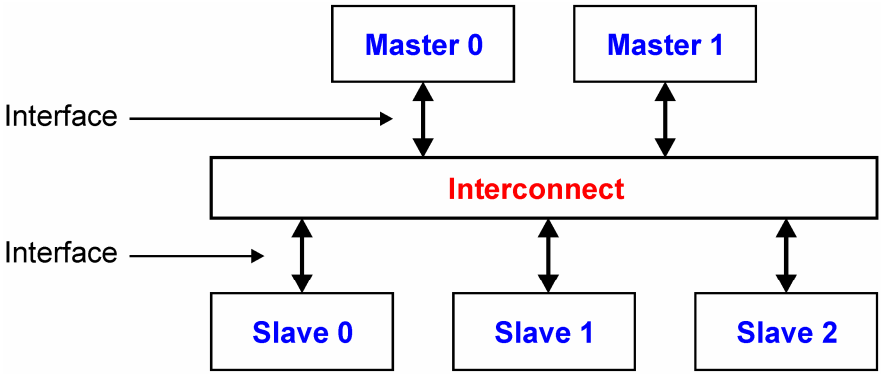
sabirnica	(prva pojava u standardu)
• ASB (*Advanced System Bus*)	(najstarije)
APB (*Advanced Peripheral Bus*)	(najstarije)
• AHB - *AMBA High-performance Bus*,	(AMBA 2)
• AXI - *Advanced eXtensible Interface 3*, (kasnije je AXI preimenovan u AXI3)	(AMBA 3)
ATB - *Advanced Trace Bus*	
• AXI4 - *Advanced eXtensible Interface 4*	(AMBA 4)
ACE - AXI Coherency Extension	
• CHI - Coherent Hub Interface etc.	(AMBA 5)

8.2.1 AXI sabirnica

8.2.1.1 Osnovni pojmovi

- tipovi AXI sučelja
  - memorijski mapirana
    - AXI-Lite
      - prenosi (čita ili piše) jedan podatak
      - služi za upravljanje periferijama, signalizaciju i sl.
    - AXI
      - može prenositi blokove do 256 podataka (*data burst*)
      - služi za prijenos većih količina podataka u i iz memorije
  - nemapirana
    - AXI-Stream
      - prenosi niz podataka, ali nije memorijski mapirano
      - koristi se kod obrade zvuka, slike i sl.
- vrste jedinica koje sudjeluju u prijenosu podataka
  - nadređena i podređena jedinca
    - nadređena jedinica (upravljač, *master*)
      - inicira AXI prijenos
    - podređena jedinica (izvršilac, *slave*)
      - odgovara na zahtjev za prijenos nadređene jedinice
  - izvor i odredište
    - izvor podataka (*source*, pošiljatelj)
      - jedinica koja odašilje informacije (podaci, adresa, upravljanje)
    - odredište podataka (*destination*, primatelj)
      - jedinica koja prima informacije
- u sustavu redoviti postoji više nadređenih i podređenih jedinica

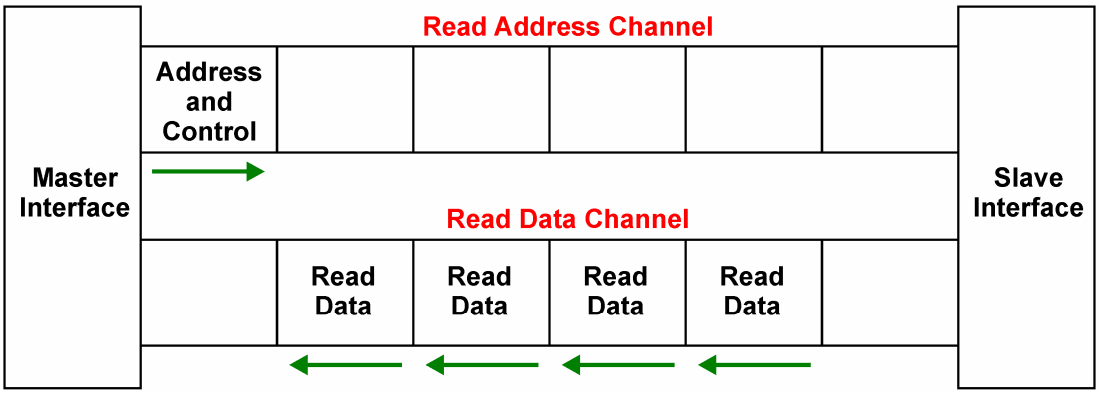
  $\Rightarrow$ spoj se ostvaruje preko AXI prospoja (*AXI Interconnect*)



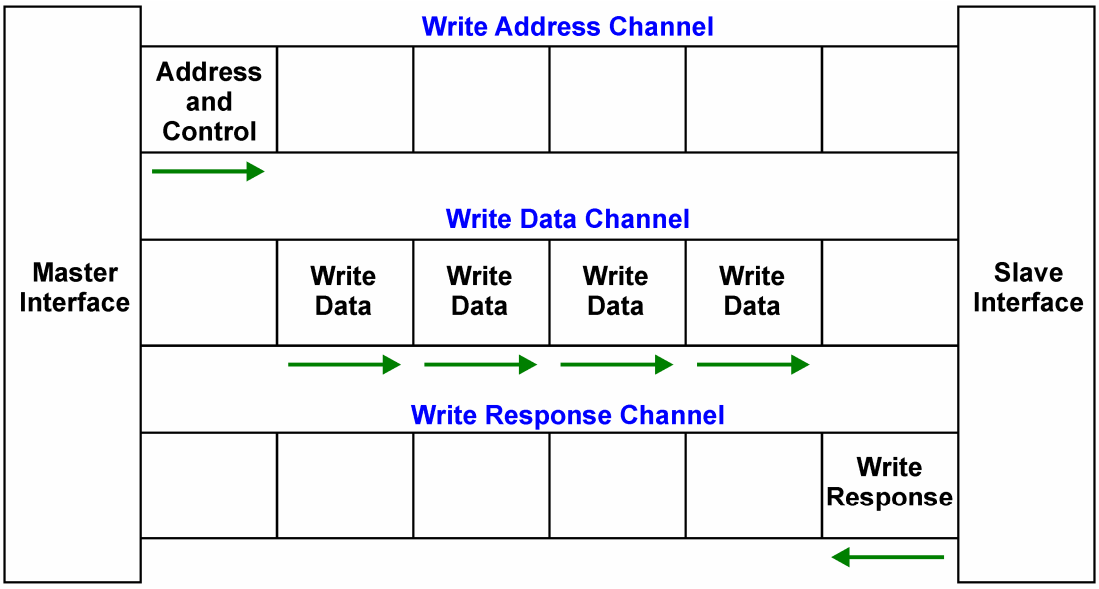
8.2.1.2 Prijenos podataka preko AXI sabirnice

- u prijenosu (*transaction*) koristi se 5 AXI kanala (*trans. channels*)
  - adresni kanal za čitanje (*Read Address Channel*)
  - podatkovni kanal za čitanje (*Read Data Channel*)
  - adresni kanal za pisanje (*Write Address Channel*)
  - podatkovni kanal za pisanje (*Write Data Channel*)
  - kanal za potvrdu upisa (*Write Response Channel*)

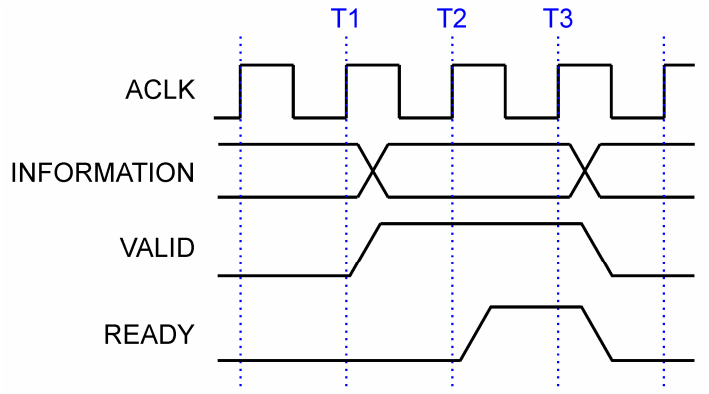
- prijenos podataka
  - kod čitanja



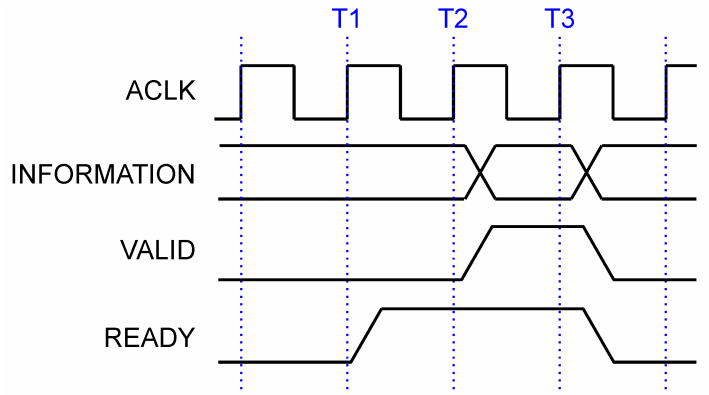
  - kod pisanja



- svih pet kanala koristi upravljani prijenos podataka (***handshaking***)
  - temelji se na principu READY/VALID
    - izvor generira VALID
      ⇒označava da su informacije (A, D, ctrl.) raspoložive
    - odredište generira READY
      ⇒označava da je spremno prihvatiti informacije
    - kad je VALID=1 i READY=1, dolazi do prijenosa
- tri slučaja rukovanja
  - VALID prije READY (*VALID before READY handshake*)

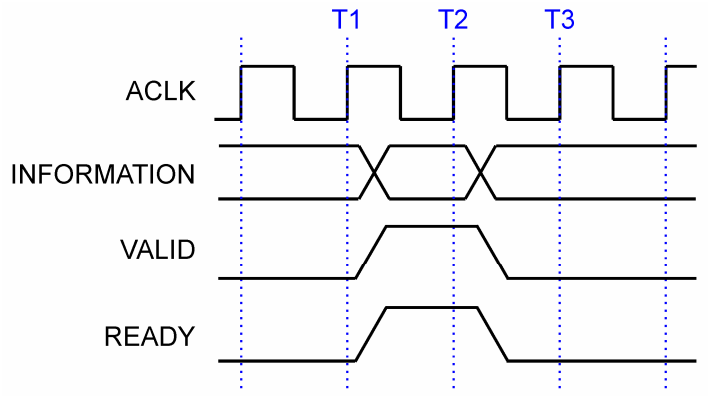


    - izvor postavlja informacije i VALID u T1
    - odredište postavlja READY u T2
    - prijenos se obavlja u T3
    - prijenos (s aspekta izvora) traje 3 perioda takta
  - READY prije VALID (*READY before VALID handshake*)



    - odredište postavlja READY u T1 (ili ranije ako je spremno)
    - izvor postavlja informacije i VALID u T2
    - prijenos se obavlja u T3
    - prijenos (s aspekta izvora) traje 2 perioda takta

- istovremeni VALID i READY (*VALID with READY handshake*)

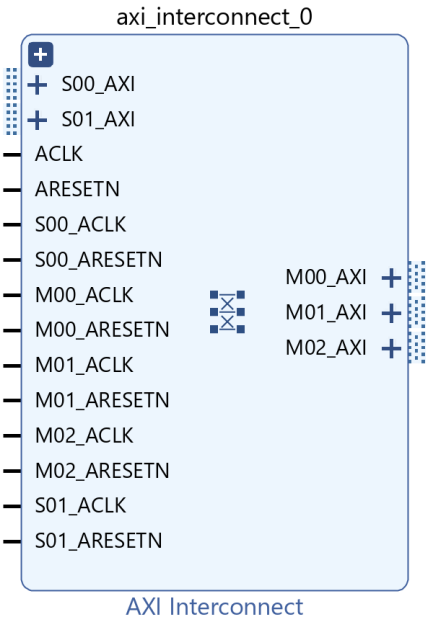


  - izvor i odredište postavljaju VALID i READY u T1
  - prijenos se obavlja u T2
  - prijenos traje 1 period takta

- uočiti
  - signali se generiraju na rastući brid takta (ACLK)
  - kad se razvija sklop s AXI sučeljem, mora zadovoljiti gornje, kao i mnoge druge specifikacije dane standardom

8.2.2 AXI sučelja u sklopu Zynq

- AXI sučelja PS prema PL
  - 9 AXI sučelja
    - sučelja opće namjene (*General Purpose AXI*)
      M_AXI_GP[1:0] (M ⇒ *master*)
      S_AXI_GP[1:0] (S ⇒ *slave*)
    - sučelja visokih performanci (*High Performance AXI*)
      S_AXI_HP[3:0]
    - *Accelerator Coherency Port*, ACP
      S_AXI_ACP
  - ukupno preko 1000 signala
  - sučelja se uključuju kroz konfiguraciju jezgre procesorskog sustava
- na ova sučelja spaja se jezgra *AXI Interconnect*
  - jezgra se može konfigurirati za proizvoljan broj jedinica
  - primjer jezgre konfigurirane za spajanje 2 nadređene i 3 podređene jedinice



  - uočiti
    - 2 podređena sučelja
    - 3 nadređena sučelja
    - zaseban ACLK i ARESETN za svako sučelje
  - svako nadređeno i podređeno sučelje imaju nešto više od 30 signala
- napomena
  - na tržištu (i u Xilinx repozitorijima) na raspolaganju je mnogo gotovih jezgri s AXI sučeljem
  - kod generiranja vlastite jezgre s AXI sučeljem, Vivado može automatski odraditi velik dio postupka

8.3 Primjer spajanja sklopova na AXI sabirnicu

Primjer

Na sklop Zynq potrebno je spojiti 7 svjetlećih dioda. Sklop koji pogoni ove diode mora imati AXI sučelje. Jednu diodu potrebno je pogoniti sklopom *led_blinker*. Napisati odgovarajuću programsku podršku koja redom pali diode spojene preko AXI sučelja.

Uhodati sklopovlje i programsku podršku na razvojnom sustava ZedBoard uz pretpostavku da su preko AXI sabirnice pojene diode LD1, LD2, ..., LD7, a na *led_blinker* dioda LD0.

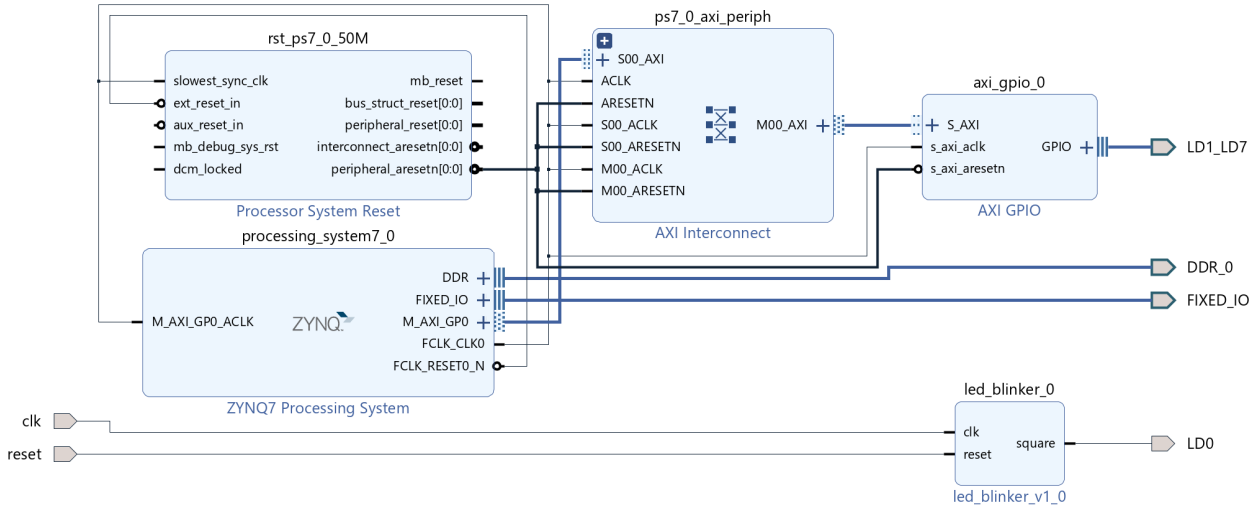
Rješenje

- uočiti
  - diode su spojene na sklopovlje u programabilnoj logici
  - PS komunicira s PL preko AXI sabirnica

  $\Rightarrow$ potrebno je koristiti
    - AXI prospoj
    - AXI ulazno izlazni sklop
    - drugo sklopovlje koje je potrebno za ispravan rad AXI jezgri

8.3.1 Sklopovlje

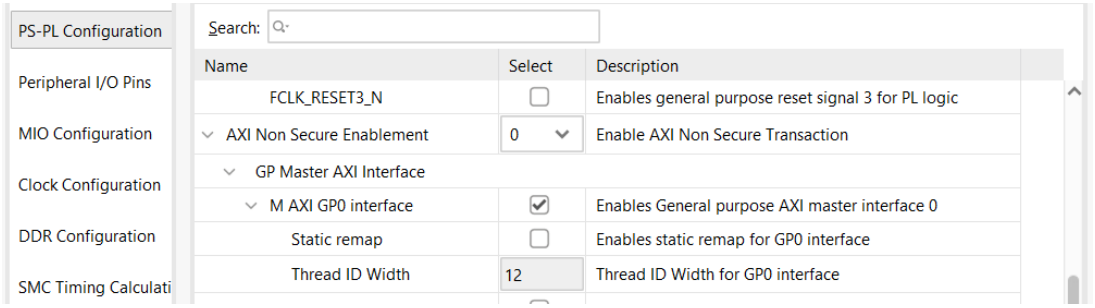
- shema sklopovlja



- **led_blinker**
  - u složenom sustavu dobro je imati komponentu koja "sigurno" radi

    $\Rightarrow$ LD0 je indikator da je programsko okruženje ispravno, kabeli nisu u prekidu, učitavanje konfiguracije je uspješno

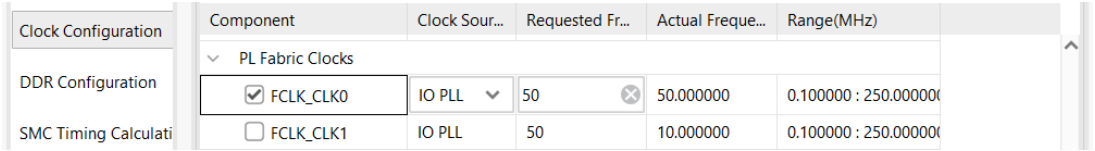
- **ZYNQ7 Processing System**
  - uključen DDR i FIXED_IO
    - nije se tražilo u zadatku
      ⇒korisno je imati dodatne resurse kod uhodavanja
  - uključen nadređeni AXI sklop (M_AXI_GP0)



  - uključen sklop za generiranje reseta za PL



  - uključen izvor takta za PL



- **AXI Interconnect**
  - konfiguriran da ima jedan nadređeni i jedan podređeni ulaz
  - za ispravan rad zahtijeva reset i takt
  - reset generira jezgra Processor System Reset
- **Processor System Reset**
  - generira odgovarajuće reset signale za razne komponente koje mogu biti spojene u sustav
  - dopušta konfiguriranje trajanja pojedinih reset signala
- **AXI GPIO**
  - podređena AXI jedinica koja se ponaša slično kao i GPIO
    ⇒može pisati u svoje priključke ili čitati iz njih
  - ovdje su priključci konfigurirani kao izlazni

- napomena
  - AXI jedinice su mapirane u memorijski prostor
    ⇒u Vivadu se pojedinim jedinicama dodjeljuju osnovne adrese
  - ove adrese prenose se u Vitis preko XSA
    ⇒koriste ih programska podrška

- razvoj temeljen na AXI sklopovima je u Vivadu velikim dijelom automatiziran
  ⇒bit će ilustrirano na laboratorijskoj vježbi

8.3.2 Upravljački program

- upravljački program za rad s GPIO
  ⇒upravljački program je nazvan **Gpio**
  ⇒vanjski identifikatori počinju s **XGpio_**
  ⇒pripadajuće zaglavlje **xgpio.h**

- podaci o jedinci upravljačkog programa (*driver instance data*)

```
typedef struct {
   UINTPTR BaseAddress;     // Device base address
   u32 IsReady;             // Device is initialized and ready
   int InterruptPresent;    // Are interrupts supported in h/w
   int IsDual;              // Are 2 channels supported in h/w
} XGpio;
```

- tip za opis konfiguracije sklopa (*configuration information for the device*)

```
typedef struct {
   u16 DeviceId;            // Unique ID  of device
   UINTPTR BaseAddress;     // Device base address
   int InterruptPresent;    // Are interrupts supported in h/w
   int IsDual;              // Are 2 channels supported in h/w
} XGpio_Config;
```

- standardna sučelja upravljačkog programa
  - pronalaženje tablice s predefiniranom konfiguracijom

```
XGpio_Config *XGpio_LookupConfig(u16 DeviceId);
```

  - konfiguriranje upravljačkog programa

```
int XGpio_CfgInitialize(XGpio * InstancePtr,
                        XGpio_Config * Config,
                        UINTPTR EffectiveAddr);
```

- specifična sučelja upravljačkog programa
  - ima ih 14

```
void XGpio_SetDataDirection(XGpio *InstancePtr,
                            unsigned Channel,
                            u32 DirectionMask);

u32 XGpio_GetDataDirection(XGpio *InstancePtr,
                           unsigned Channel);

u32 XGpio_DiscreteRead(XGpio * InstancePtr,
                       unsigned Channel);

void XGpio_DiscreteWrite(XGpio * InstancePtr,
                         unsigned Channel,
                         u32 Data);

        ⋮
```

- napomene
  - **`Channel`** govori o kojem se kanalu jezgre radi
    (Skup priključaka 2 je skriven na komponenti *axi_gpio_x* i može se učiniti vidljivim tijekom konfiguracije jezgre.)

    ima vrijednost 1 ili 2
  - **`DirectionMask`** govori je li priključak ulazni ili izlazni

    0 - output
    1 - input
  - **`Data`** je upisana vrijednost pozicionirana prema dolje
    - npr. ako axi_gpio_0 ima 5 priključaka
      ⇒za ispis svih jedinica je Data = 0x0000001F

8.3.3 Programska podrška

```
#include "xparameters.h"
#include "xgpio.h"

XGpio   Gpio_Pl;            // Driver instance for GPIO PL Device

int main() {
   int Status;

   // ============ Gpio PL Driver Initialization ==============

   XGpio_Config *GpioPlConfigPtr;

   // XGpio_Config * XGpio_LookupConfig (u16 DeviceId);
   GpioPlConfigPtr = XGpio_LookupConfig (XPAR_GPIO_0_DEVICE_ID);
                           // XPAR_GPIO_0_DEVICE_ID=0 (xparameters.h)

   // int XGpio_CfgInitialize(XGpio * InstancePtr,
   //                         XGpio_Config * Config,
   //                         UINTPTR EffectiveAddr)
   Status = XGpio_CfgInitialize(&Gpio_Pl,
                              GpioPlConfigPtr,
                              GpioPlConfigPtr->BaseAddress);

   if (Status != XST_SUCCESS) {   //  XST_SUCCESS=0L; (xstatus.h)
      return XST_FAILURE;         //  XST_FAILURE=1L; (xstatus.h)
   }

   // ================ GPIO PL Initialization =================

   // Not needed because in Vivado AXI GPIO is configured as output.
   // void XGpio_SetDataDirection(XGpio *InstancePtr,
   //                             unsigned Channel, u32 DirectionMask)
   // XGpio_SetDataDirection(&Gpio_Pl, 1, 0x00000000);     // Outputs

   // ================    GPIO PL Write     ==================

   unsigned int j=0x1;

   while(1){

      int k;

      //void XGpio_DiscreteWrite(XGpio * InstancePtr,
      //                         unsigned Channel,  // 1 or 2
      //                         u32 Data)
      XGpio_DiscreteWrite(&Gpio_Pl, 1, j);

      for (k=0;k<20000000;k++);  // Dummy loop.

      j=j<<1;
      if (j>=128){
         j=(unsigned int)0x1;
      }
   }

   return XST_SUCCESS;
}
```

8.4 Pitanja za provjeru znanja

1. Objasniti pojmove: nadređena jedinca, podređena jedinica, izvor podataka, odredište podataka. Nabrojiti tipove AXI sučelja. Navesti kanale koji se koriste kod čitanja i pisanja preko AXI sabirnice.
2. Nacrtati i opisati valne oblike kod upravljani prijenos podataka (*handshaking*) za slučaj VALID prije READY.
3. Nacrtati i opisati valne oblike kod upravljani prijenos podataka (*handshaking*) za slučaj READY prije VALID.
4. Nacrtati i opisati valne oblike kod upravljani prijenos podataka (*handshaking*) za slučaj istovremeni VALID i READY.

8.5 Zadaci za samostalan rad

1. Napraviti pripremu za laboratorijsku vježbu 6.

9 Takt

9.1 Osnovni pojmovi

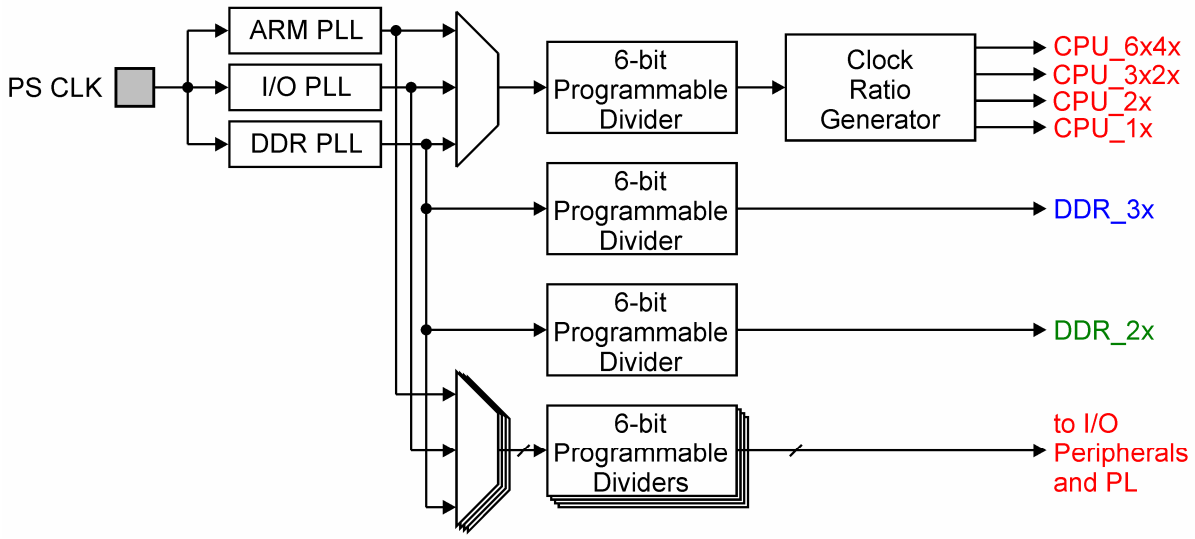
- domena takta (*clock domain*)
  - obuhvaća sve sklopove koji rade na jednom ili više sinkroniziranih signala takta
  - primjer
    - domena
      - takt od 100 MHz i takt od 50 MHz koji je dobiven dijeljenjem signala od 100 MHz
    - nije domena
      - dva signala od 100 MHz koje generiraju dva različita oscilatora

        $\Rightarrow$ između njih postoji mala razlika u frekvenciji (fazi)
- *domain crossing*
  - prijelaz signala s jedne domene na drugu

    $\Rightarrow$ potrebna je sinkronizacija

9.2 Takt u sklopu Zynq

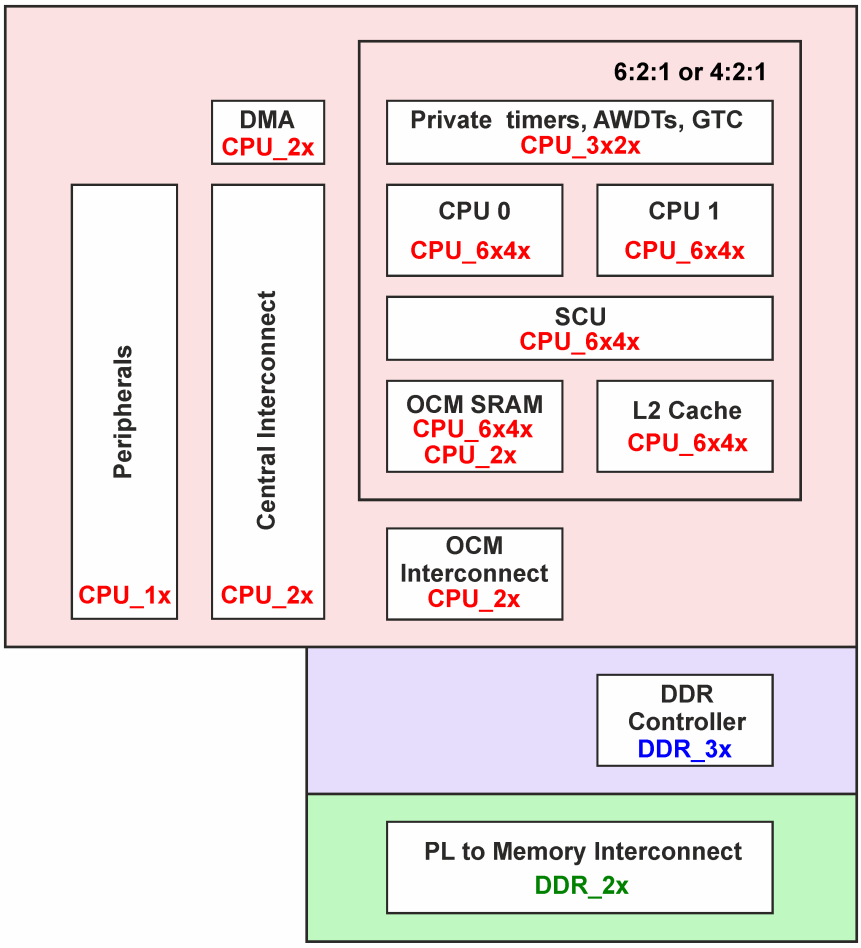
- Zynq ima tri PLL sintezatora takta



  $\Rightarrow$ 3 domene takta
- potrebno je konfigurirati sklopovlje

  $\Rightarrow$ parametri se zadaju kroz konfiguraciju jezgre

- pojednostavljen prikaz domena takta u Zynqu



  - CPU domena (CPU_6x4x, CPU_3x2x, CPU_2x, CPU_1x)
    - obuhvaća veći dio procesorskog sustava
    - koristi 4 signala takta čije se frekvencije odnose kao
      - 4:2:2:1 (koristi se kraća oznaka 4:2:1)
      - 6:3:2:1 (koristi se kraća oznaka 6:2:1)
    - uočiti

      $f_{CPU_3x2x} = f_{CPU_6x4x} / 2$
  - DDR_3x
    - obuhvaća DDR memorijski kontroler
  - DDR_2x
    - obuhvaća AXI sučelja visokih performanci AXI_HP[4:0]
- prilikom konfiguriranja jezgre odabire se
  - omjer frekvencija u CPU domeni 4:2:1 ili 6:2:1
  - frekvencija f_{CPU_6x4x}
- maksimalna frekvencija takta procesora na ZedBoardu

  $f_{CPU_6x4x} = 667$ MHz

9.3 Pitanja za provjeru znanja

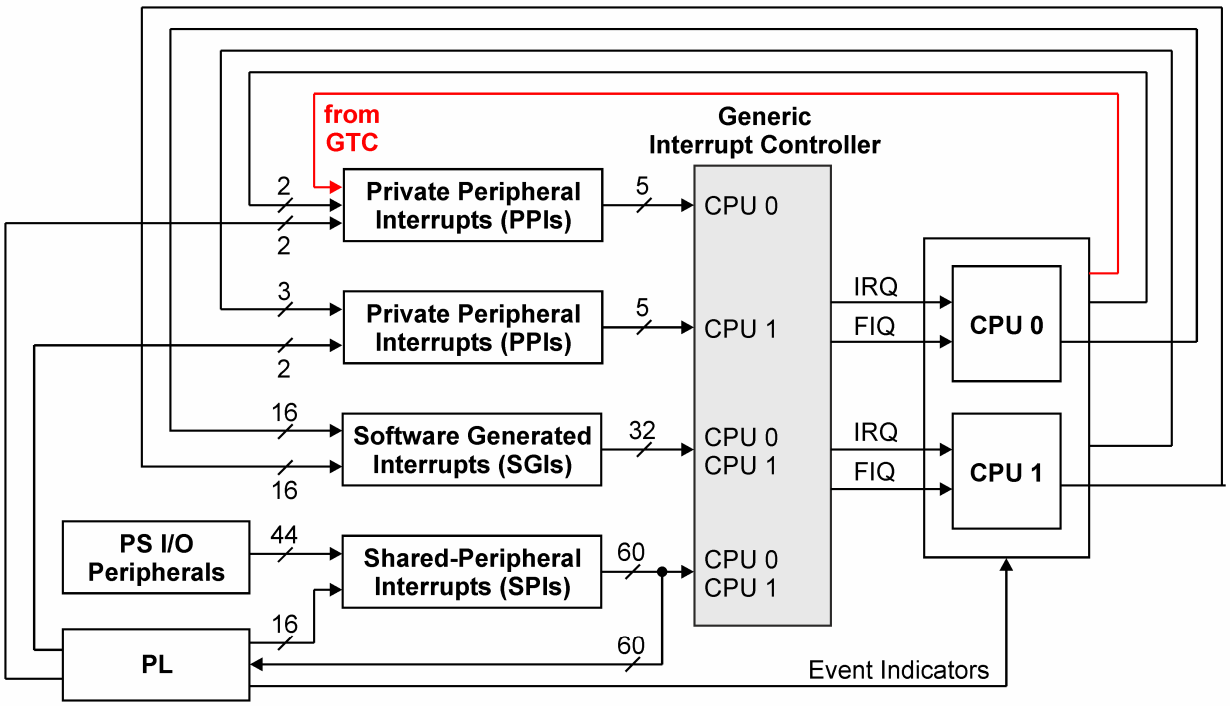
1. Što je domena takta, a što *domain crossing*? Nacrtati blokovsku shemu i opisati izvore takta sklopa Zynq. Nabrojiti njegove domene takta i navesti koji sklopovi im pripadaju. Što označavaju kratice 6:2:1 i 4:2:1?

9.4 Zadaci za samostalan rad

1. Pokrenuti konfiguraciju procesorskog sustava u Vivadu i proučiti dio koji se odnosi na konfiguraciju takta.

10 Prekidi

- podsjetimo se
  - poglavlje 3.2.1.4
    - razmatra prekidni kontroler
    - prekide s obzirom na njihov izvor
- poglavlje 10 (ovo poglavlje)
  - ilustrira konfiguriranje svih komponenata koje sudjeluju u posluživanju prekida
- primjer za analizu (*case study*)
  - generiranje zadanih vremenskih intervala pomoću prekida
  - intervale će generirati brojilo
- podsjetimo se (poglavlje "3.1 Osnovna blokovska shema")
  - PS sadrži 8 brojila
    - *watch-dog* (AWDT) i privatno brojilo u svakom procesoru
    - globalno brojilo (GTC)
    - dva trostruka brojila (TTC0 i TTC1)
    - sistemski *watch-dog* (SWDT)



10.1 Globalno brojilo

- *Global Timer Counter*, GTC

10.1.1 Sklopovlje

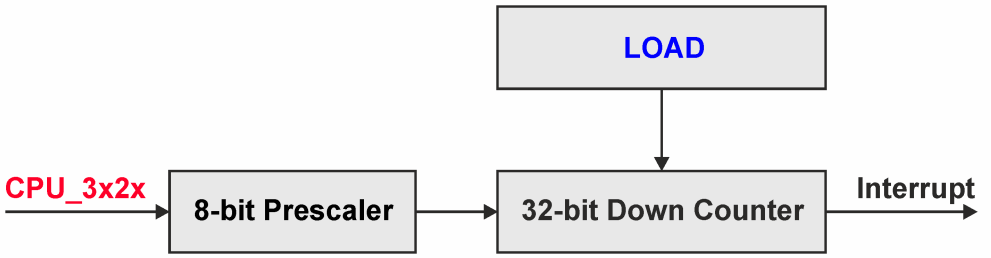
- jedno brojilo

  ⇒zajedničko za oba procesora

  - na nekim blokovskim shemama prikazano je kao dio SCU



- pojednostavljena shema globalnog brojila



- princip rada
  - brojilo broji prema dolje
  - generira zahtjev za prekid kad dosegne vrijednost 0
  - načini rada
    - *single-shot*

      ⇒izbroji do 0 i stane
    - *auto-reload*

      ⇒nakon što dosegne 0 automatski napuni iz registra LOAD i broji ponovo
- takt na kojem radi globalno brojilo

  CPU_3x2x

  ⇒1/2 frekvencije takta procesora
- period brojanja u *auto-reload* načinu rada

$$T = \frac{(PRESCALER_value + 1)(LOAD_value + 1)}{f_{CPU_3x2x}}$$

- napomena
  - na sličan način građena su o ostala brojila u sklopu Zynq

10.1.2 Upravljački program

- upravljački program za rad s GTC
  ⇒upravljački program je nazvan **ScuTimer**
  ⇒vanjski identifikatori počinju s **XScuTimer_**
  ⇒pripadajuće zaglavlje **xscutimer.h**

- podaci o jedinci upravljačkog programa (*driver instance data*)

```
typedef struct {
   XScuTimer_Config Config;  // Hardware Configuration
   u32 IsReady;              // Device is initialized and ready
   u32 IsStarted;            // Device timer is running
} XScuTimer;
```

- tip za opis konfiguracije sklopa (*configuration information for the device*)

```
typedef struct {
   u16 DeviceId;         // Unique ID of device
   u32 BaseAddr;         // Base address of the device
} XScuTimer_Config;
```

- standardna sučelja upravljačkog programa
  - pronalaženje tablice s predefiniranom konfiguracijom

```
XScuTimer_Config *XScuTimer_LookupConfig(u16 DeviceId);
```

  - konfiguriranje upravljačkog programa

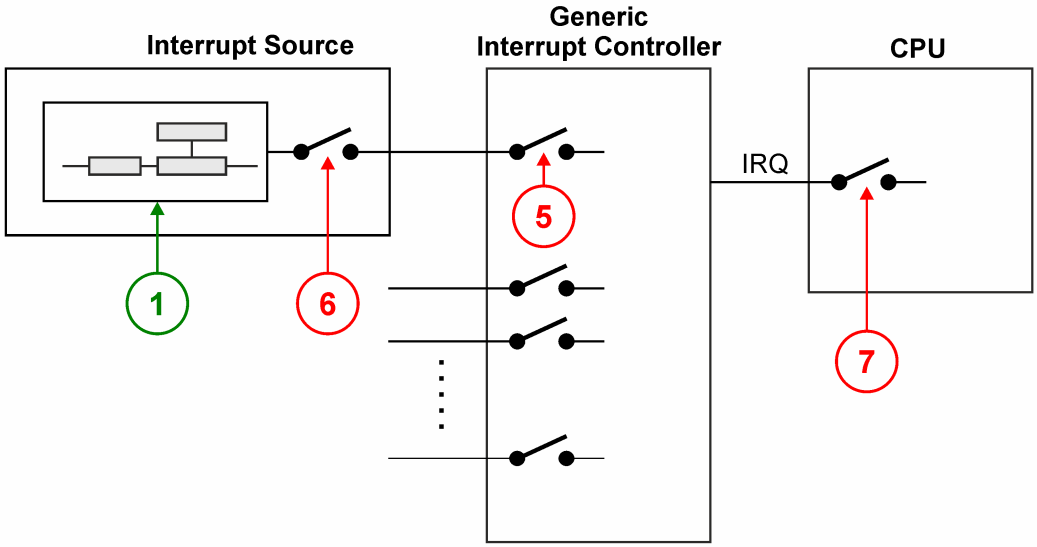
```
s32 XScuTimer_CfgInitialize(XScuTimer *InstancePtr,
                            XScuTimer_Config *ConfigPtr,
                            u32 EffectiveAddress)
```

- specifična sučelja upravljačkog programa

```
int XScuTimer_IsExpired(XScuTimer *InstancePtr);
                        // ispituje je li brojilo odbrojilo do 0
void XScuTimer_RestartTimer(XScuTimer *InstancePtr);
void XScuTimer_LoadTimer(XScuTimer *InstancePtr, u32 Value);
                        // upisuje vrijednost u registar LOAD
u32 XScuTimer_GetCounterValue(XScuTimer *InstancePtr);
void XScuTimer_EnableAutoReload(XScuTimer *InstancePtr);
void XScuTimer_DisableAutoReload(XScuTimer *InstancePtr);
void XScuTimer_EnableInterrupt(XScuTimer *InstancePtr);
void XScuTimer_DisableInterrupt(XScuTimer *InstancePtr);
void XScuTimer_GetInterruptStatus(XScuTimer *InstancePtr);
void XScuTimer_ClearInterruptStatus(XScuTimer *InstancePtr);
s32 XScuTimer_SelfTest(XScuTimer *InstancePtr); // Tests LOAD R/W
```

10.2 Konfiguriranje prekida

- prekidna struktura



- postupak inicijalizacije prekida

1. inicijalizirati jedinicu koja će tražiti prekide (izvor prekida)
   - koristi se **upravljački program dotične jedinice**
   – ovdje još ne omogućiti da jedinca zatraži prekid
2. inicijalizirati okruženje za posluživanje prekida
   - funkcija: `Xil_ExceptionInit()`
   – u ovoj verziji upravljačkih programa ova funkcija ne radi ništa
   – zadržana je zbog kompatibilnosti sa starijim verzijama
3. odabrati krovnu prekidnu funkcije i pridružiti ju prekidnom kontroleru
   - funkcija za pridruživanje: `Xil_ExceptionRegisterHandler()`
   - krovna prekidna funkcija: `XScuGic_InterruptHandler()`
4. pridružiti našu prekidnu funkciju željenom izvoru prekida
   - funkcija za pridruživanje: `XScuGic_Connect()`
   – naša prekidna funkcija bit će napisana kasnije
5. omogućiti prekid na razini prekidnog kontrolera
   - funkcija: `XScuGic_Enable()`
6. omogućiti da jedinica zatraži prekid
   - funkcija čije ime ima oblik `Xyyyyy_EnableInterrupt()`
     – yyyyy predstavlja naziv jedinice
7. omogućiti prekida na razini procesora
   - funkcija: `Xil_ExceptionEnable()`

- funkcije za posluživanje prekida
  - krovna prekidna funkcija
    - **XScuGic_InterruptHandler()**
      ⇒ovu funkciju daje Xilinx
    - poziva korisnikovu prekidnu funkciju
    - prosljeđuje joj **CallBackRef**
      ⇒**CallBackRef** je pokazivač na jedinicu koja je zatražila prekid
  - korisnikova (naša) prekidna funkcija
    - poslužuje željeni prekid
      ⇒treba ju napisati
- uočiti
  - kod konfiguriranja prekida koriste se pokretački programi za
    - upravljanje prekidima na razini procesora
    - upravljanje prekidnim kontrolerom

10.2.1 Upravljanje prekidima na razini procesora

- upravljački program konfiguriranje prekida na razini procesora
  ⇒nema poseban naziv
  ⇒pripadajuće zaglavlje **xil_exception.h**
- funkcije za konfiguriranje prekida na razini procesora

```
void Xil_ExceptionInit(void);

void Xil_ExceptionEnable(void);  // Enables the IRQ exception.

void Xil_ExceptionDisable(void); // Disables the IRQ exception.

void Xil_EnableNestedInterrupts(void);

void Xil_DisableNestedInterrupts(void);

extern void Xil_ExceptionRegisterHandler(u32 Exception_id,
                                         Xil_ExceptionHandler Handler,
                                         void *Data);

extern void Xil_ExceptionRemoveHandler(u32 Exception_id);

extern void Xil_GetExceptionRegisterHandler(u32 Exception_id,
                                        Xil_ExceptionHandler *Handler,
                                        void **Data);

extern void Xil_DataAbortHandler(void *CallBackRef);

extern void Xil_PrefetchAbortHandler(void *CallBackRef);

extern void Xil_UndefinedExceptionHandler(void *CallBackRef);
```

10.2.2 Upravljanje prekidnim kontrolerom

- upravljački program za rad s prekidnim kontrolerom
  - ⇒upravljački program je nazvan **ScuGic**
  - ⇒vanjski identifikatori počinju s **XScuGic_**
  - ⇒pripadajuće zaglavlje **xscugic.h**
- podaci o jedinci upravljačkog programa (*driver instance data*)

```
typedef struct{
   XScuGic_Config *Config;   // Configuration table entry
   u32 IsReady;              // Device is initialized and ready
   u32 UnhandledInterrupts;  // Intc Statistics
} XScuGic;
```

- tip za opis konfiguracije sklopa (*configuration information for the device*)

```
typedef struct{
   u16 DeviceId;           // Unique ID of device
   u32 CpuBaseAddress;     // CPU Interface Register base address
   u32 DistBaseAddress;    // Distributor Register base address
   XScuGic_VectorTableEntry HandlerTable[XSCUGIC_MAX_NUM_INTR_INPUTS];
                           // Vector table of interrupt handlers
} XScuGic_Config;
```

- standardna sučelja upravljačkog programa
  - pronalaženje tablice s predefiniranom konfiguracijom

```
XScuGic_Config *XScuGic_LookupConfig(u16 DeviceId);
```

  - konfiguriranje upravljačkog programa

```
s32  XScuGic_CfgInitialize(XScuGic *InstancePtr,
                           XScuGic_Config *ConfigPtr,
                           u32 EffectiveAddr);
```

- specifična sučelja upravljačkog programa
  - češće korištena sučelja

```
s32  XScuGic_Connect(XScuGic *InstancePtr, u32 Int_Id,
                     Xil_InterruptHandler Handler,
                     void *CallBackRef);

void XScuGic_Disconnect(XScuGic *InstancePtr, u32 Int_Id);

void XScuGic_Enable(XScuGic *InstancePtr, u32 Int_Id);

void XScuGic_Disable(XScuGic *InstancePtr, u32 Int_Id);
```

```
s32  XScuGic_SoftwareIntr(XScuGic *InstancePtr,
                          u32 Int_Id,
                          u32 Cpu_Id);

void XScuGic_GetPriorityTriggerType(XScuGic *InstancePtr,
                                    u32 Int_Id,
                                    u8 *Priority,
                                    u8 *Trigger);

void XScuGic_SetPriorityTriggerType(XScuGic *InstancePtr,
                                    u32 Int_Id,
                                    u8 Priority,
                                    u8 Trigger);

void XScuGic_InterruptMaptoCpu(XScuGic *InstancePtr,
                               u8 Cpu_Id,
                               u32 Int_Id);

void XScuGic_InterruptUnmapFromCpu(XScuGic *InstancePtr,
                                   u8 Cpu_Id,
                                   u32 Int_Id);

void XScuGic_Stop(XScuGic *InstancePtr);
        ⋮
```

10.3 Primjer prekida globalnog brojila

Primjer

U računalnom sustavu koji sadrži sklop Zynq potrebno je konfigurirati globalno brojilo tako da daje zahtjev za prekid u intervalima od 1 sekunde. Nadalje, konfigurirati sklopovlje za upravljanje prekidima te napisati program za posluživanje prekida ovog brojila.

U glavnom programu konfigurirati serijsko sučelje tako da radi s brzinom prijenosa od 9600 bit/s te u petlji ispisivati poruke oblika "Proteklo je x sekundi.", pri čemu x označava proteklo vrijeme.

Uhodati sklopovlje i programsku podršku na razvojnom sustava ZedBoard. Pretpostaviti da procesor radi na frekvenciji takta od 666,667 MHz.

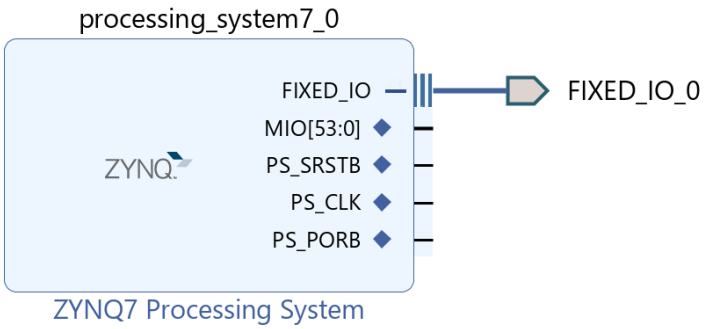
Rješenje

- sklopovlje
  - spoj serijskog sučelja ⇒ isto kao u poglavlju "7.4 *Hello World!*"
  - globalno brojilo ⇒ potrebno konfigurirati
- programska podrška mora konfigurirati
  - UART ⇒ isto kao u poglavlju "7.4 *Hello World!*"
  - GTC, GIC, prekide na razini procesora ⇒ potrebno konfigurirati

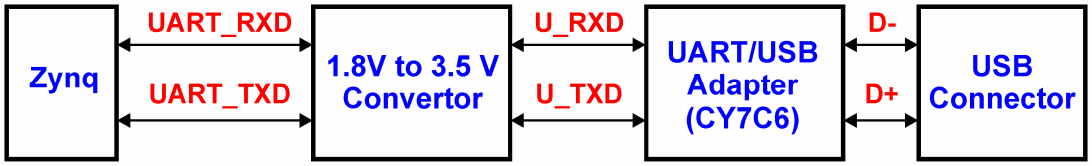
10.3.1 Sklopovlje

10.3.1.1 Procesorski sustav i vanjsko serijsko sklopovlje

- procesorski sustav
  - vidi poglavlje "7.4.1.2 Procesorski sustav"

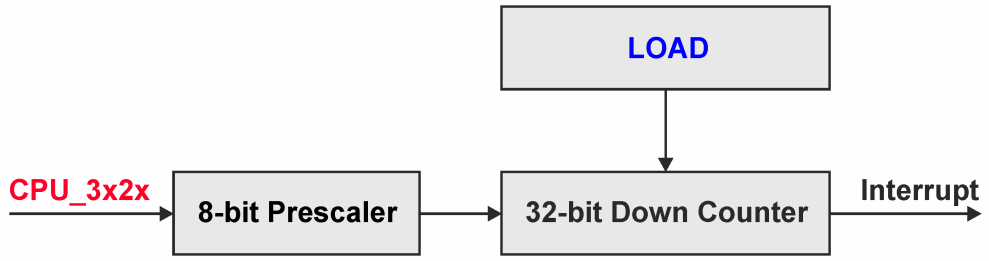


- vanjsko serijsko sklopovlje (za ZedBoard)
  - vidi poglavlju "7.4.1.1 Asinkrono serijsko sučelje"



10.3.1.2 Globalno brojilo

- izračun vrijednosti za LOAD



- period brojanja u *auto-reload* načinu rada

$$T = \frac{(PRESCALER_value + 1)(LOAD_value + 1)}{f_{CPU_3x2x}}$$

$\Rightarrow$ potrebno je odrediti *PRESCALER_vaule* i *LOAD_value*

- primjer izračuna za ovaj slučaj

$$LOAD_value = \frac{T \times f_{CPU_3x2x}}{PRESCALER_value + 1} - 1$$

- takt na kojem radi globalno brojilo $\Rightarrow$ 1/2 frekvencije takta procesora

$$f_{CPUmax} = f_{CPU_6x4x} = 666.667 \text{ MHz}$$

$$f_{CPU_3x2x} = f_{CPU_6x4x} / 2 = 333.3335 \text{ MHz}$$

- primjer izračuna

$$LOAD_value = \frac{T \times f_{CPU_3x2x}}{PRESCALER_value + 1} - 1 = \frac{1 \times 333.3335e6}{0 + 1} - 1 = 0x13DE43FB$$

$PRESCALER_value = 0 \Rightarrow LOAD_value \leq 0xFFFFFFFF$ OK

- ako bi bilo $LOAD_value > 0xFFFFFFFF$
  $\Rightarrow$ povećati $PRESCALER_value$

10.3.2 Programska podrška

```
#include "xparameters.h"
#include "xuartps.h"

#include "xscutimer.h"                          // <--- INTERRUPTS
#include "xscugic.h"                            // <--- INTERRUPTS
#include "xil_exception.h"                      // <--- INTERRUPTS

#include "xil_printf.h"

#define TIMER_LOAD_VALUE   0x13DE43FB           // <--- INTERRUPTS

static void GtcIntHandler(void *CallBackRef);

XUartPs   Uart_Ps; // Driver instance for UART
XScuTimer Gtc;     // Driver instance for GTC   // <--- INTERRUPTS
XScuGic   Gic;     // Driver instance for GIC   // <--- INTERRUPTS

volatile unsigned int sTicks=0;                 // <--- INTERRUPTS


// ====================== Main ========================

int main() {
   int Status;
```

```
// ====================== UART ========================

XUartPs_Config *ConfigUart;

ConfigUart = XUartPs_LookupConfig(XPAR_XUARTPS_0_DEVICE_ID);
if (NULL == ConfigUart) {return XST_FAILURE;}

Status = XUartPs_CfgInitialize(&Uart_Ps,
                               ConfigUart,
                               ConfigUart->BaseAddress);
if (Status != XST_SUCCESS) {return XST_FAILURE;}

XUartPs_SetBaudRate (&Uart_Ps, 9600);


// ================== Global Timer ====================

XScuTimer_Config *ConfigGtc;

ConfigGtc = XScuTimer_LookupConfig(XPAR_XSCUTIMER_0_DEVICE_ID);
if (NULL == ConfigGtc) {return XST_FAILURE;}

Status = XScuTimer_CfgInitialize(&Gtc,
                                 ConfigGtc,
                                 ConfigGtc->BaseAddr);
if (Status != XST_SUCCESS) {return XST_FAILURE;}

Status = XScuTimer_SelfTest(&Gtc);
if (Status != XST_SUCCESS) {return XST_FAILURE;}

XScuTimer_EnableAutoReload(&Gtc);                       // <1>
XScuTimer_LoadTimer(&Gtc, TIMER_LOAD_VALUE);            // <1>
XScuTimer_Start(&Gtc);                                  // <1>
// XScuTimer_EnableInterrupt(&Gtc);  // Not now! Later!


// ====================== GIC =========================

XScuGic_Config *ConfigGic;

ConfigGic = XScuGic_LookupConfig(XPAR_SCUGIC_SINGLE_DEVICE_ID);
if (NULL == ConfigGic) {return XST_FAILURE;}

Status = XScuGic_CfgInitialize(&Gic,
                               ConfigGic,
                               ConfigGic->CpuBaseAddress);
if (Status != XST_SUCCESS) {return XST_FAILURE;}
```

```
    // ================= Exception System ==================

    Xil_ExceptionInit();                                        // <2>

    Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_IRQ_INT,
                        (Xil_ExceptionHandler)XScuGic_InterruptHandler,
                        &Gic);                                  // <3>

    Status = XScuGic_Connect(&Gic, XPAR_SCUTIMER_INTR,
                             (Xil_ExceptionHandler)GtcIntHandler,
                             &Gtc);                             // <4>
    if (Status != XST_SUCCESS) {return Status;}

    XScuGic_Enable(&Gic, XPAR_SCUTIMER_INTR);                   // <5>

    XScuTimer_EnableInterrupt(&Gtc);   // Now is the moment! // <6>

    Xil_ExceptionEnable();                                      // <7>

    // ==================== Main job ======================

    unsigned int Sec=0;
    while(1) {
       if (Sec<sTicks) {
           Sec=sTicks;
           xil_printf ("Elapsed time: %5d s\n\r", Sec);
       }
    }

    return 0;
}

// ================== Interrupt handler ==================

static void GtcIntHandler(void *CallBackRef) {

    XScuTimer *LocalGtc = (XScuTimer *) CallBackRef;

    sTicks++;

    XScuTimer_ClearInterruptStatus(LocalGtc);

}
```

- uočiti
  - s obzirom da se `XScuTimer *LocalGtc` generira lokalno u prekidnoj funkciji, deklaracija `XScuTimer Gtc` ne mora biti globalna

10.4 Pitanja za provjeru znanja

1. Opisati po koracima postupak inicijalizacije prekida u slučaju kad programska podrška koristi upravljačke programe iz Standalone operacijskog sustava.

10.5 Zadaci za samostalan rad

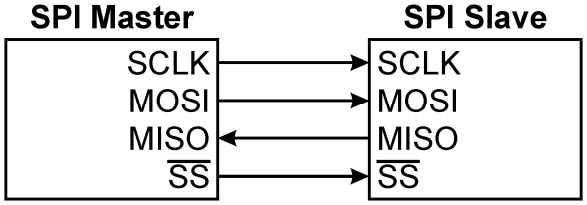
1. Napraviti primjer opisan u poglavlju "10.3 Primjer prekida globalnog brojila" u pomoću alata Vivado/Vitis te ga pripremiti za ispitivanje na razvojnom sustavu ZedBoard.

11 Periferno sklopovlje u SoC Zynq

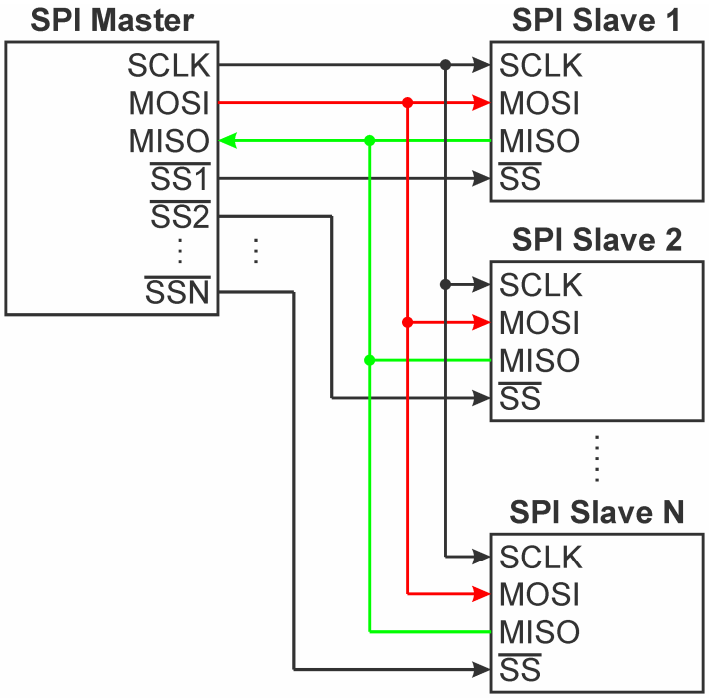
- sklop Zynq sadrži razne ugrađene periferije (vidi poglavlje 3.1)
- dosad su razmatrani
  - GPIO
  - UART
- u daljnjem tekstu bit će ukratko opisani
  - SPI
  - I^2C
  - USB
- naglasak će biti stavljen na sklopovlje i princip rada

11.1 Serial Peripheral Interface

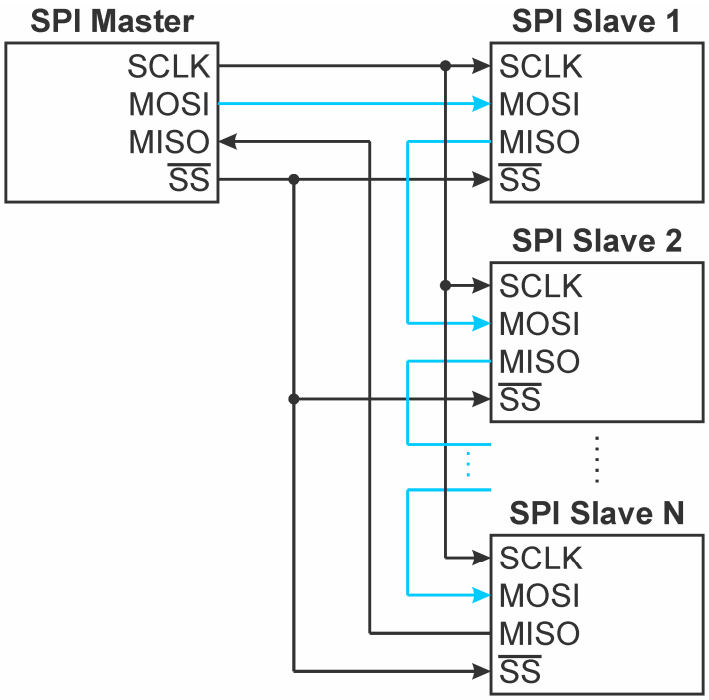
- *Serial Peripheral Interface*, SPI
- SPI sabirnicu uvela je kompanija Motorola
- karakteristike SPI sabirnice
  - serijska (*serial*)
  - sinkrona (*synchronous*)
  - dvosmjerna (*full duplex*)
- brzina prijenosa
  $\Rightarrow$ do 100 MBit/s
- signali
  - Serial Clock, **SCLK**
  - Master Output Slave Input, **MOSI**
  - Master Input Slave Output, **MISO**
  - Slave Select, **SS**
- spoj nadređene i podređene jedinice (*master*, *slave*)
  - jedna podređena jedinica



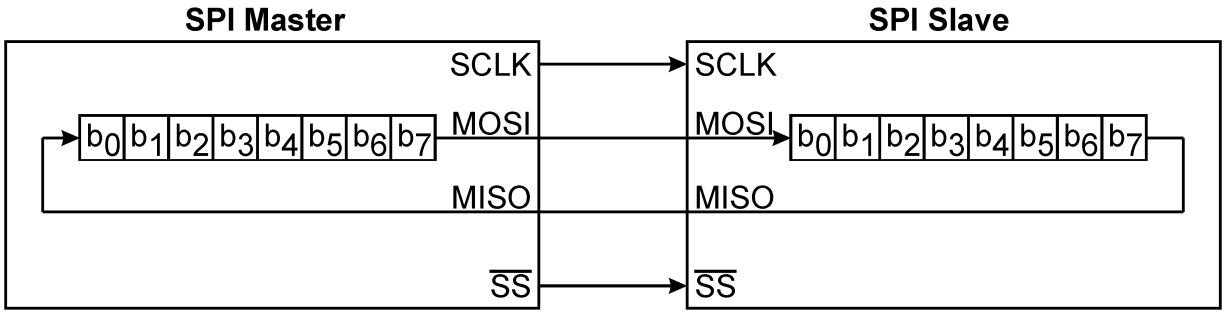
- više od jedne podređene jedinice - paralelan spoj



- više od jedne podređene jedinice - spoj u lanac



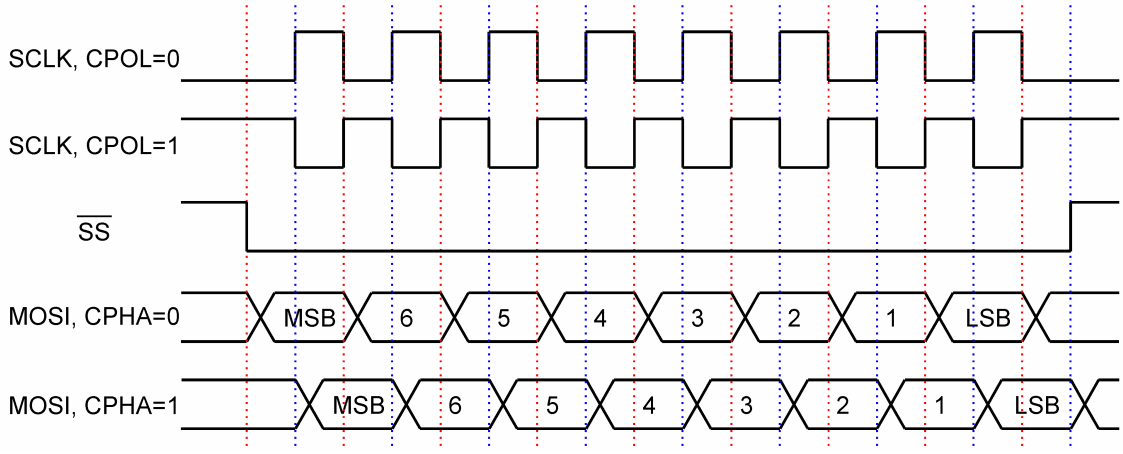
- način prijenosa podataka



  - odašiljački i prijemni registar zajedno čine posmačni registar
  - komunikacija je uvijek dvosmjerna
  - prenosi se 8, 12, 16 ili neki drugi dogovoreni broj bitova
  - nadređena jedinica
    - inicira komunikaciju, generira takt i emitira podatke
  - podređena jedinica
    - emitira podatke

- uočiti
  - ako je potrebna jednosmjerna komunikacija
    $\Rightarrow$ podatak koji je došao u drugom smjeru se zanemaruje
  - adresiranje izvršioca izvedeno je pomoću linije $\overline{SS}$
  - adresiranje nije dio protokola
    $\Rightarrow$ komunikacija je vrlo učinkovita

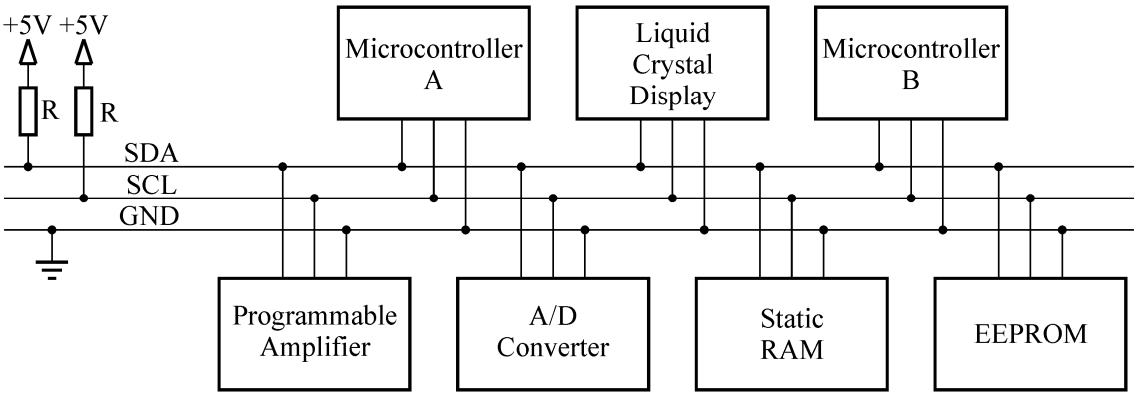
- signali
  - prijenos podatka sinkron je s taktom
  - moguće su 4 kombinacije polariteta i faze takta



- primjena SPI sabirnice
    - komunikacija sa serijskim vanjskim memorijama
        - npr. flash memorije s SD i SDIO sučeljem
        - *boot* memorija na Zynq-u (Quad SPI)
    - komunikacija između dvaju procesora
    - sučelja LCD pokaznika
    - upis u programsku memoriju nekih mikrokontrolera (npr. CC2510)

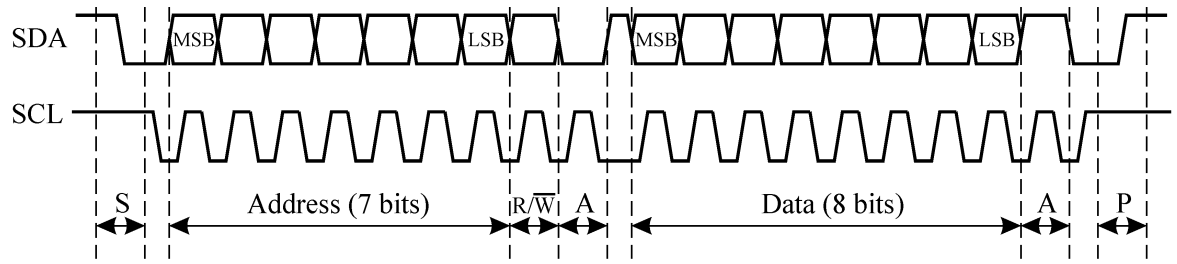
11.2 I2C

- Inter IC => IIC => I^2C
- I^2C sabirnicu uvela je kompanija Philips Semiconductors
- zamišljena za povezivanje podsustava u raznim uređajima
- karakteristike I^2C sabirnice
    - serijska, sinkrona, dvosmjerna
- brzina prijenosa
    - standardni način rada (*standard mode*) ⇒ do 100 kbit/s
    - brzi način rada (*fast mode*) ⇒ do 400 kbit/s
    - brzi način rada plus (*fast mode plus*) ⇒ do 1 Mbit/s
    - vrlo brzi način rada (*high speed mode*) ⇒ do 3.4 Mbit/s
- signali
    - Serial Data, **SDA**
    - Serial Clock, **SCL**
- spajanje komponenata na I^2C sabirnicu



⇒spajanje ne traži dodatnu logiku (*glue logic*)

- signali na sabirnici



  - uočiti
    - prijenosi se po 8 bitova
      - start bit (S)
      - stop bit (P)
      - 7 bitna adresa jedinice
        (postoji i način s 10-bitnom adresom)
      - *read/write*, R/nW
      - potvrda (*acknowledge*, A)
    - nadređena jedinica generira takt i upravlja prijenosom
      - postavlja SDA u 0 (S), a zatim postavlja CLK u 0
      - postavlja MSB poruke i generira jedan puls takta
        - ponavlja to za svaki od 8 bitova poruke
      - oslobađa sabirnicu i čeka da podređena jedinica potvrdi da je primila poruku (A)
      - itd., ponavlja slanje za drugi bajt ili više bajtova
      - na kraju postavlja SCL u 1 (P), a zatim SDA u 1

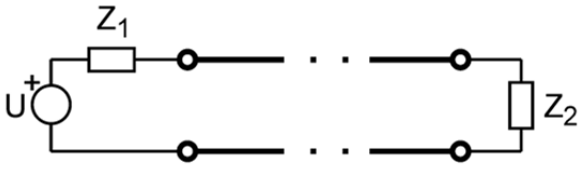
- primjena I^2C sabirnice
  - pretvornici
    - primjer: I^2C mjerila temperature, vlage, akceleracije, ADC itd.
  - komponente za proširenje GPIO
    - primjer: I^2C registri
  - mnoge složene komponente imaju I^2C sučelje za konfiguriranje
    - programski definirani prijamnici
    - sintezatori frekvencija
  - itd.

11.3 USB

- Universal Serial Bus (USB)

11.3.1 Prijenos signala linijom

- podsjetimo se linija
  - vidi predmet Električni krugovi
- prijenos signala linijom



- parametri linije
  - primarni R [Ω/m], G [S/m], L [H/m], C [F/m]
  - sekundarni
    - zrcalna impedancija

$$Z_0 = \sqrt{\frac{R + sL}{G + sC}}$$

      - Z_0 je ulazna impedancija beskonačno duge linije
    - zrcalna konstanta prijenosa

$$\gamma = \sqrt{(R + sL)(G + sC)}$$

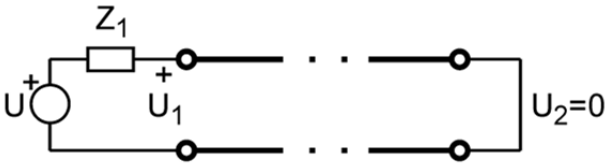
- posebni slučajevi
  - linija bez gubitaka $\Rightarrow$ R = 0, G = 0
  - linija bez izobličenja $\Rightarrow \frac{R}{L} = \frac{G}{C}$
  - u oba slučaja Z_0 je realan i iznosi

$$Z_0 = \sqrt{\frac{L}{C}}$$

- propagacija pulsa duž linije
  - beskonačno duga linija
    $\Rightarrow$ impuls doveden na ulaz linije propagira duž linije
  - linija konačne duljine
    - $Z_2 = Z_0$ (linija zaključena zrcalnom impedancijom)
    - obzirom da je Z_0 ulazna impedancija beskonačno duge linije, linija koja je zaključena zrcalnom impedancijom s ulaza se električki "vidi" kao beskonačna
      $\Rightarrow$ impuls doveden na ulaz linije propagira do kraja linije
    - $Z_2 \neq Z_0$ (linija nije zaključena zrcalnom impedancijom)
      $\Rightarrow$ dolazi do refleksije na diskontinuitetu impedancije
      - faktor refleksije

$$\Gamma = \frac{Z_2 - Z_0}{Z_2 + Z_0}$$

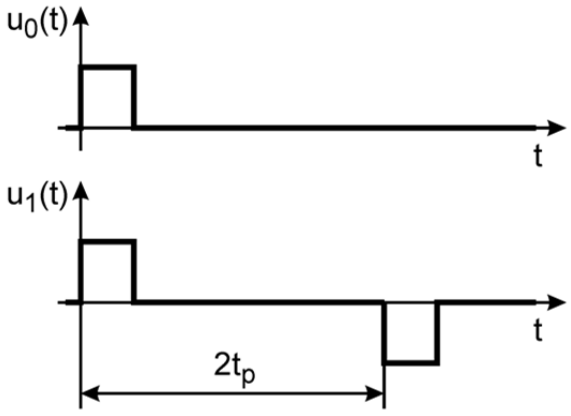
- primjer refleksije na liniji
  - pretpostavimo da je $Z_1 = Z_0$, a $Z_2 = 0$



  - faktor refleksije

$$\Gamma = \frac{Z_2 - Z_0}{Z_2 + Z_0} = \frac{0 - Z_0}{0 + Z_0} = -1$$

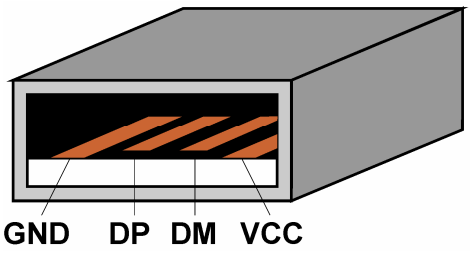
  - primjer valnih oblika signala na liniji



t_p je vrijeme propagacije signala od početka do kraja linije

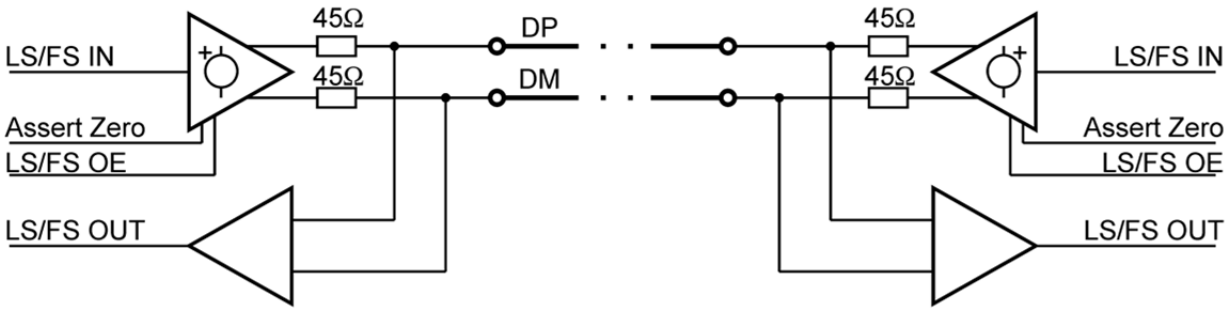
11.3.2 Sklopovlje USB sučelja

- postojeći standardi
  - USB 1.0 ⇒ izlazi iz upotrebe
    - teoretska brzina prijenosa 1.5 MBit/s
      ⇒ *low speed* (LS)
  - USB 1.1 ⇒ izlazi iz upotrebe
    - teoretska brzina prijenosa 12 MBit/s
      ⇒ *full speed* (FS)
  - USB 2.0 ⇒ trenutno je u upotrebi
    - teoretska brzina prijenosa 480 MBit/s
      ⇒ *hi speed* (HS)
  - USB 3.0 ⇒ trenutno je u upotrebi
    - teoretska brzina prijenosa 4.8 GBit/s
      ⇒ *super speed* (SS)
- signali USB priključka - primjer za konektor tipa "A", USB 1.0, 1.1 i 2.0
  - priključci za napajanje
    - masa, GND
    - napajanje, VCC
  - priključci simetrične podatkovne linije
    - DP ili D+
    - DM ili D-

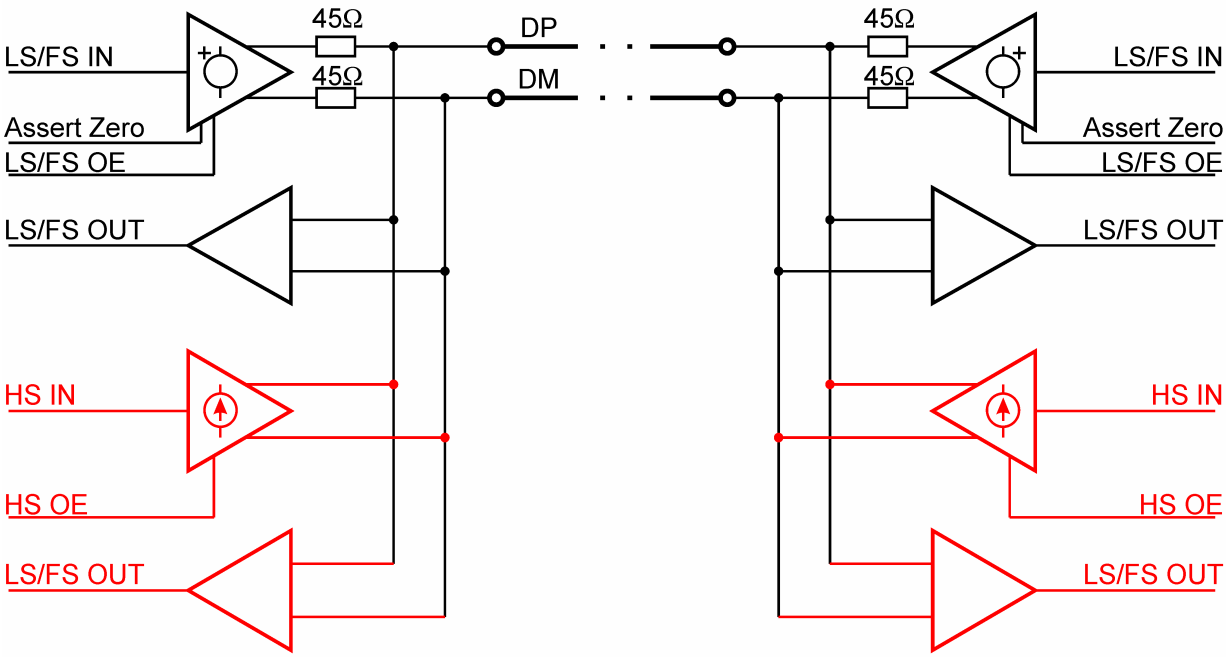


  - uočiti
    - priključci GND i VCC su dulji
      ⇒ kod uključivanja prvo se dovodi napajanje
- prijenos
  smjer se mijenja (*half duplex*) prema protokolu
- zrcalna impedancija linije koja se koristi za USB prijenos iznosi
  45 Ω
- diferencijalni napon na liniji
  ± 400 mV

- pogonsko sklopovlje koje odgovara standardu USB 2.0 mora podržavati
  low speed, *full speed* i *high speed*
- sklopovlje potrebno za *low speed* i *full speed* komunikaciju



  - uočiti
    - linija je zaključena zrcalnom impedancijom na oba kraja
    - ovo sklopovlje podržava standarde USB 1.0 i USB 1.1
- dodatno sklopovlje potrebno za *high speed* komunikaciju

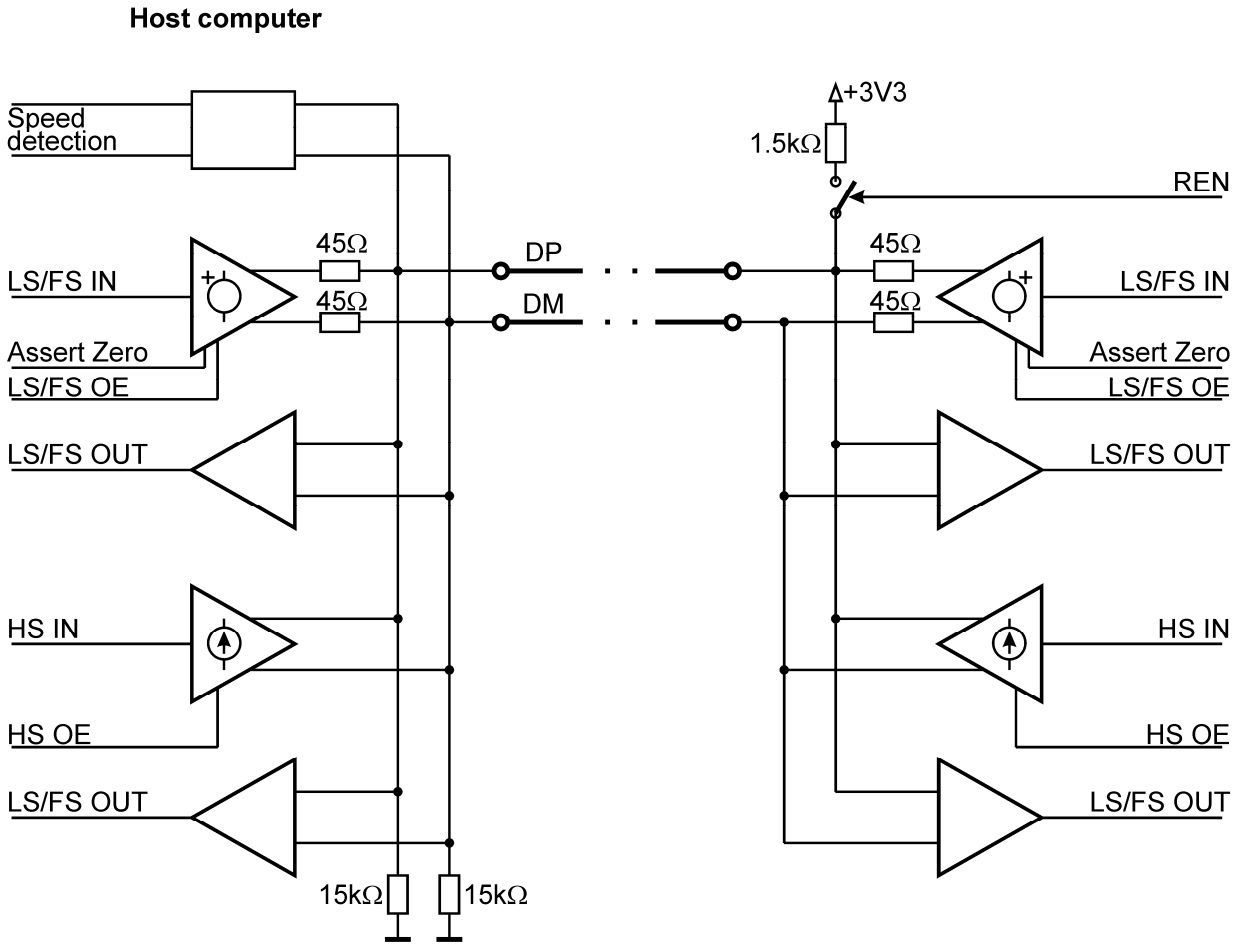


  - *high speed* komunikacija izvodi se na sljedeći način
    - obje linije u LS/FS pojačalu daju napon od 0 V
      (signal *Assert Zero*)
      ⇒ linije su i dalje zaključene impedancijom od 45 Ω

    ⇒ pojačalo koje se koristi za *high speed* mora imati strujni izlaz
  - uočiti
    - kad je isključeno, sklopovlje za HS komunikaciju ne narušava rad sklopovlja za LS i FS komunikaciju

- brzine prijenosa određuju se
    - za *low speed* i za *full speed*
        $\Rightarrow$ sklopovskom identifikacijom
    - za *high speed*
        - prvo se sklopovski identificira FS
        - korištenjem protokola (pregovaranje) jedinice dogovore brzinu za HS komunikaciju
- sklopovska identifikacija brzine prijenosa
    - jedinica koja se priključuje na USB ima pritezni otpornik od 1.5 kΩ spojen prema napajanju
        - *low speed* $\Rightarrow$ pritezni otpornih je na liniji DM
        - *full speed* $\Rightarrow$ pritezni otpornih je na liniji DP
    - komunikacija uvijek (kod USB 2.0) počinje kao *full speed*
        $\Rightarrow$ postoji pritezni otpornik na liniji DP
    - kod prijelaza na *high speed*, ovaj otpornik potrebno je isključiti
        $\Rightarrow$ potrebna je sklopka
- definiranje razine za slučaj kad jedinica nije spojena
    - kad jedinica nije spojena, aktivni su prijemni sklopovi
    - da bi bila definirana razina na njihovim ulazima, potrebno je spojiti sa DP i DM prema masi otpornike iznosa od oko 15 kΩ

- pojednostavljena blokovska shema cjelokupnog sklopovlja



- računalo domaćin sadrži i izvor napajanja, VCC
  - najveća struja koju izvor može dati iznosi
    - nakon što je obavljena identifikacija
      - tipično 500 mA
    - prije nego što je obavljena identifikacija (u trenutku uključenja USB jedinice)
      - tipično 50mA

11.4 Pitanja za provjeru znanja

1. Nacrtati spoj dviju jedinica preko SPI sabirnice te opisati ulogu signala na sabirnici. Nacrtati i opisati načine spajanja nadređene jedinice s više podređenih jedinica. Nacrtati i opisati način prijenosa podataka između sklopova povezanih SPI sabirnicom. Navesti primjene SPI sabirnice.
2. Nacrtati i opisati način spajanja sklopova na I^2C sabirnicu. Nacrtati i opisati valne oblike kod prijenosa dvaju bajtova preko I^2C sabirnice. U čemu se razlikuju načini rada I^2C sabirnice? Navesti primjene I^2C sabirnice.
3. Navesti izraz za zrcalnu impedanciju linije. Koji oblik taj izraz poprima za slučaj linije bez gubitaka? Koje je fizikalno značenje zrcalne impedancije? Objasniti pojavu refleksije na liniji. Kako se ona izbjegava? Što je faktor refleksije?
4. Nacrtati sklopovsku izvedbu USB sučelja. Objasniti ulogu pojedinih dijelova u *low/full speed* odnosno *high speed* načinu rada. Kako se određuje način rada u kojem se odvija komunikacija.

12 Višeprocesorski sustav

12.1 Način korištenja višejezgrenih procesora u URS-ovima

- isti vrijedi i za sustave koji imaju više samostojećih procesore
- načini korištenja jezgri (primjer Zynq-7000) (*processing models*)
    - simetričnog višeprocesorski sustav (*symmetric multiprocessing*, SMP)
    - asimetričnog višeprocesorski sustav (*asymmetric multiprocessing*, ASM)

12.1.1 Simetrični višeprocesorski sustavi

- isti (jedan) operacijski sustav koristi se na oba procesora
- operacijski sustav automatski koristi oba procesora
- programer može
    - dodijeliti procese pojedinim procesorima
    - dodijeliti jednom od procesora da bude upravljač
        - upravljač radi inicijalizaciju
            - sustava u cjelini i drugog procesora
- primjer
    - osobno računalo
    - Linux na Zynq-7000
        - može koristiti više procesorskih jezgri

12.1.2 Asimetrični višeprocesorski sustavi

- često susrećemo u URS-ovima
- svaki procesor izvršava svoj OS
    - isti OS može se koristiti na svim procesorima (rjeđe)
    - različite jezgre koriste različite OS (češće)

- primjer asimetričnog višeprocesorski sustav (dvije jezgre)
    - jedna jezgra koristi Linux
        $\Rightarrow$ cjelovi OS i nudi niz mogućnosti (mreža, aplikacije itd.)
    - druga jezgra koristi RTOS ili *bare machine*
        $\Rightarrow$ traži manje memorije
        $\Rightarrow$ pogodan za operacije koje rade u stvarnom vremenu

- uočiti
    - različiti OS se međusobno nadopunjuju jer svaki ima dobra svojstva za neki od zadataka koje sustav obavlja

- komunikacija između procesora
    - preko prekida (*inter-processor interrupt*)
    - preko zajedničke memorije (*shared memory*)
    - prosljeđivanjem poruka (*message passing*)

- pristup procesora ostalom sklopovlju (primjer Zynq -7000)
    - fizička memorija je zajednička
    - većina periferija pridružuje je po jednom procesoru
    - prekidni kontroler dijele oba procesora
        - jedan procesor se odabire kao upravljač prekidnog kontrolera (*interrupt controller master*)

12.2 Pitanja za provjeru znanja

1. Opisati razliku između simetričnog i asimetričnog višeprocesorskog sustava. Kakva programska podrška se izvršava na pojedinim procesorima u oba slučaja? Kako se kod asimetričnog višeprocesorskog sustava izvodi komunikacija između procesora? Kako procesori na Zynqu pristupaju memoriji, periferijama i prekidnom kontroleru?

13 Završne napomene

- izneseno gradivo obuhvatilo je
    - arhitekturu programabilnih SoC platformi
    - razvoj sklopovlja i programske podrške ugradbenih računalnih sustava u SoC okruženju
    - alate za razvoj računalnih sustava temeljenih na SoC platformama
- smjernice za daljnji rad
    - razvoj digitalnog sklopovlja koje se implementira u programabilnoj logici
    - rad s brzim periferijama
    - razvoj programske podrške za rad u stvarnom vremenu
    - rad u višeprocesorskom okruženju
    - primjena operacijskih sustava

14 Literatura

- arhitektura ARMv7-A

  [1] *ARM Architecture Reference Manual - ARMv7-A and ARMv7-R edition, Revision: C.d, 29 March 2018*

- procesorska jezgra ARM Cortex-A9

  [2] *Cortex-A9 Technical Reference Manual, Revision: r4p1, 15 June 2012*

- višeprocesorska jezgra ARM Cortex-A9MPCore

  [3] *Cortex-A9 MPCore Technical Reference Manual, Revision: r4p1, 15 June 2012*

- Zynq-7000

  [4] *Zynq-7000 SoC Data Sheet: Overview*, DS190 (v1.11.1), July 2, 2018

  [5] *Zynq-7000 SoC Technical Reference Manual*, UG585 (v1.13), April 2, 2021

- programska podrška

  [6] *Zynq-7000 SoC Software Developers Guide, UG821 (v12.0), September 30, 2015*

- razvojni sustav ZedBoard

  [7] *ZedBoard (Zynq Evaluation and Development) Hardware User's Guide, Version 2.2, 27 January 2014*

- alati za razvoj sklopovlja

  [8] *Vivado Design Suite User Guide - Creating and Packaging Custom IP*, UG1118 (v2020.2) January 22, 2021

  [9] *Vivado Design Suite User Guide - Design Flows Overview, UG892 (v2020.2) February 12, 2021*

  [10] *Vivado Design Suite User Guide - Using the Vivado IDE*, UG893 (v2020.2) January 28, 2021

- alati za razvoj programske podrške

  [11] *Vitis Unified Software Platform Documentation - Embedded Software Development, UG1400 (v2020.2) December 15, 2020*